



MACHINE LEARNING CON PYTHON

TEMA 8: Deep Learning y Modelos Generativos



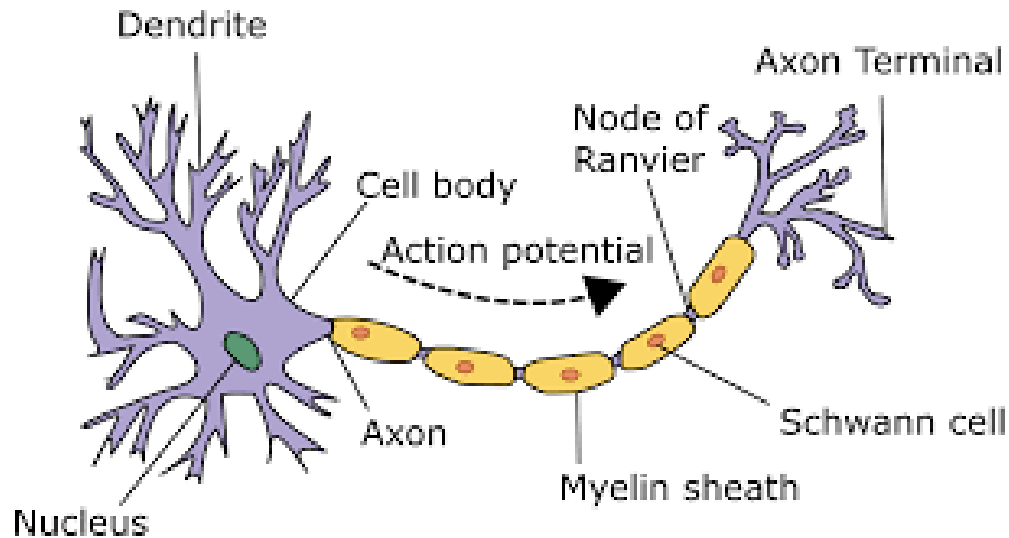
Contenidos Tema 8

- / 1. Fundamentos de redes neuronales, deep learning, y Tensorflow**
- / 2. Arquitecturas de Deep Learning y Generativas: teoría y ejemplos**
- / 3. Procesamiento de Lenguaje Natural (NLP) y Modelos de Lenguaje (LLMs)**

/ BIBLIOGRAFÍA DEL CURSO

Deep Learning: Redes Neuronales Artificiales

- Introducidas en el artículo científico de 1943 (McCulloch and Pitts): "*a logical calculus of ideas immanent in nervous activity*"
- Estudio basado en lógica proposicional sobre cómo funcionan las neuronas biológicas en el cerebro animal.



BULLETIN OF
MATHEMATICAL BIOPHYSICS
VOLUME 5, 1943

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH AND WALTER PITTS

FROM THE UNIVERSITY OF ILLINOIS, COLLEGE OF MEDICINE,
DEPARTMENT OF PSYCHIATRY AT THE ILLINOIS NEUROPSYCHIATRIC INSTITUTE,
AND THE UNIVERSITY OF CHICAGO

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes.

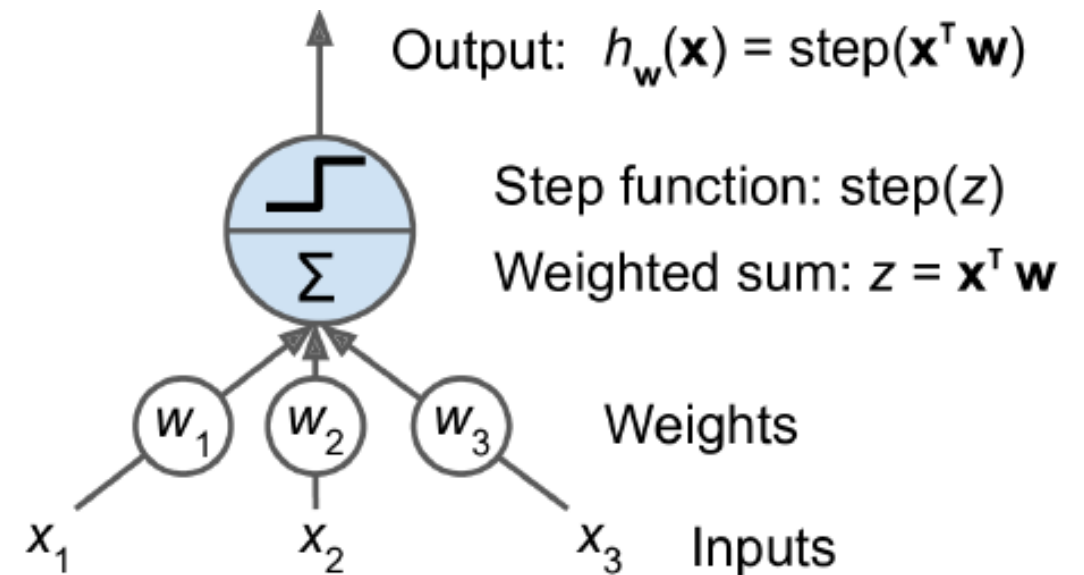
Deep Learning: Redes Neuronales Artificiales

1957: basado en un tipo especial de neuronas artificiales llamadas *Threshold Logic Unit* (TLU).

- Cada entrada numérica de información, $x = x_1, x_2, \dots, x_n$, lleva asociado un peso.
- El objetivo es *aprender* estos pesos.

Una **neurona TLU**:

- Calcula la suma ponderada de las entradas.
- Aplica una *función de paso* sobre esa suma.
- Devuelve el resultado como salida.

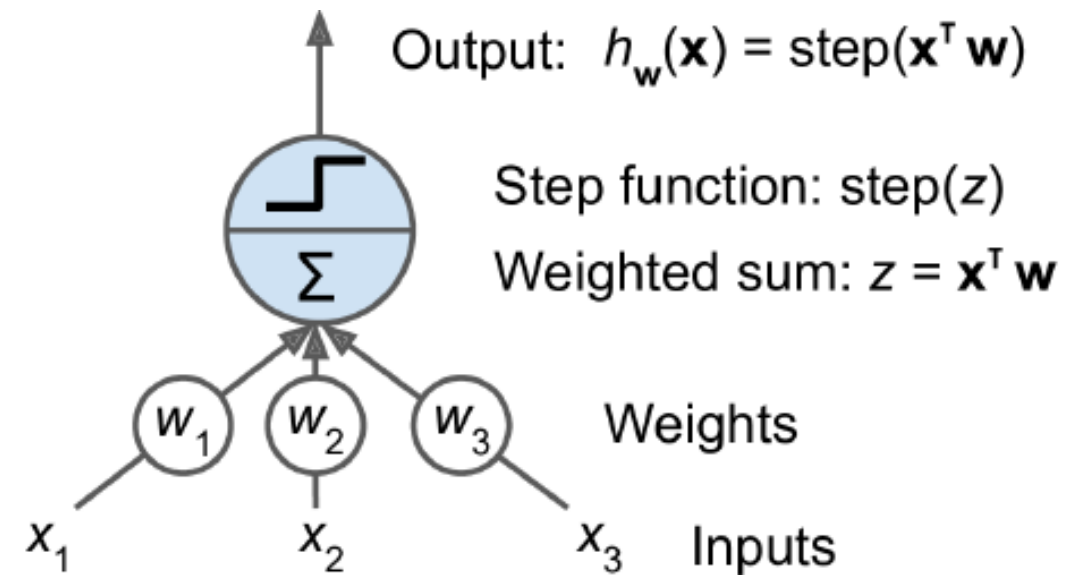


Deep Learning: Redes Neuronales Artificiales

- **Función de paso:** se aplica a la suma ponderada z de las entradas que llegan a la neurona
- Algunas de las más sencillas son:

- $heaviside(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$

- $sgn(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ 1 & \text{if } z > 0 \end{cases}$

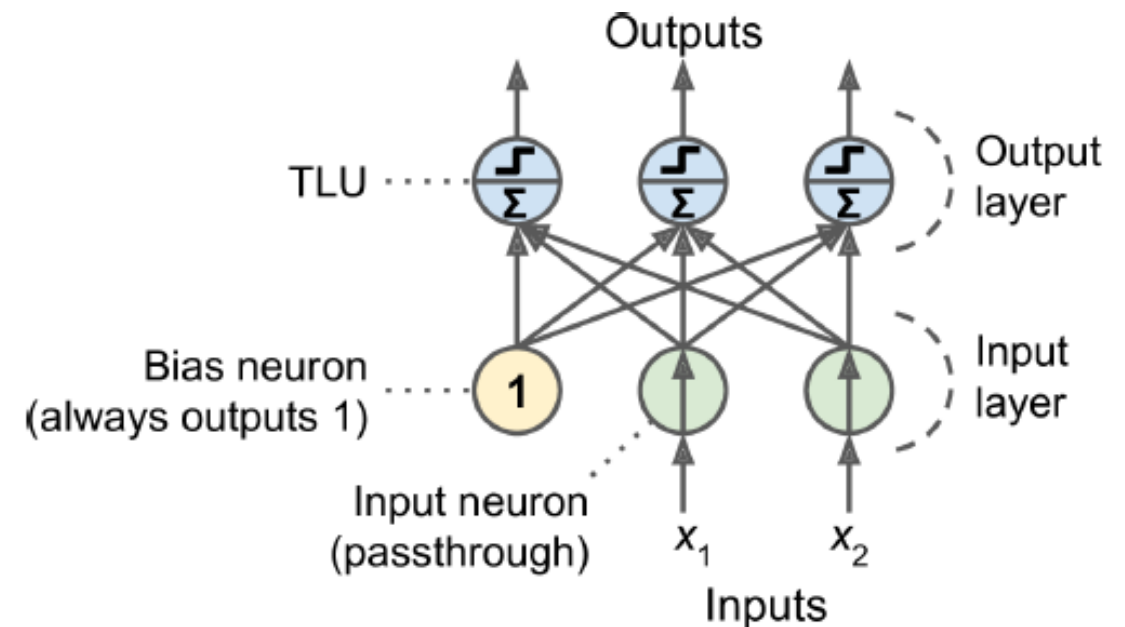


Deep Learning: Redes Neuronales Artificiales

- **Perceptron:** una única capa compuesta por varias TLUs, cada una conectada a todas las entradas (atributos de datos).
- Los datos son recogidos por neuronas en una capa de entrada, que en el caso más sencillo no aplican ninguna transformación en la información.

Salida = Entrada $\rightarrow h(x_i) = x_i$

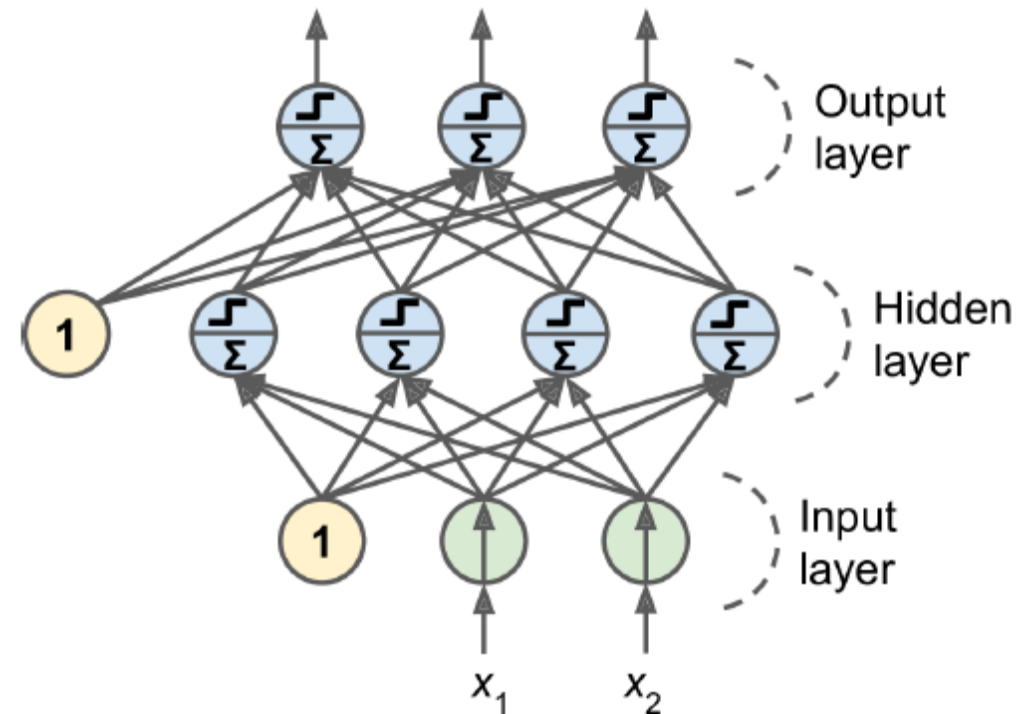
Para n entradas, habrá una capa de entrada con $n+1$ neuronas. Se añade una neurona de sesgo donde siempre $x_0 = 1$, y asume el rol de término de sesgo.



Redes Neuronales Artificiales(ANNs): perceptron

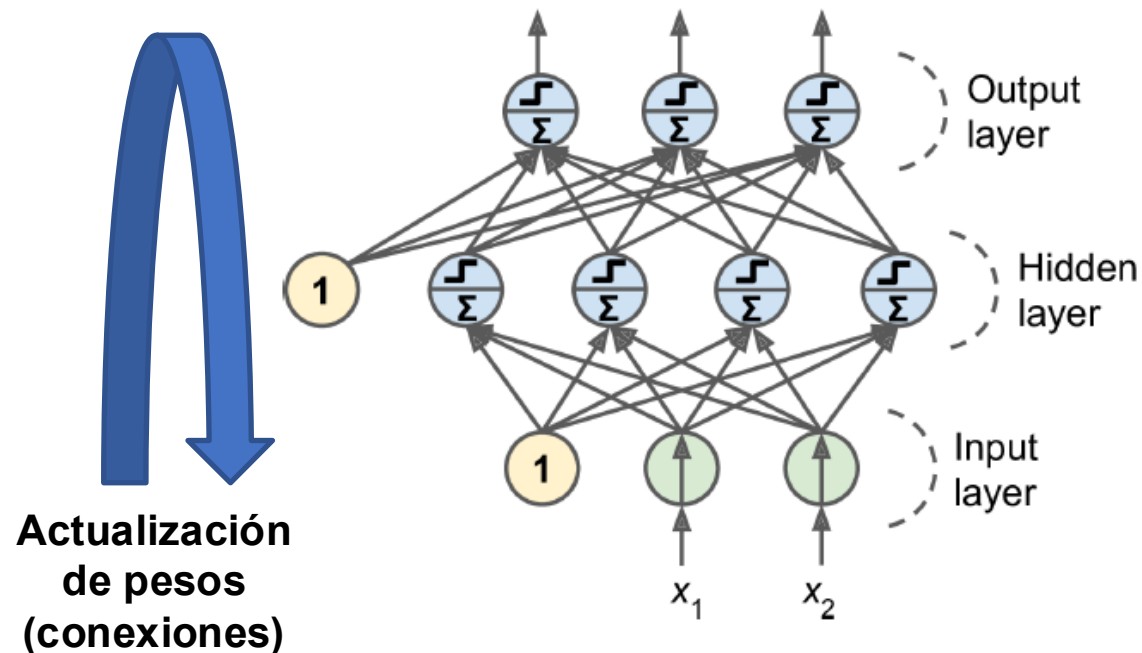
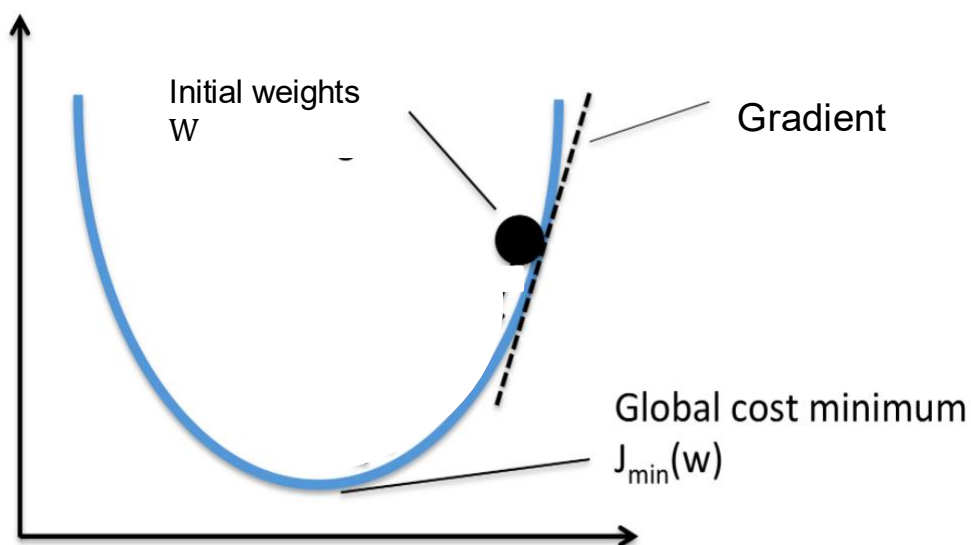
- **Multi-Layer Perceptron (MLP)** : en el perceptrón multicapa existen una o varias capas (capas ocultas) entre la capa de entrada y la de salida.
- Todas las capas excepto la de salida incorporan la neurona de sesgo.
- Las neuronas en capas consecutivas están totalmente conectadas → Capas densas

Cuando el número de capas ocultas en una red neuronal crece, empezamos a hablar de una red neuronal profunda (**Deep Neural Network**). De ahí viene el término **Deep Learning**: el área de la IA y el Machine Learning que se ocupa de desarrollar redes neuronales profundas para abordar problemas más complejos.



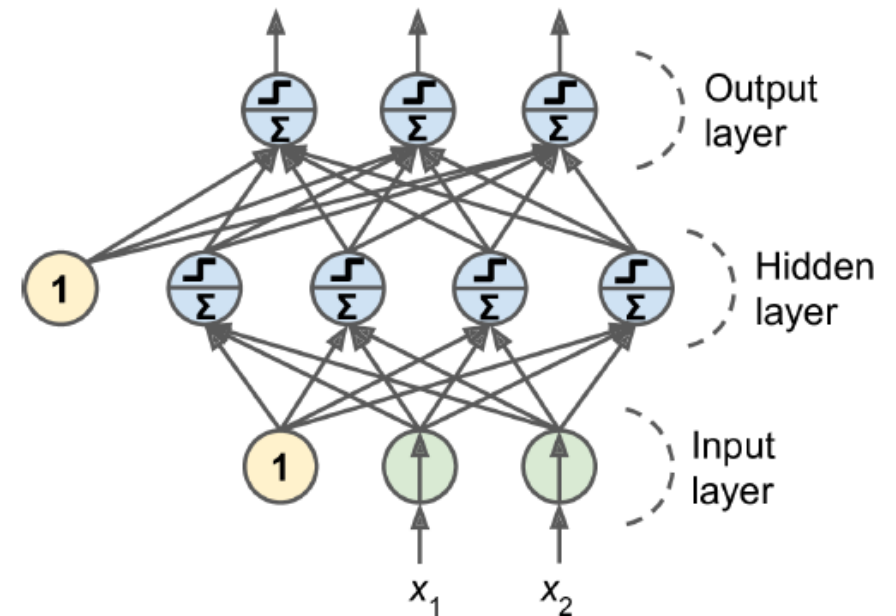
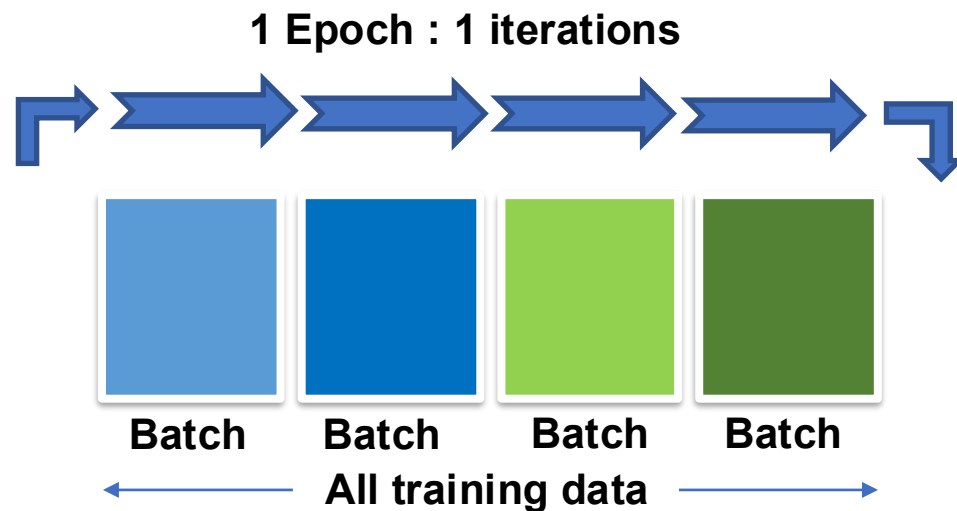
Redes Neuronales Artificiales (ANNs): backpropagation

- Hasta 1986, era difícil incluso entrenar una arquitectura tan sencilla como un MLP.
- El algoritmo de propagación hacia atrás fue introducido en este año.
 - Basado en el **Descenso de Gradiente**, calcula el gradiente de la función de pérdida atravesando la red neurona hacia adelante y después hacia atrás.
 - Tras cada iteración, los pesos de las conexiones entre neuronas –inicializados aleatoriamente– se van actualizando para reducir los errores cometidos.



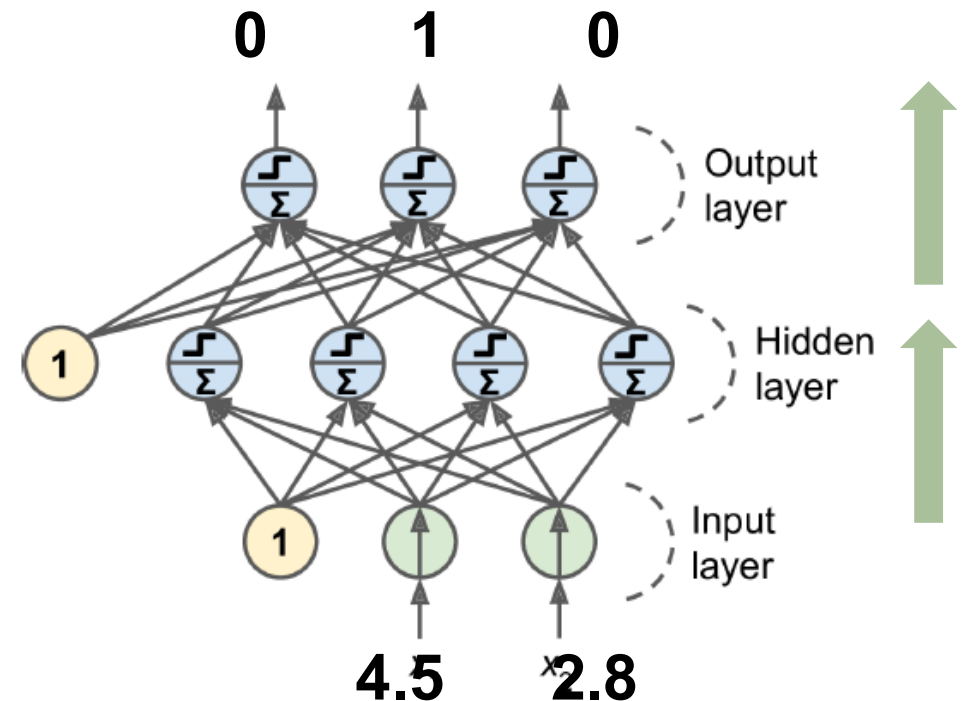
Redes Neuronales Artificiales (ANNs): backpropagation

- Durante el **entrenamiento** de la red neuronal, se itera sobre el conjunto de entrenamiento al completo, cada uno de estos **ciclos de iteración** se llama *epoch*.
- Los **ejemplos de entrenamiento se van pasando por lotes**, no individualmente,, por ejemplo con un tamaño de 32 instancias por lote (*batch*).



Redes Neuronales Artificiales(ANNs): backpropagation

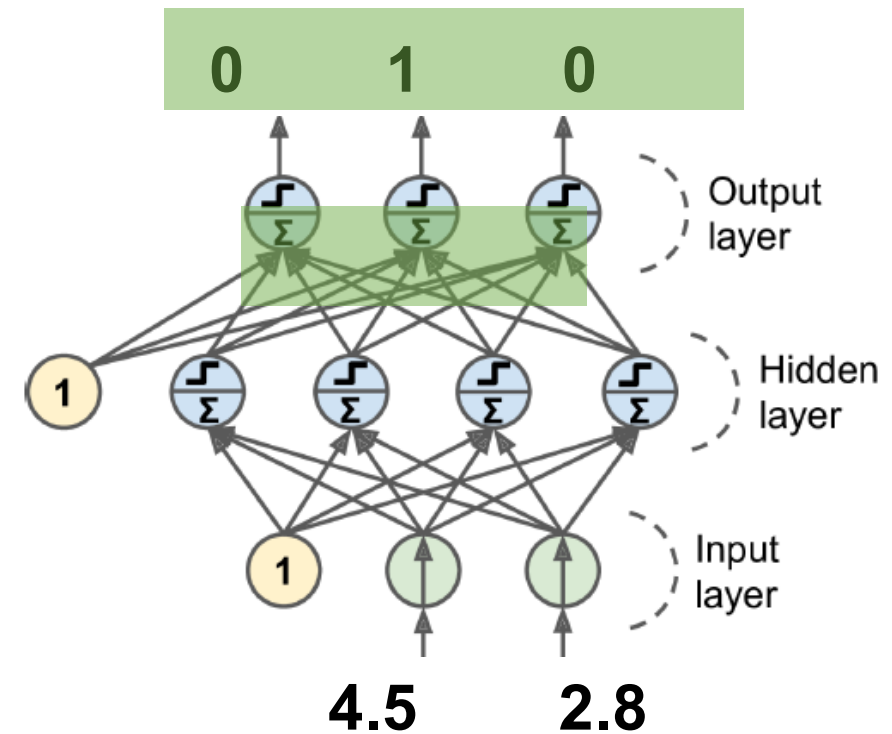
- Una iteración comienza pasando un lote de instancias a la capa de entrada.
- Para cada instancia, se calcula la predicción del modelo transmitiendo la información hasta la capa de salida: **forward pass**
 - En cada capa oculta, se toma la información de entrada y las neuronas la transforman antes de pasarla a las neuronas de la siguiente capa.
 - Se “memorizan” los resultados intermedios, ya que se necesitarán en la etapa de retroceso para calcular errores y ajustar pesos.



Redes Neuronales Artificiales(ANNs): backpropagation

¿Cómo funciona el algoritmo de *backpropagation*?

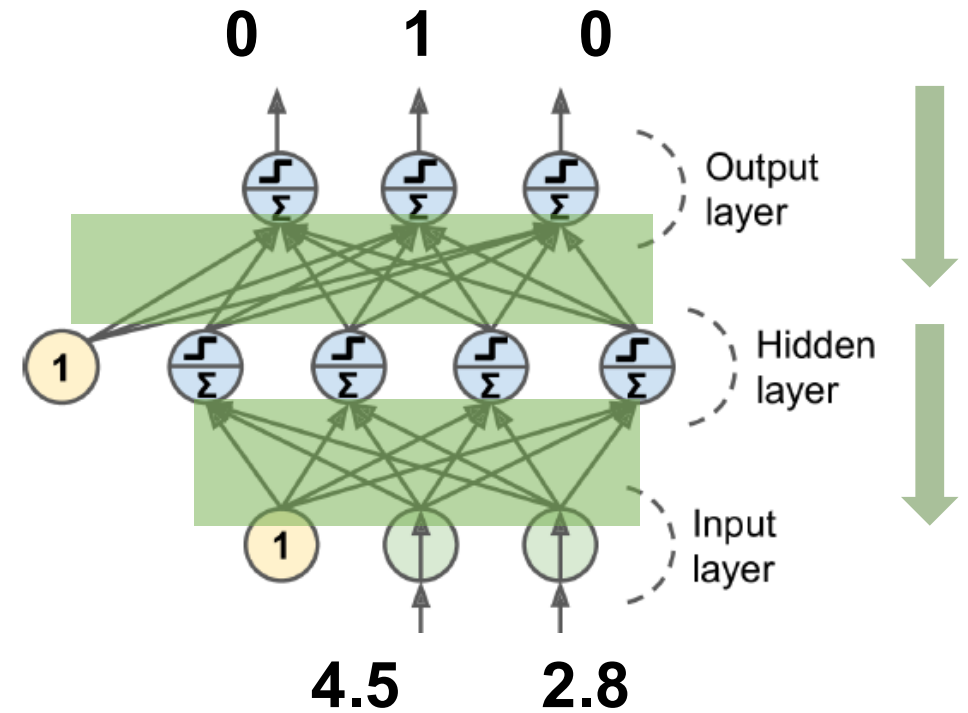
- Basándose en procesos de cálculo, estimar cuánto ha contribuido cada conexión entre neuronas al error cometido.



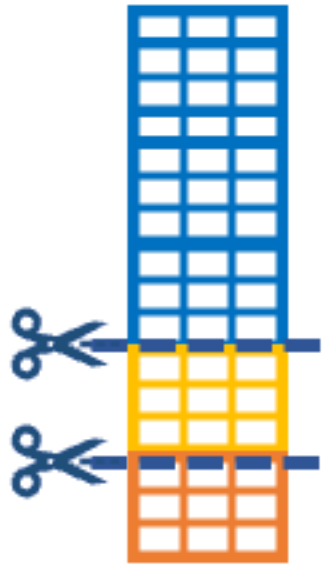
Redes Neuronales Artificiales(ANNs): backpropagation

¿Cómo funciona el algoritmo de *backpropagation*?

- Basándose en procesos de cálculo, estimar cuánto ha contribuido cada conexión entre neuronas al error cometido.
- **Continuar hacia atrás:** medir la contribución al error cometido por parte de cada conexión procedente de la capa previa:
backward pass
- Aplicar descenso de gradiente para actualizar los pesos de las conexiones según los gradientes del error cometido.



El enfoque de particionado training-validation-test

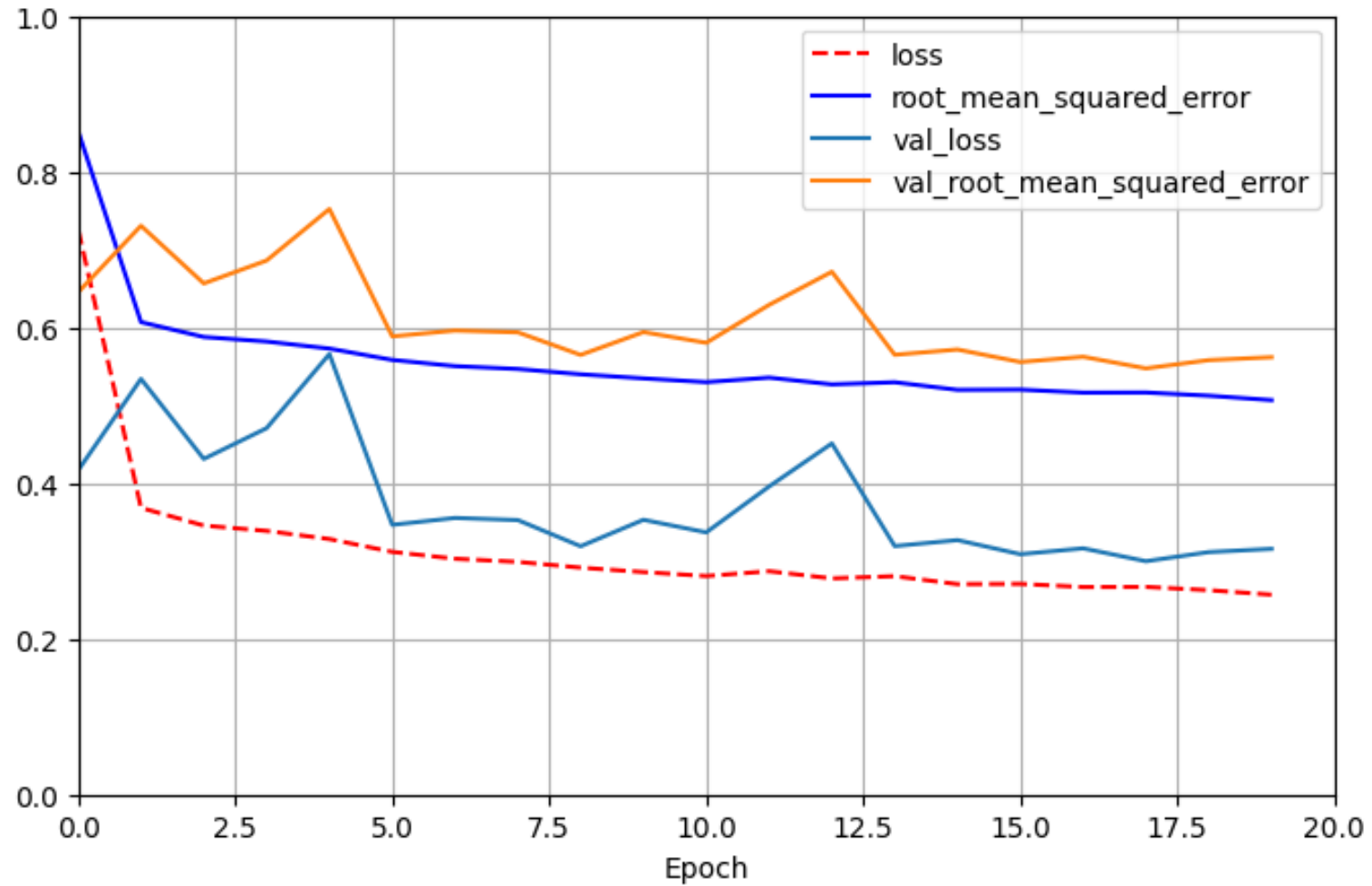


Training set ($\approx 70\%$): datos proporcionados al modelo junto con etiquetas conocidas para aprender los patrones de los datos.

Validation set ($\approx 15\%$): usando entre rondas de iteración (epochs) para evaluar cómo de bien va el proceso de entrenamiento sobre datos no usados en el mismo, y así facilitar un mejor ajuste del modelo.

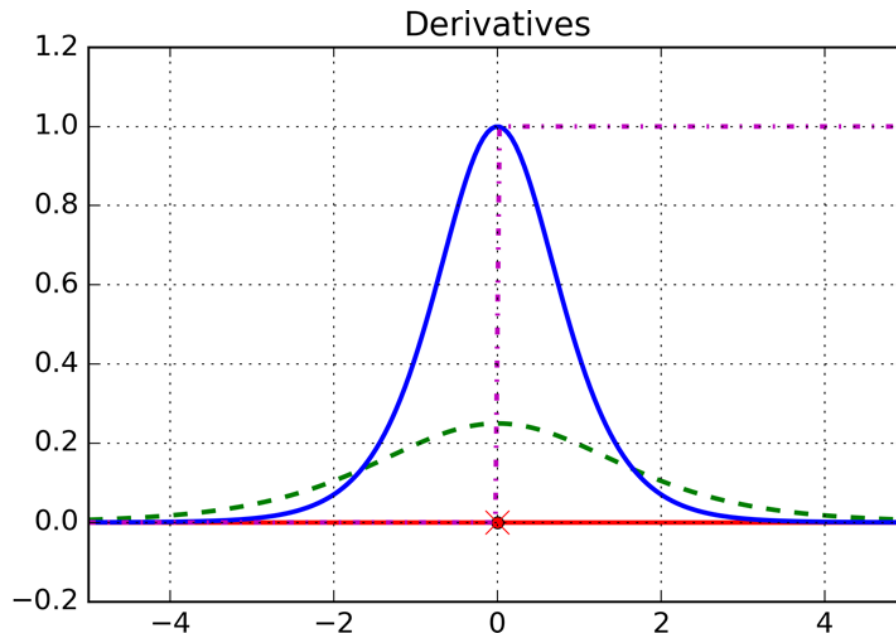
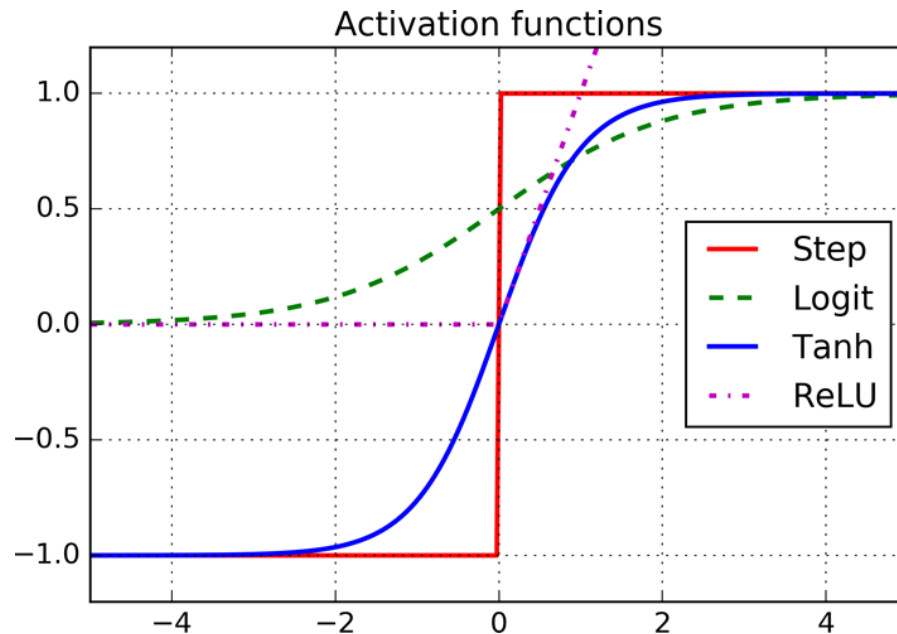
Test set ($\approx 15\%$): datos nunca pasados al modelo durante el entrenamiento, y usados una vez el modelo ha sido totalmente entrenado para evaluar su rendimiento real y capacidad de generalización.

Curvas de aprendizaje (learning curves)



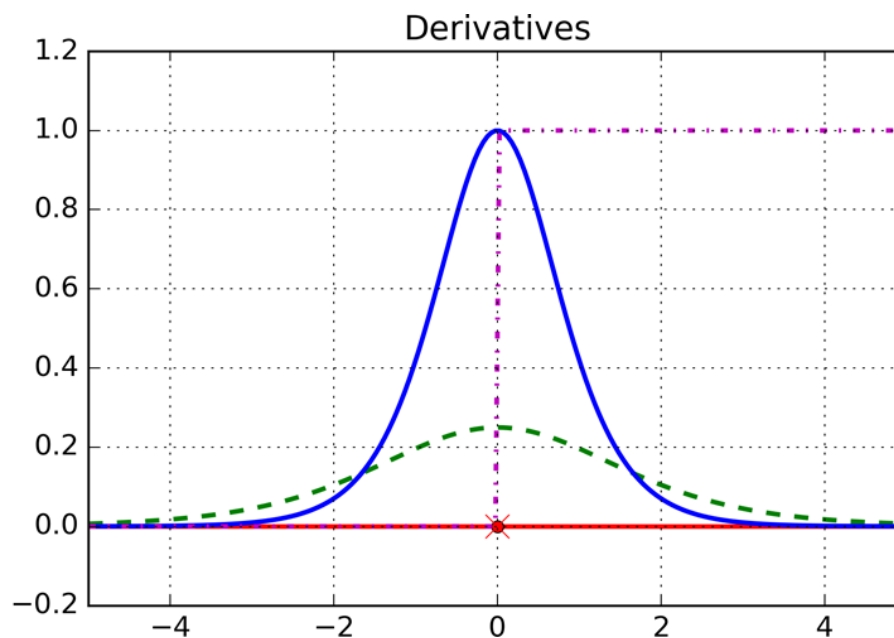
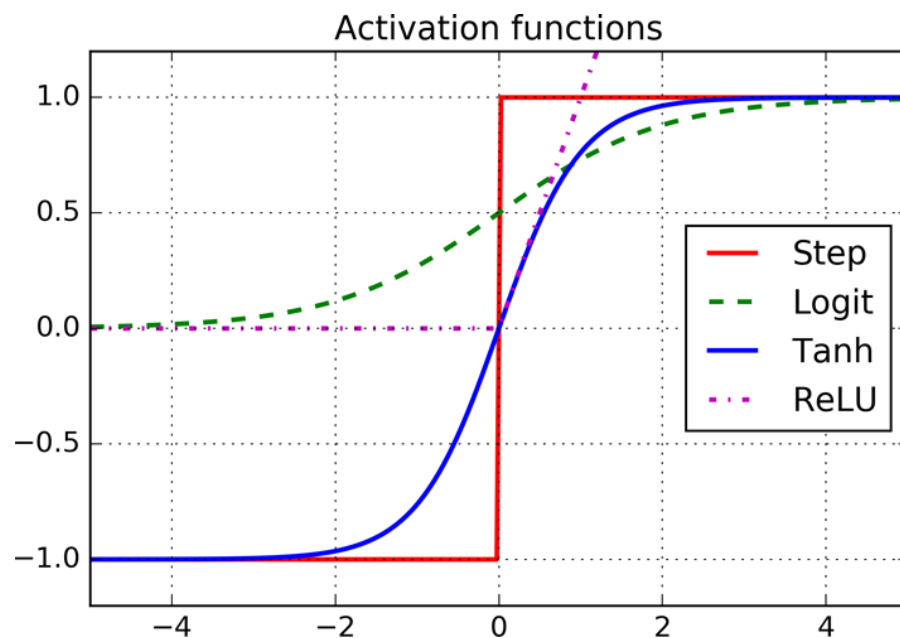
Deep Learning: funciones de activación

- Para el proceso de entrenamiento basado en *backpropagation*, es necesario definir funciones de activación neuronales que sean derivables (la función de paso no lo era).
- Sigmoid/Logistic: $\sigma(z) = 1/(1 + \exp(-z))$



Deep Learning: funciones de activación

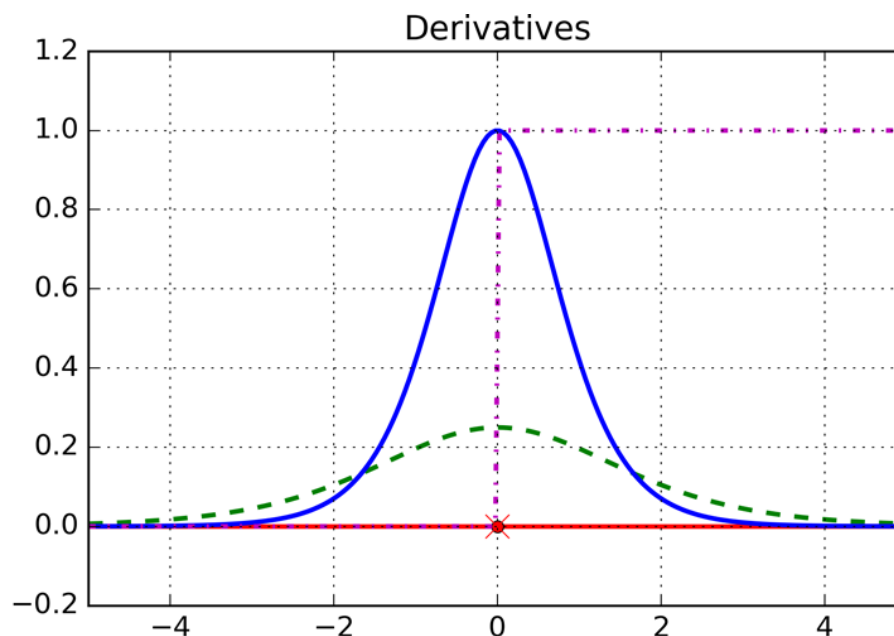
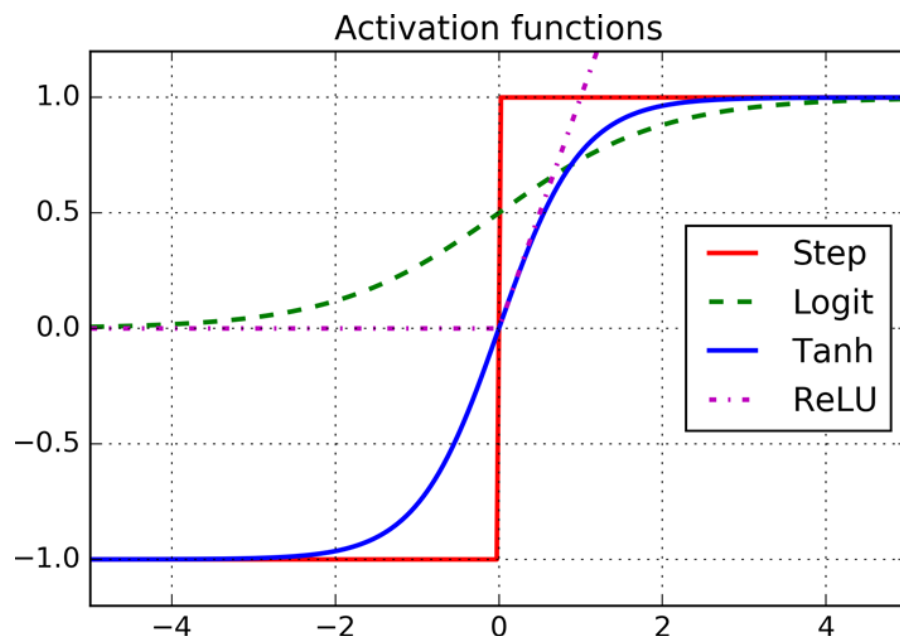
- Para el proceso de entrenamiento basado en *backpropagation*, es necesario definir funciones de activación neuronales que sean derivables (la función de paso no lo era).
- Hyperbolic tangent (tanh): $\tanh(z) = 2\sigma(2z) - 1$



Deep Learning: funciones de activación

- Para el proceso de entrenamiento basado en *backpropagation*, es necesario definir funciones de activación neuronales que sean derivables (la función de paso no lo era).
- Rectified Linear Unit: $\text{ReLU}(z) = \max(0, z)$

Las funciones de activación son muy importantes para disipar la *linealidad* intrínseca al cálculo de $z = \mathbf{x}^T \mathbf{w}$, algo crucial para poder aprender de datos complejos no lineales.

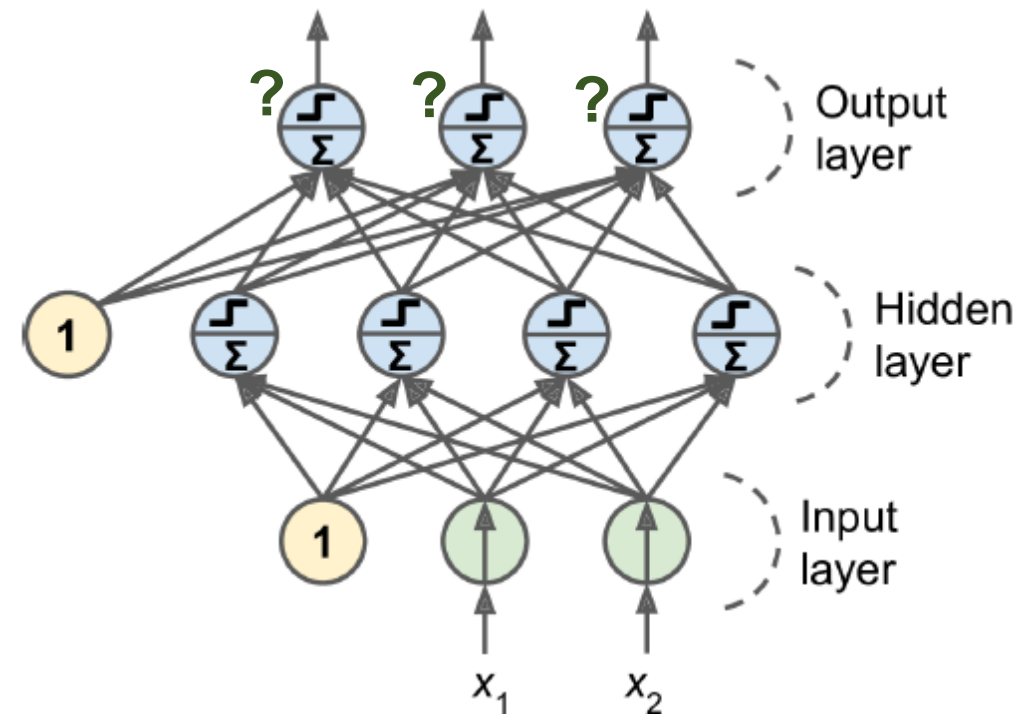


Arquitectura de un MLP para Regresión

La arquitectura MLP puede usarse para tareas de regression:

- Regresión clásica (una etiqueta): una neurona en la capa de salida.
- Regresión multivariante (predecir múltiples etiquetas, p.ej. precio y vida útil de un vehículo): múltiple neuronas de salida, una por variable a predecir.

Los MLPs para regresión no tienen ninguna función de activación en las neuronas de salida.



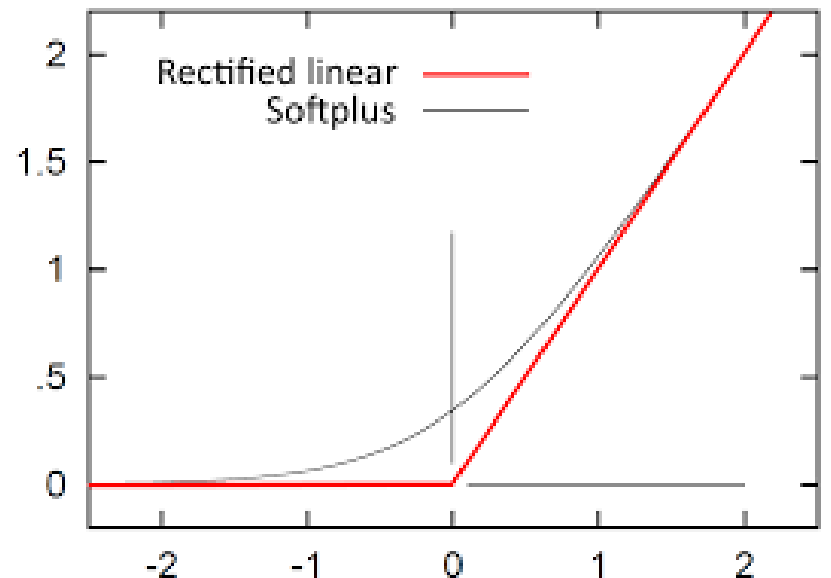
Arquitectura de un MLP para Regresión

La arquitectura MLP puede usarse para tareas de regression:

- Regresión clásica (una etiqueta): una neurona en la capa de salida.
- Regresión multivariante (predecir múltiples etiquetas, p.ej. precio y vida útil de un vehículo): múltiple neuronas de salida, una por variable a predecir.

Los MLPs para regresión no ninguna función de activación en las neuronas de salida.

- Excepciones:
 - Activación ReLU para asegurar valores positivos.
 - Su versión "suavizada": $\text{softplus}(z) = \log(1 + \exp(z))$
 - Función logística o tangente hiperbólica para mantener salidas dentro de un rango, p.ej. Porcentajes.



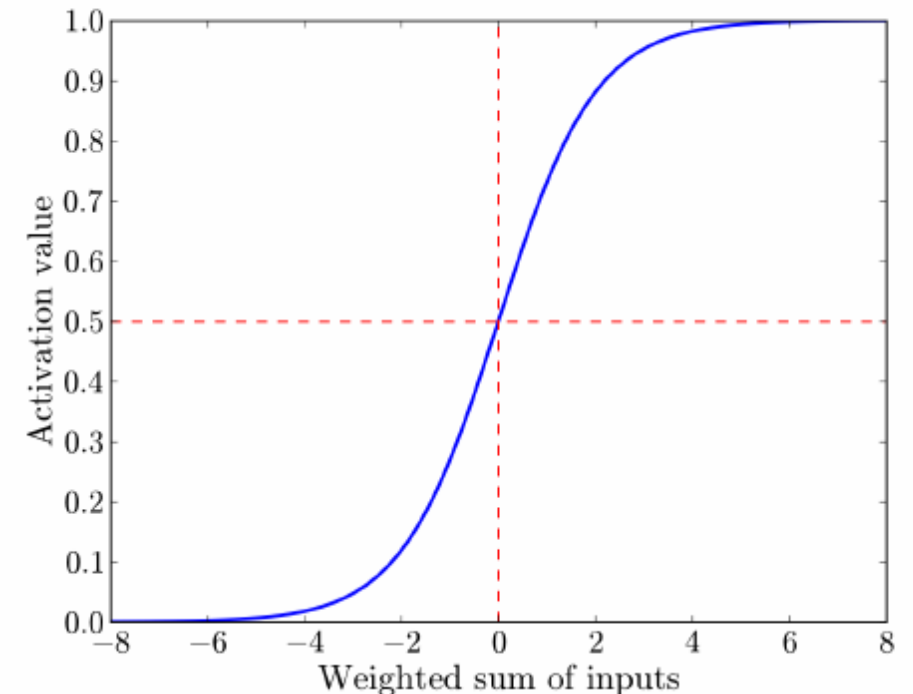
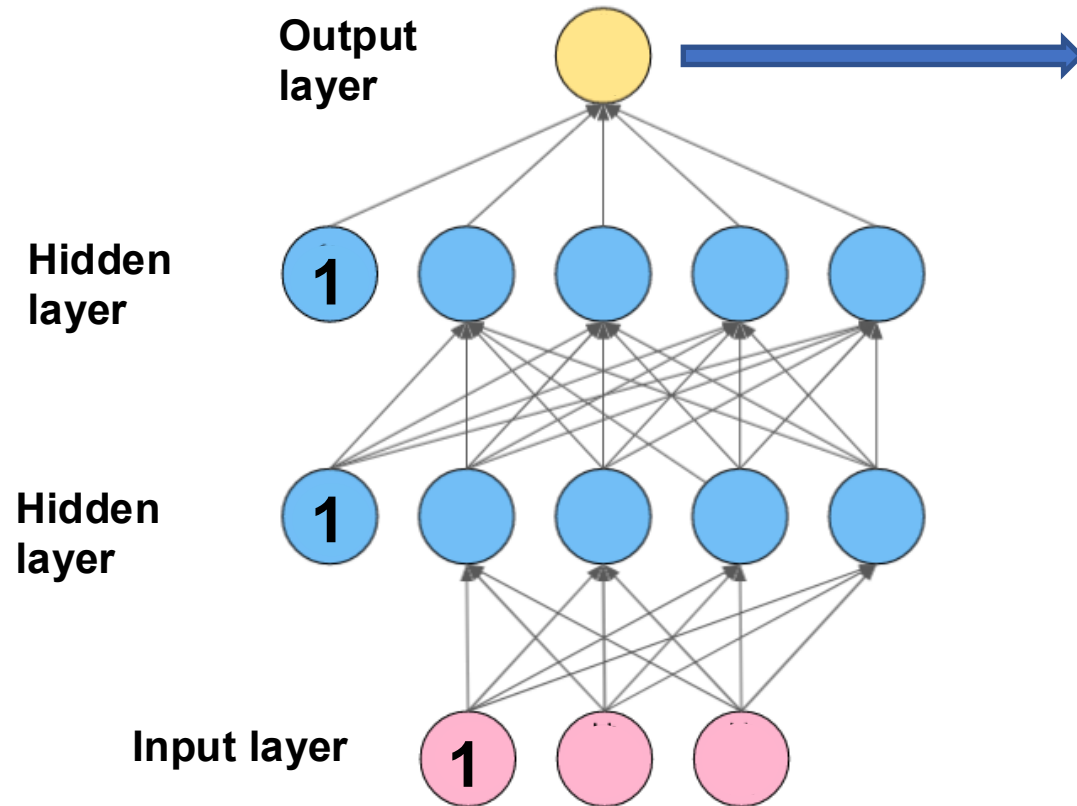
Arquitectura de un MLP para Regresión

Hiperparámetro	Cómo definirlo
# Input neurons	Tantas neuronas en la capa de entrada como atributos del dataset que se usarán (p.ej. 3 para el dataset <i>50Startups</i>).
# Hidden layers	De una a cinco capas ocultas, dependiendo del problema y la complejidad del dataset.
# Neurons/hidden layer	5 a 100 neuronas/capa, dependiendo del problema y la complejidad del dataset.
# Output neurons	1 neurona por variable (etiqueta) a predecir
Hidden layer activation	Normalmente ReLU
Output layer activation	Sin activación (por defecto) ReLU-softplus para asegurar salidas positivas Logistic/tanh para asegurar salidas acotadas a un intervalo numérico
Loss function	RMSE (o MAE si el dataset tiene muchos outliers)

Arquitectura de un MLP para Clasificación

MLPs para clasificación binaria:

- Solo se necesita una neurona de salida, con función de activación logística.
 - La salida es la **probabilidad estimada de clase positiva**.



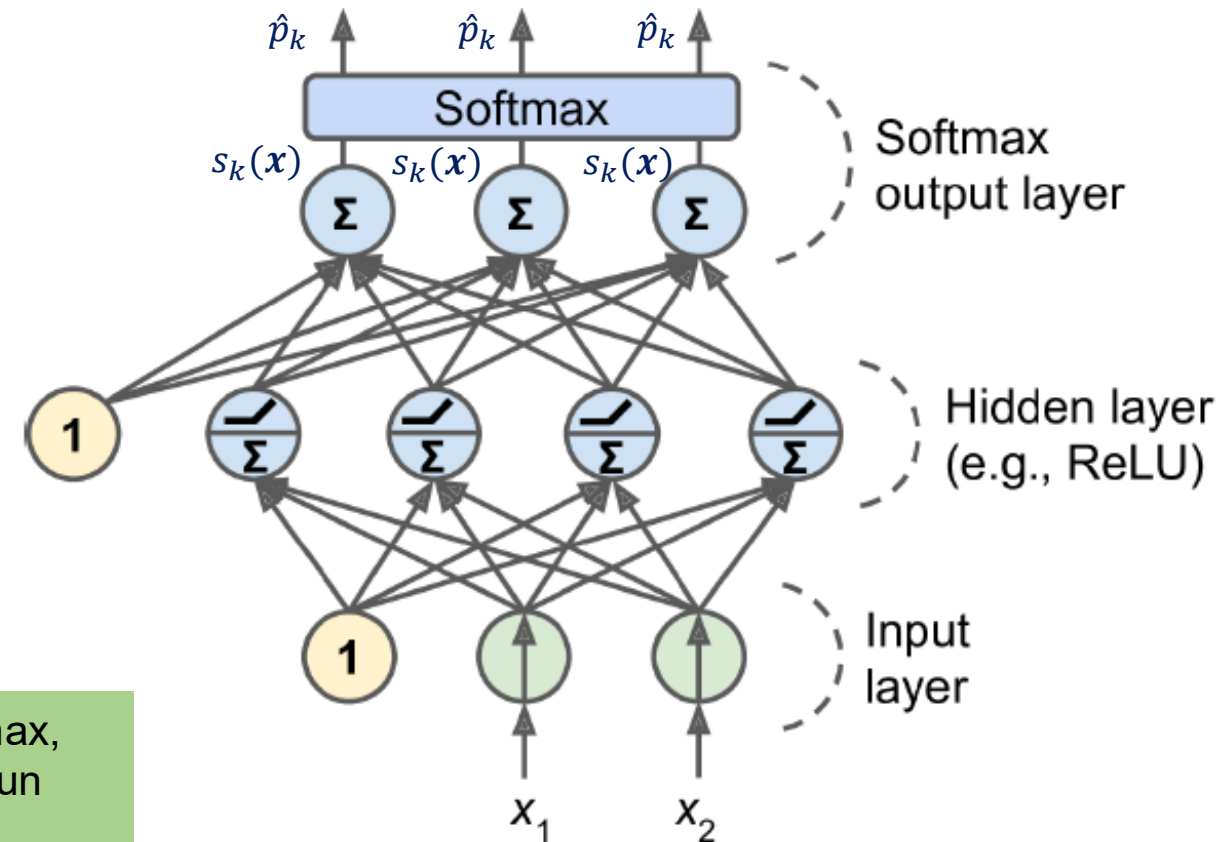
Arquitectura de un MLP para Clasificación

MLPs para clasificación con $K \geq 3$ clases:

- Una neurona de salida para cada posible clase (p.ej. 3 para el dataset iris, 10 para MNIST, ...).
- Una **activación softmax** aplicada a la capa al completo: así, las probabilidades de cada clase se normalizan para sumar 1
- Cada neurona de salida devuelve el score $s_k(x)$ de cada clase k
- Softmax toma todos los $s_k(x)$ y los convierte en **probabilidades de clase**:

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$

RECUERDA: Esta capa de salida con arquitectura softmax, será más adelante, en los modelos de lenguaje (LLMs), un elemento importante de los mismos.



Arquitectura de un MLP para Clasificación

Hyperparameter	Guidelines
# Input neurons	Tantas neuronas como atributos (4 en iris, 784 en MNIST de imágenes de 28x28, ...)
# Hidden layers	1 a 5 capas, según complejidad del problema y dataset
# Neurons/hidden layer	5 a 100 neuronas/capa, según problema y datos
Hidden layer activation	Normalmente ReLU
# Output neurons	Clasificación binaria: 1 Clasificación binaria multi-etiqueta: 1 por atributo de clase a predecir Clasificación bajo múltiples clases: 1 por class
Output layer activation	Clasificación binaria: Logistic Bajo múltiples clases: softmax
Loss function	Cross entropy. $J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$

IGUALES QUE EN MLPs PARA REGRESIÓN

Tensorflow vs. PyTorch

Característica	TensorFlow (con Keras)	PyTorch
¿Quién lo creó?	Google	Meta (Facebook)
¿Cómo se siente al usarlo?	Como una herramienta especializada.	Como escribir código Python normal.
Facilidad para aprender	Fácil si usas su extensión Keras (es muy directo).	Fácil si ya dominas Python (es muy intuitivo).
Buscar y corregir errores (depuración)	Más difícil; a veces da mensajes complejos.	Más fácil; te avisa como cualquier programa de Python.
¿Quién lo usa más?	Grandes empresas (para apps y servidores).	Universidades y científicos (para inventar arquitecturas nuevas).
Llevarlo a móviles/web	Excelente . Está muy preparado para eso.	Bueno , pero un poco más secundario.



Ajuste de hiperparámetros en Deep Learning

¿Cómo encontrar la mejor configuración de ajustes “manuales” para nuestro modelo?

- Con tantos hiperparámetros en los modelos basados en redes neuronales, es como buscar una aguja en un pajar.

Hiperparámetros del modelo (arquitectura)

Number of layers

Number of neurons per layer

Activation Functions

Paths Between Layers (functional API)

...

Hiperparámetros del algoritmo de entrenamiento

Learning rate

Number of iterations

Algorithm Optimizer

Batch Size

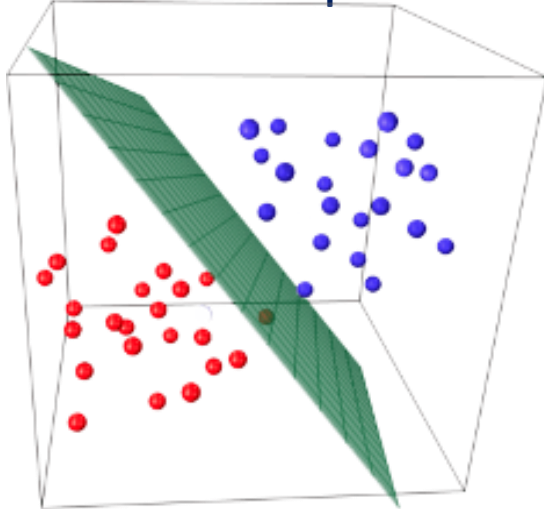
Weights initialization

...

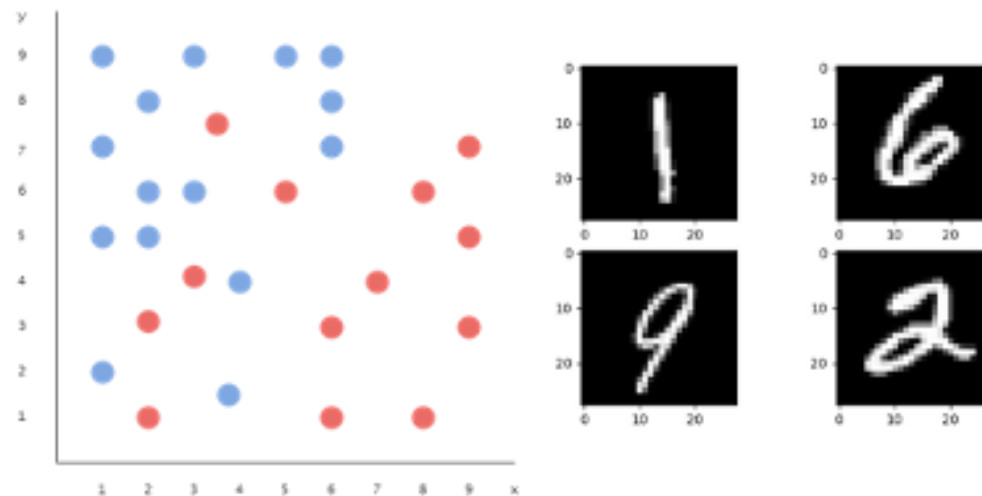
Ajuste de hiperparámetros en Deep Learning

¿Cuántas capas ocultas?

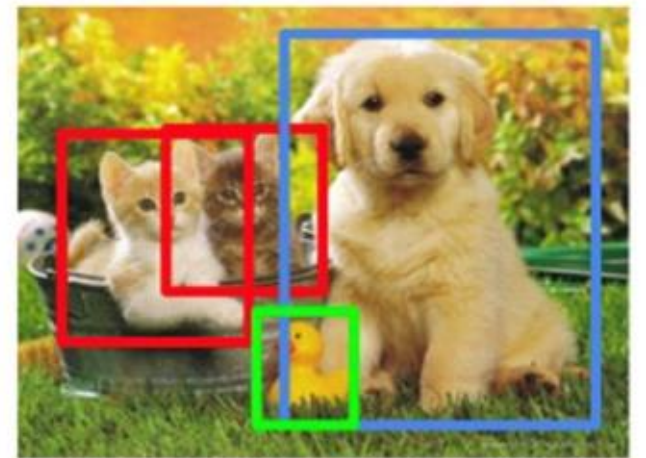
Datos linealmente separables: sin capas ocultas.



Complejidad media: 1-2 capas ocultas.



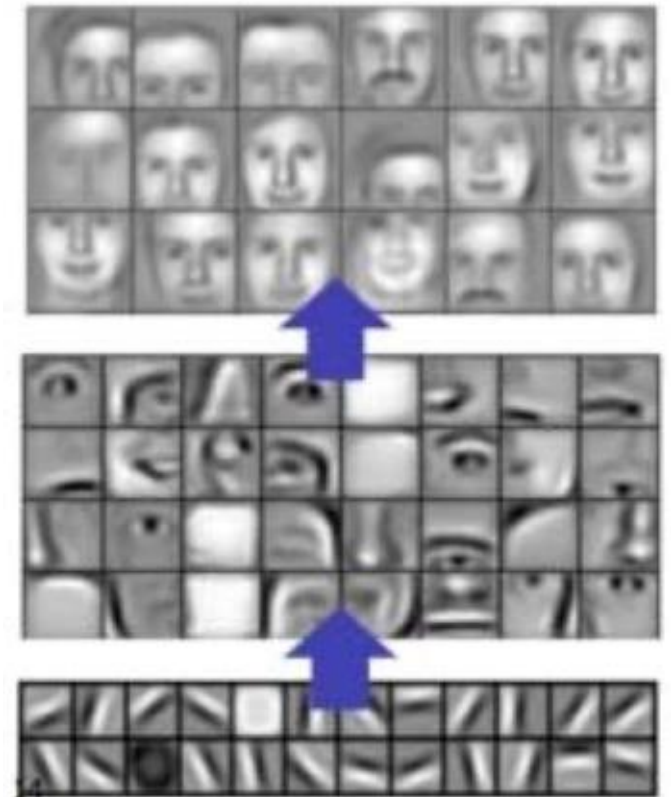
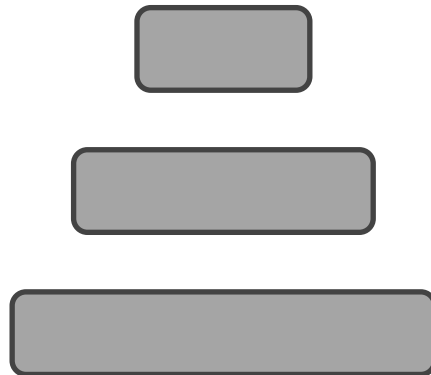
Complejidad alta: 3+ capas ocultas



Ajuste de hiperparámetros en Deep Learning

¿Cuántas neuronas por capa oculta?

- Enfoque clásico: piramidal
 - Muchos patrones de bajo nivel, gradualmente se van combinando en un número menor de patrones de alto nivel.



Ajuste de hiperparámetros en Deep Learning

¿Cuántas neuronas por capa oculta?

- Enfoque actual:
 - Capas ocultas de igual tamaño, o a lo sumo solo la primera de ellas más grande.

Dado un dataset con m atributos:

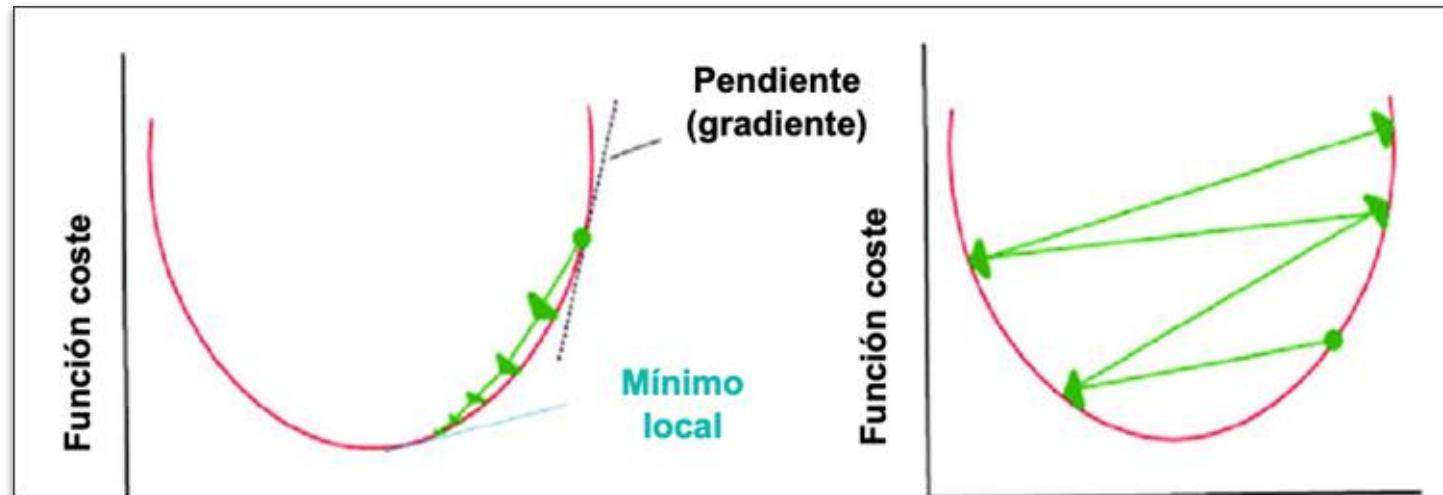
- Datos simples: $\frac{2}{3} m$ neuronas/capa
- Datos complejos: $m, 2m$ neuronas/capa
- Se puede probar a aumentar este número lentamente, pero dejar de hacerlo si aparecen signos de *overfitting*

Como norma general, suele ser más eficaz aumentar el número de capas ocultas que expandir el tamaño de las capas

Ajuste de hiperparámetros en Deep Learning

¿Qué ratio de aprendizaje (learning rate)?

- Uno de los hiperparámetros más importantes a tener en cuenta en deep learning
 - Demasiado pequeño $\rightarrow$ entrenamiento más lento y costoso
 - Demasiado grande $\rightarrow$ entrenamiento inestable, difícil de converger (*bouncing*)



Ajuste de hiperparámetros en Deep Learning

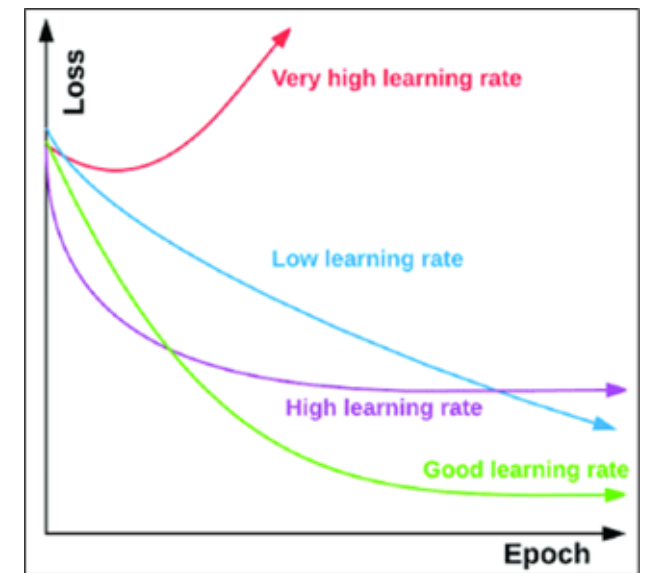
¿Qué ratio de aprendizaje (learning rate)?

Comenzar con un *learning rate* pequeño, entre 10^{-5} y 0.01

Aumentar gradualmente **x10** siempre que la función de pérdida (*loss*) no aumente

Cuando lleguemos a **0.1**, aumentar mucho más suavemente, p.ej., **+0.1**

Si la función de pérdida se vuelve estable o aumenta de forma explosiva, reducir *learning rate* de inmediato

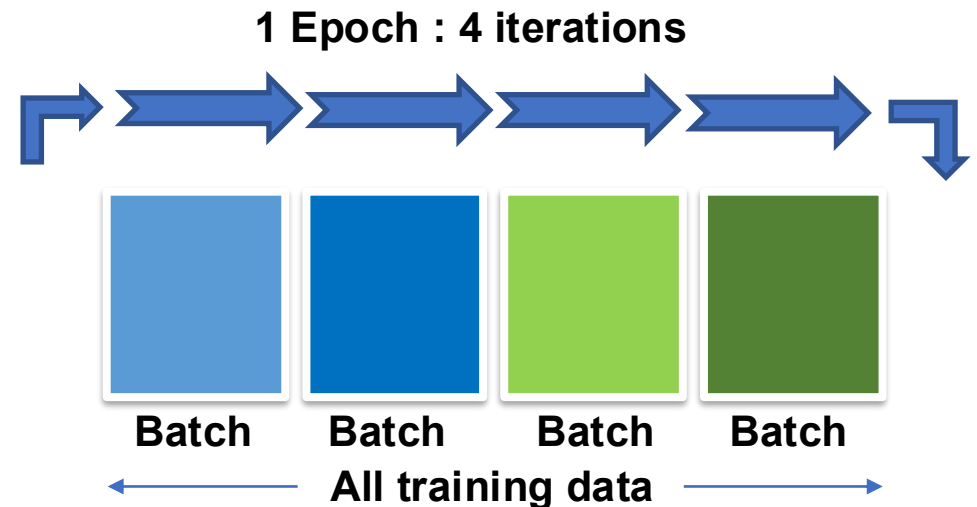


Ajuste de hiperparámetros en Deep Learning

¿Qué tamaño de lote (*batch size*) y número de rondas de entrenamiento (*epochs*)?

Mantener a lo sumo **32 instancias por lote**, o considerar **64/128** solo si tenemos mayores recursos de cómputo como GPUs o TPUs.

Por defecto usar un valor alto de **#epochs**, añadiendo un mecanismo de *early stopping* que detenga el proceso de entrenamiento si el nivel de mejora entre dos *epochs* consecutivos es mínimo



Optimización de modelos de Deep Learning

Optimizer: un algoritmo subyacente al proceso de entrenamiento, que "investiga" las relaciones entre los atributos de entrada que conforman el dataset, y las etiquetas de salida.

- Responsable de inicializar y ajustar los parámetros “entrenables”, e ir gradualmente minimizando el error cometido por el modelo de red neuronal.

```
[...]  
optimizer = tf.keras.optimizers.SGD()
```

Optimizers populares:

- ***Stochastic Gradient Descent*** (SGD): opción por defecto en Keras.

Optimización de modelos de Deep Learning

Optimizer: un algoritmo subyacente al proceso de entrenamiento, que "investiga" las relaciones entre los atributos de entrada que conforman el dataset, y las etiquetas de salida.

- Responsable de inicializar y ajustar los parámetros “entrenables”, e ir gradualmente minimizando el error cometido por el modelo de red neuronal.

```
[...]  
optimizer =  
tf.keras.optimizers.Adam(learning_rate=0.001)
```

Optimizers populares

- **Adam:** muy popular, tiene un *learning rate* adaptativo.



¿Qué puede ir mal al entrenar un modelo de deep learning y cómo mitigarlo?

SOBREAJUSTE - OVERFITTING

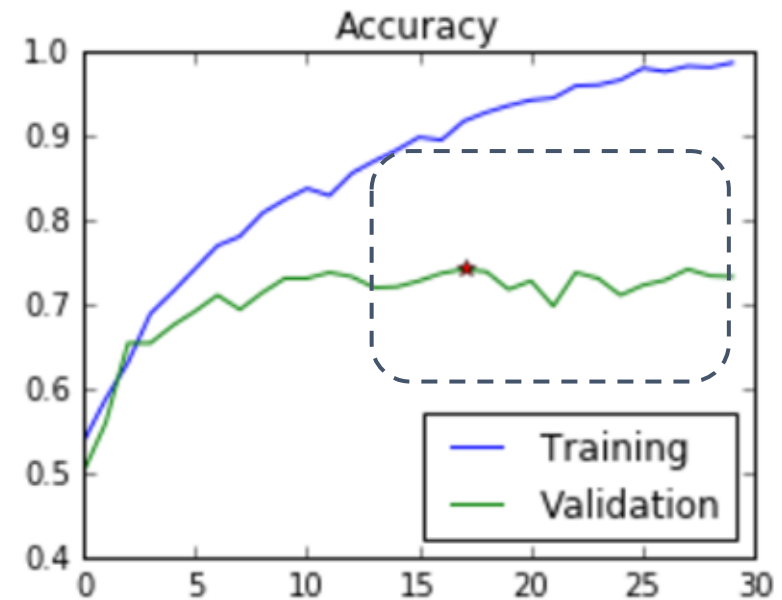
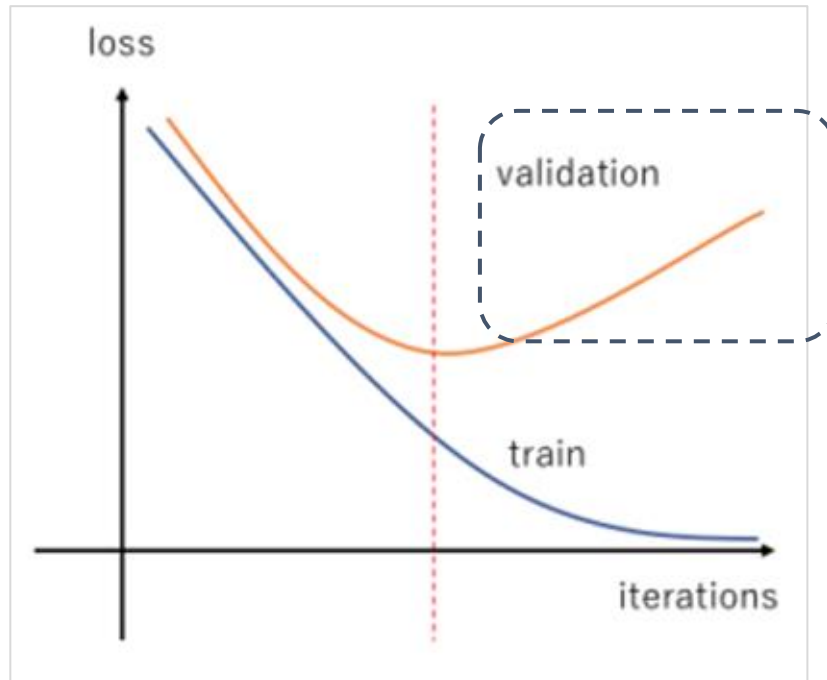
Entrenar un buen modelo supone equilibrar dos objetivos:

- **Aprender patrones útiles de inferencia** a partir de los datos de entrenamiento.
- Saber cómo hacer predicciones sobre futuros datos no vistos: **generalizar**

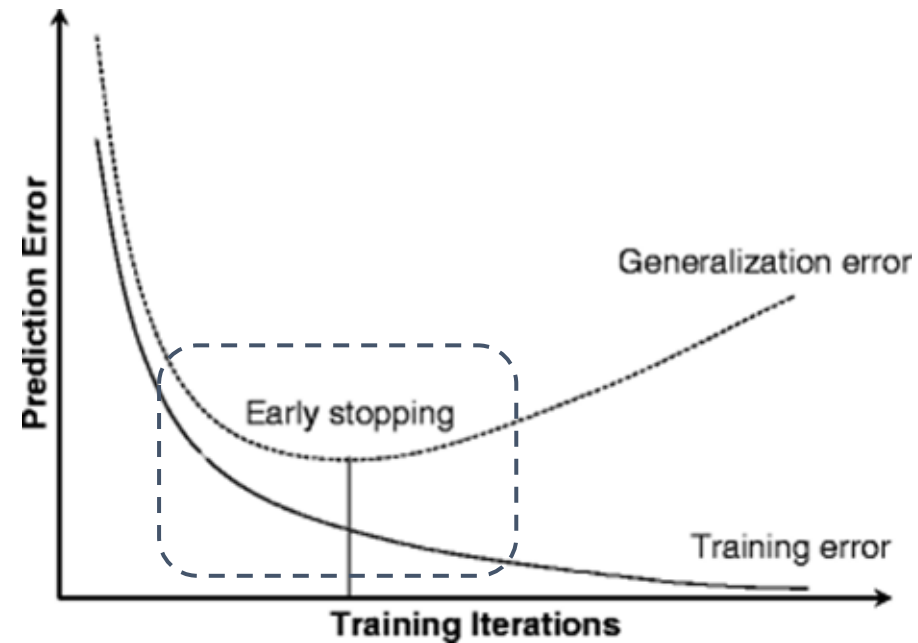
En resumen, un buen modelo debe saber aprender sin "memorizar"

OVERFITTING

Se puede detectar fácilmente con las curvas de aprendizaje durante el entrenamiento de la red neuronal.



OVERFITTING

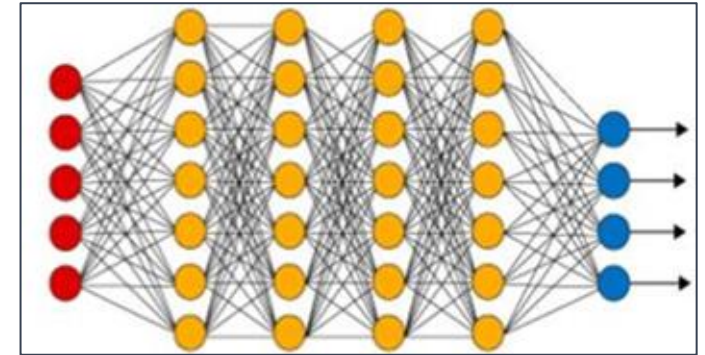


Una detención temprana (*early stopping*) durante el proceso de entrenamiento es a menudo una forma de evitar el problema.

Combatir el sobreajuste en DL

Si el *early stopping* no es suficiente, hay otras vías de actuación:

- Reducir complejidad del modelo (tamaños, capas...)
- Restringir pesos (regularización): *dropout*, *batch normalization*...
 - Las redes neuronales profundas pueden tener hasta billones de parámetros a aprender.
- Usar más datos de entrenamiento
 - Más diversidad de ejemplos y variaciones de todo tipo entre estos, ayuda a construir un modelo más capaz de generalizar bien a nuevos datos futuros no vistos antes.



REGULARIZACIÓN EN KERAS: L1, L2

Estrategias basadas en restringir los pesos de algunas conexiones en la red.

- Añadir **términos de regularización** a la **función de costo**
- λ controla la intensidad de la regularización

$$L_1(X, \omega) = L(X, \omega) + \lambda \sum |\omega_i|$$
$$L_2(X, \omega) = L(X, \omega) + \lambda \sum \omega_i^2$$

L1 (Lasso): penaliza el valor absoluto de algunos pesos, a menudo reduciéndolos a cero

L2 (ridge): tiende a reducir pesos pero sin tantas reducciones a cero

REGULARIZACIÓN EN KERAS: L1, L2

Estrategias basadas en restringir los pesos de algunas conexiones en la red.

- Añadir **términos de regularización** a la **función de costo**
- λ controla la intensidad de la regularización

$$L_1(X, \omega) = L(X, \omega) + \lambda \sum |\omega_i|$$

$$L_2(X, \omega) = L(X, \omega) + \lambda \sum \omega_i^2$$

L1 (Lasso): penaliza el valor absoluto de algunos pesos, a menudo reduciéndolos a cero

L2 (ridge): tiende a reducir pesos pero sin tantas reducciones a cero

```
layer = tf.keras.layers.Dense(100,  
    activation="relu",  
    kernel_initializer="he_normal",  
    kernel_regularizer=  
        tf.keras.regularizers.l2(0.01))
```

Ejemplo Keras con $\lambda = 0.01$.

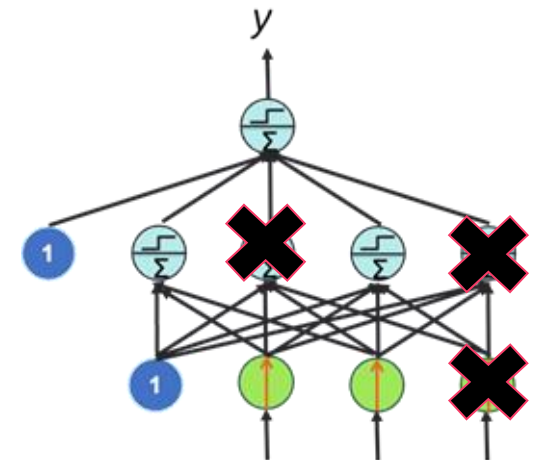
Keras admite tres tipos de regularización a nivel de capa: *l1*, *l2*, o *l1_l2* que combina ambas.

REGULARIZACIÓN EN KERAS: DROPOUT

- Añadir una “capa de dropout” sobre una capa ordinaria cualquiera:
 - Asigna a cada una de las neuronas en la capa afectada una probabilidad p de que pueda ser anulada en una iteración determinada (epoch) del entrenamiento.
 - Ejemplo que aplica *dropout* a la capa de entrada y oculta, con $p = 0.3$ y $p = 0.5$ respectivamente.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

model = Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(input_dim,)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(rate=0.5),
    Dense(output_dim, activation='softmax')
])
```



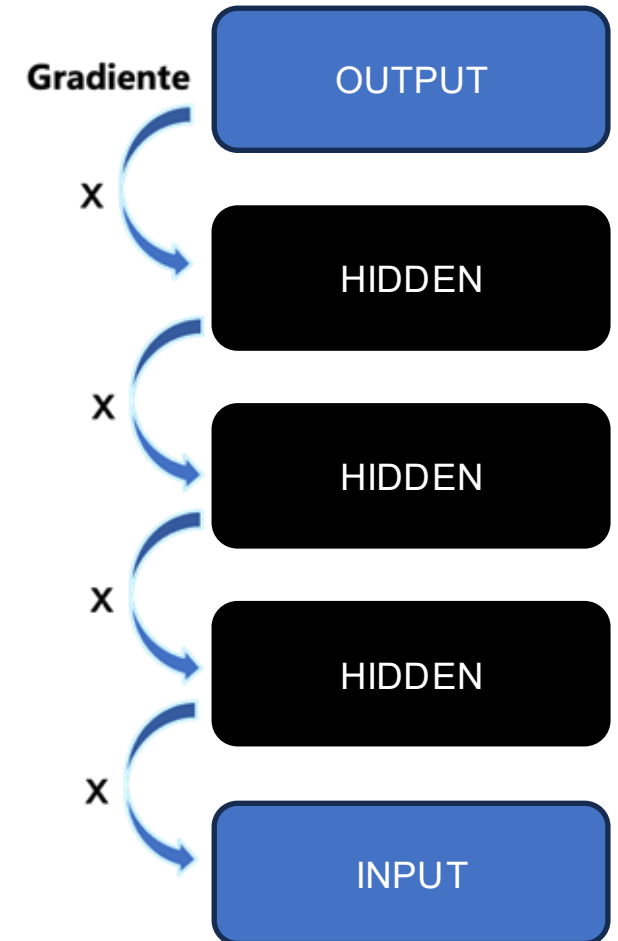
REGULARIZACIÓN EN KERAS: BATCH NORMALIZATION

- Originalmente pensado para mejorar la eficiencia y convergencia a óptimos durante el entrenamiento.
- También con utilidad demostrada si se usa para regularization:
 - Normaliza, para mini-lotes de instancias, los valores de activación que salen de la capa afectada. Al igual que dropout, se aplica a una capa determinada.

```
model = Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(input_dim,)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(output_dim, activation='softmax')
])
```

EXPLOSIÓN Y DESVANECIMIENTO DE GRADIENTES

- Problema que se origina durante el proceso de *backpropagation*
- El **gradiente del error se multiplica repetidamente** conforme se retrocede por la red.
- **Gradientes inicialmente pequeños o grandes** pueden volver anormalmente grandes o pequeños, afectando al proceso natural de actualización de pesos del modelo.
- Las capas iniciales a menudo causan este problema, especialmente con activaciones `sigmoid()` or `tanh()`

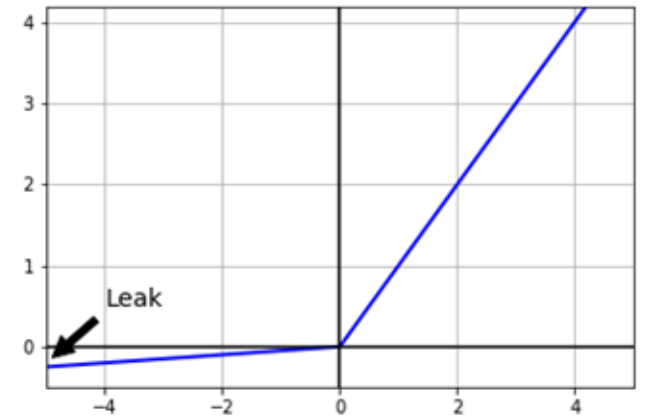


EXPLOSIÓN Y DESVANECIMIENTO DE GRADIENTES

¿Cómo prevenirlo en Keras?

- Intentar reducir #layers
- Inicialización de pesos “inteligente”: *Glorot, He, ...*
- Funciones de activación robustas, p.ej., *LeakyRELU*
- Regularización para evitar desvanecimiento.
- *Gradient clipping* para evitar explosión.

$$\text{LeakyRELU} = \max(\alpha x, x)$$



He initialization y *Gradient clipping* en Keras:

```
dense = tf.keras.layers.Dense(50,  
activation="relu",  
kernel_initializer="he_normal")
```

```
optimizer = SGD(clipvalue=1.0)  
model.compile(loss='mean_squared_error',  
optimizer=optimizer)
```

REDES NEURONALES CONVOLUTIVAS (CNN)

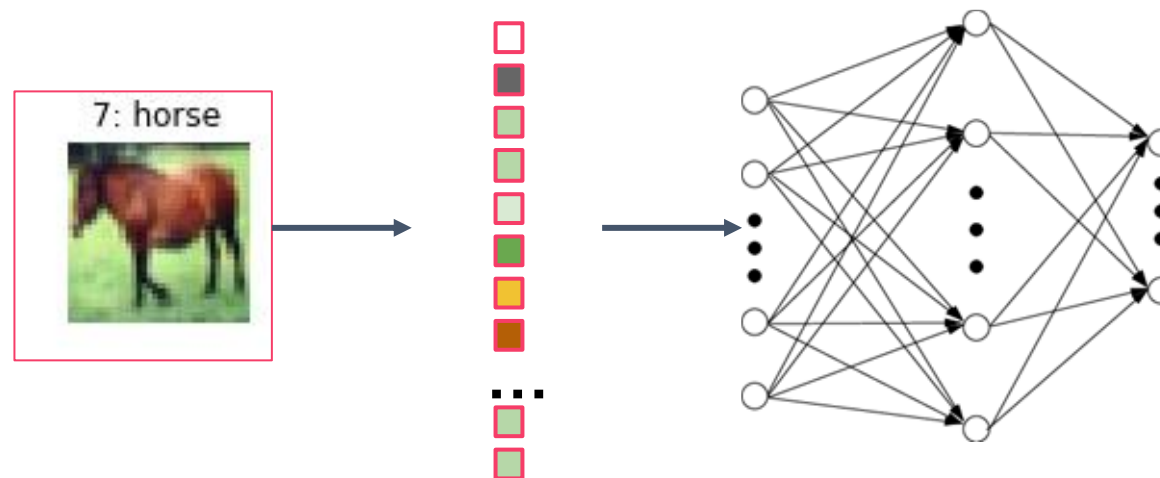
- Las arquitecturas convencionales de redes neuronales tienen limitaciones para llevar a cabo tareas de inferencia sobre datos visuales (imágenes)
 - No se les da bien capturar y aprender estructuras y patrones “espaciales” o de textura, color, etc. en imágenes.



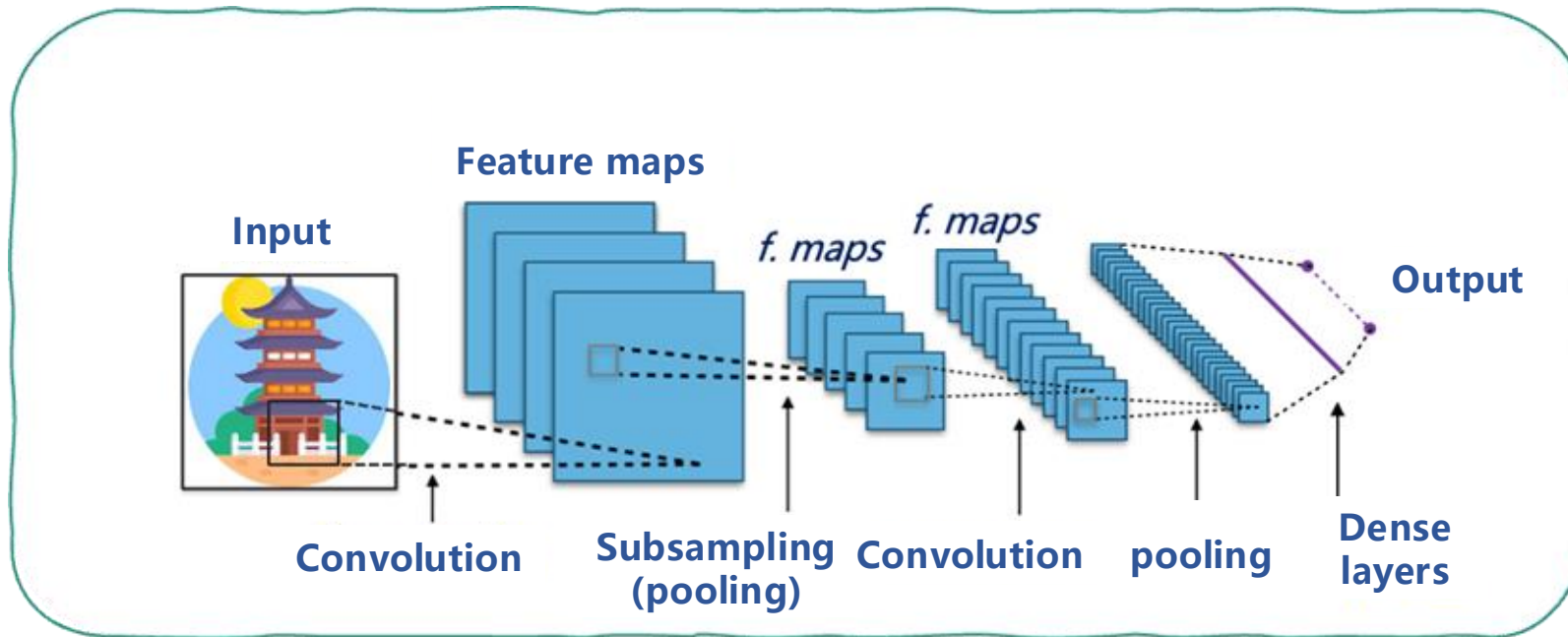
CIFAR-10: dataset
para clasificación de
imágenes

REDES NEURONALES CONVOLUTIVAS (CNN)

- Las arquitecturas convencionales de redes neuronales tienen limitaciones para llevar a cabo tareas de inferencia sobre datos visuales (imágenes)
 - No se les da bien capturar y aprender estructuras y patrones “espaciales” o de textura, color, etc. en imágenes.
 - Esa información espacial se pierde al “aplanar” la organización inicialmente bidimensional de los datos en píxeles



REDES NEURONALES CONVOLUTIVAS (CNN)



CNNs: inspiradas por el córtex visual humano, introducen en la arquitectura nuevos tipos de capas y operaciones especializadas en extraer patrones visuales

CONVOLUCIÓN

Porción 3x3 de imagen

0.6	0.4	0.6
0.1	-0.2	-0.3
-0.5	-0.4	-0.3

x

Filtro

1	1	1
0	0	0
-1	-1	-1

= 2.8

Porción 3x3 de imagen

-0.7	0.6	0.2
0.1	0.5	-0.3
-0.3	-0.4	0.5

x

1	1	1
0	0	0
-1	-1	-1

= -0.1

- Aplicar una convolución sobre una región cuadrada de una imagen consiste en aplicarle un filtro or *kernel*

- Producto a nivel de píxel
- Suma de los productos por píxel

Cuánto más similar es la porción de píxels al filtro aplicado, mayor es el resultado obtenido

FILTROS DE CONVOLUCIÓN HABITUALES

Detección de bordes
horizontales

1	1	1
0	0	0
-1	-1	-1

Detección de bordes
verticales

1	0	-1
1	0	-1
1	0	-1



FILTROS DE CONVOLUCIÓN HABITUALES

Detección de bordes horizontales

1	1	1
0	0	0
-1	-1	-1

Detección de bordes verticales

1	0	-1
1	0	-1
1	0	-1

Difuminado: se aplica a la región el promedio de las intensidades de sus píxeles

$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$

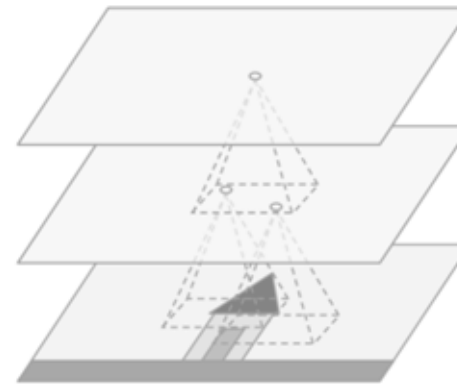


x

CAPA DE CONVOLUCIÓN

Es una **capa no densa**, donde cada neurona recibe conexiones de entrada únicamente desde una región rectangular de la capa previa: **campo receptivo**

Una neurona en la capa de convolución recibiendo conexiones de un campo receptivo 3x3 en la capa previa.

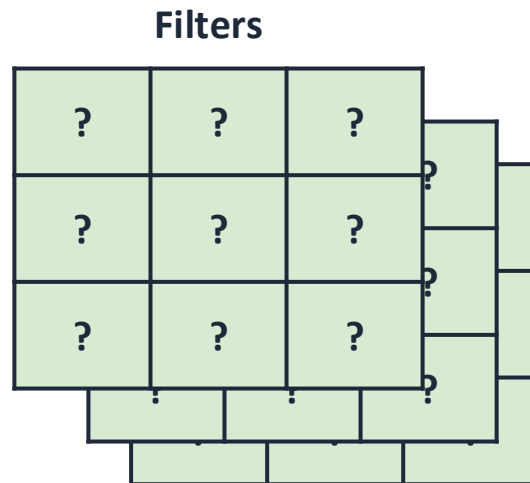


Las primeras capas convolutivas aprenden patrones de bajo nivel en la imagen, que en posteriores convoluciones se irán combinando en características de más alto nivel

CAPA DE CONVOLUCIÓN

Una capa de convolución consta de $N \geq 1$ filtros, cuyos valores son los ***pesos*** a aprender por estas capas durante el entrenamiento training

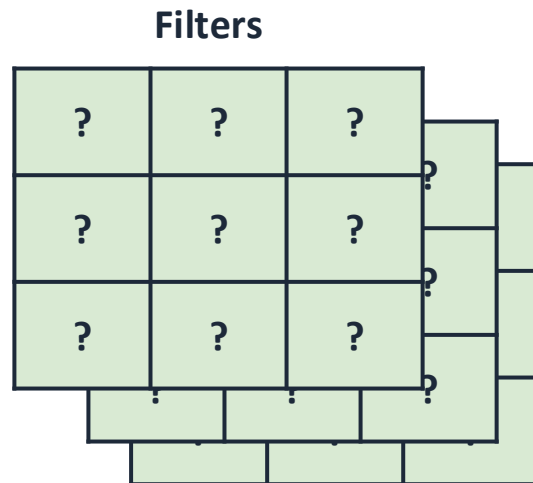
- Cada filtro aprendido “barre” el tensor asociado a una imagen y se va especializando en capturar un tipo de característica visual: formas, colores, bordes, ...



CAPA DE CONVOLUCIÓN

Una capa de convolución consta de $N \geq 1$ filtros, cuyos valores son los **pesos** a aprender por estas capas durante el entrenamiento training

- Cada filtro aprendido “barre” el tensor asociado a una imagen y se va especializando en capturar un tipo de característica visual: formas, colores, bordes, ...



Convolution layer with two filters, each one of size 3x3

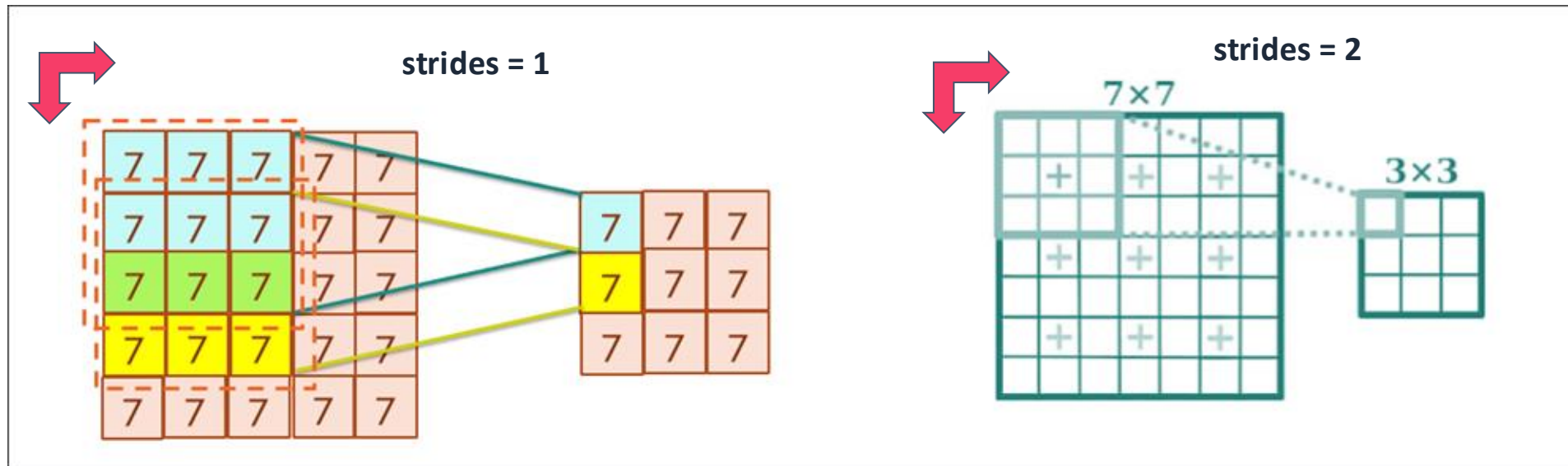
```
from tensorflow.keras import layers

input_layer = layers.Input(shape=(64, 64, 1))
conv_layer_1 = layers.Conv2D(
    filters = 2,
    kernel_size = (3, 3),
    strides = 1,
    padding = "same"
)(input_layer)
```

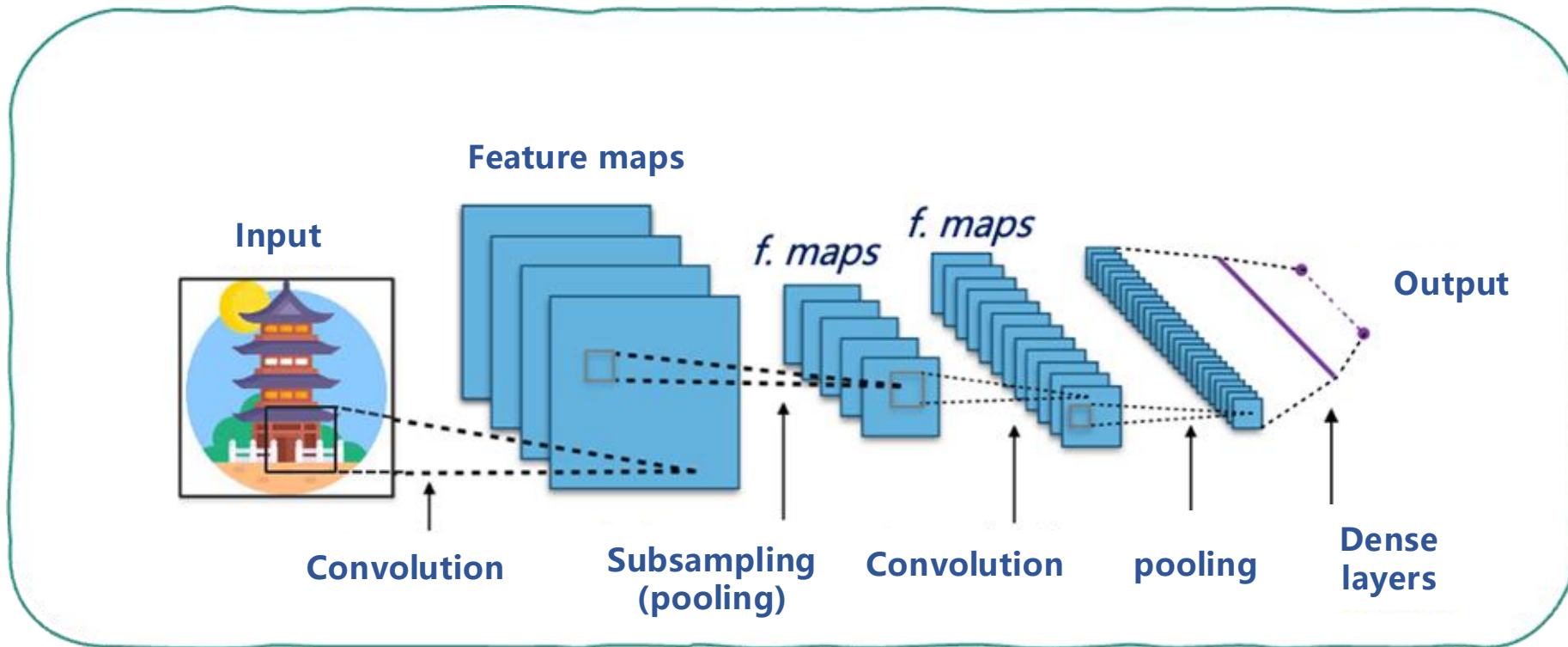
CAPA DE CONVOLUCIÓN: STRIDE Y PADDING

Stride: desplazamiento entre los dos campos receptivos conectados a dos neuronas adyacentes en la siguiente capa

- Un mayor *strides* implica menos “saltos”, y un tamaño menor en el tensor resultante en la capa posterior.



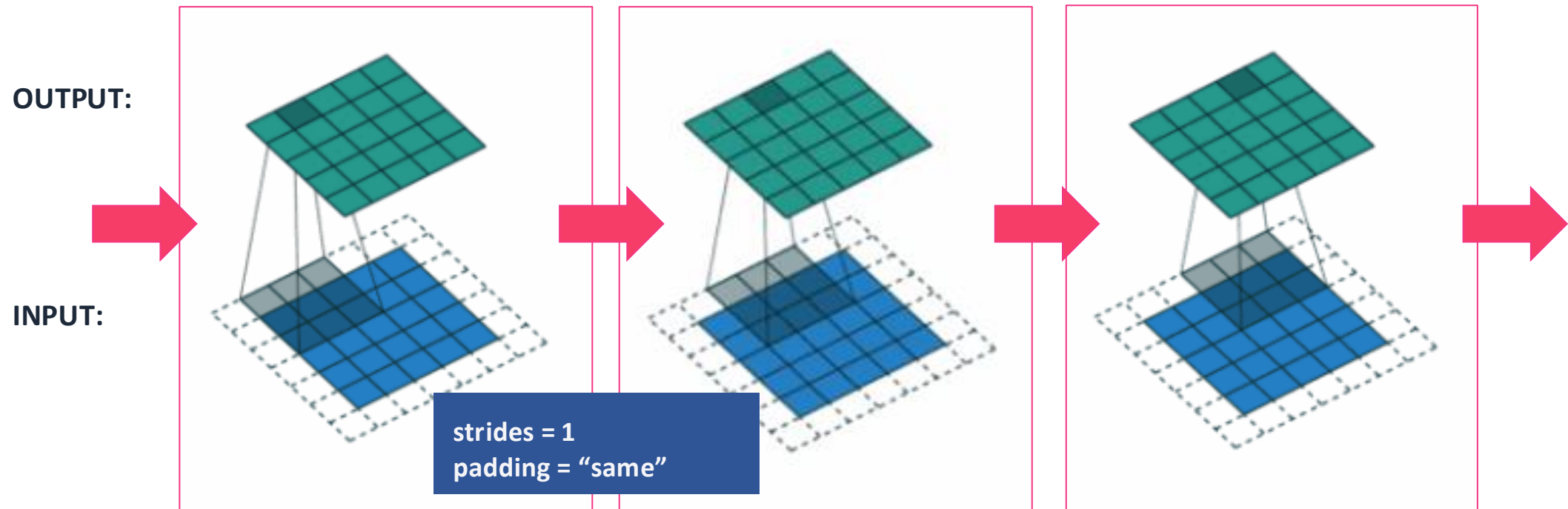
CAPA DE CONVOLUCIÓN: STRIDE Y PADDING



Los **filtros y strides** aplicados a lo largo de varias **capas de convolución consecutivas** hacen que los tensores con información que se propaga vayan siendo cada vez más pequeños, mientras que el **número de canales** (niveles de profundidad) **aumenta**

CAPA DE CONVOLUCIÓN: STRIDE Y PADDING

Padding: usar *padding="same"* para rellenar con 0s los bordes de un tensor que sirve de entrada a una capa de convolución, **ayudar a alcanzar ayuda a alcanzar un tensor de salida de igual tamaño** cuando *strides = 1*



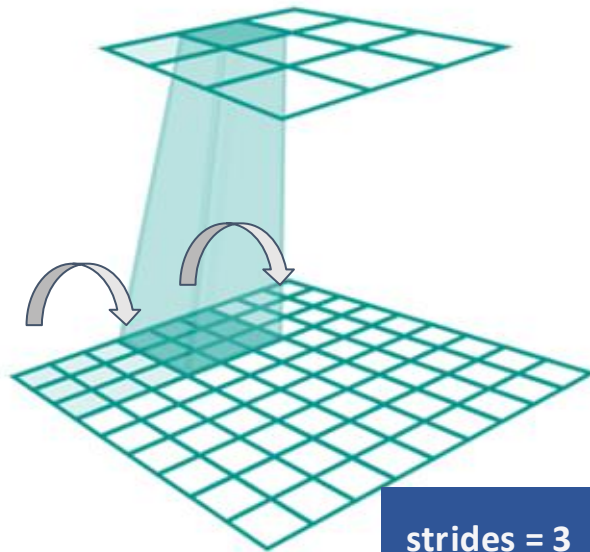
CAPA DE CONVOLUCIÓN: STRIDE Y PADDING

Padding: cuando usamos $strides = n > 1$ y $padding = "same"$ es posible controlar el tamaño del tensor que va obteniéndose en cada etapa de la red

$$\text{Output size} = \left(\frac{\text{input height}}{\text{stride}}, \frac{\text{output height}}{\text{stride}}, \#filters \right)$$

OUTPUT:

INPUT:



$strides = 3$
 $padding = "same"$

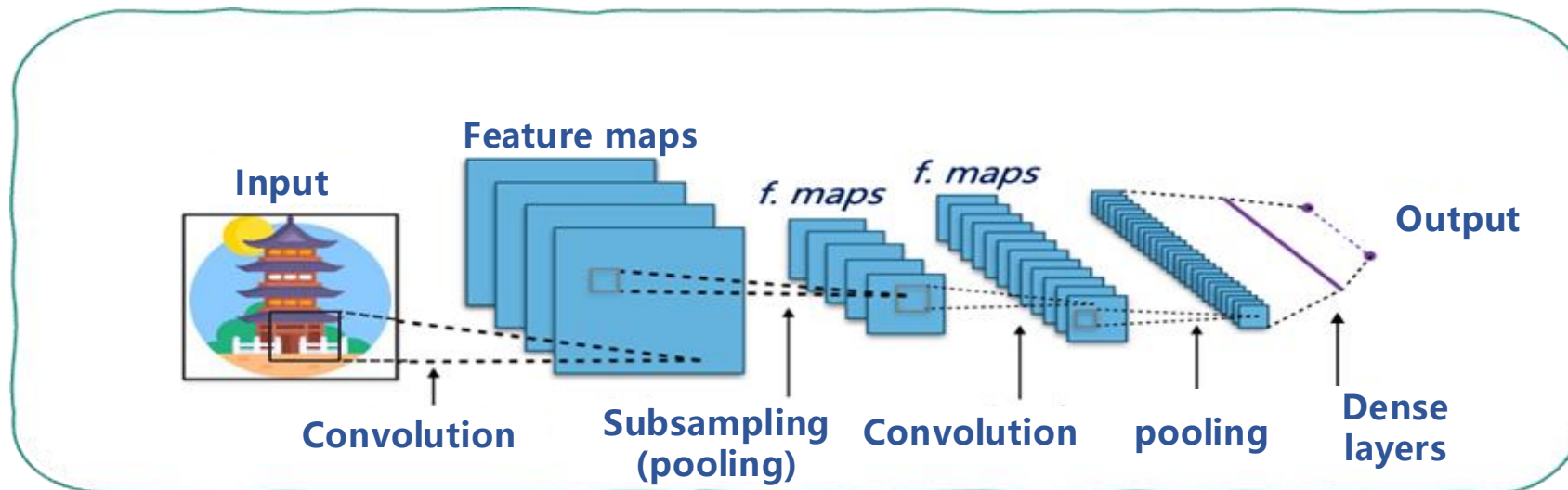
```
from tensorflow.keras import layers

input_layer = layers.Input(shape=(9,9,1))
conv_layer_1 = layers.Conv2D(
    filters = 2,
    kernel_size = (3,3),
    strides = 3,
    padding = "same"
)(input_layer)
```

CAPAS DE MUESTREO (POOLING LAYERS)

Pooling layer: se añaden entre capas de convolución consecutivas para reducir gradualmente el tamaño de la información procesada (*feature maps*)

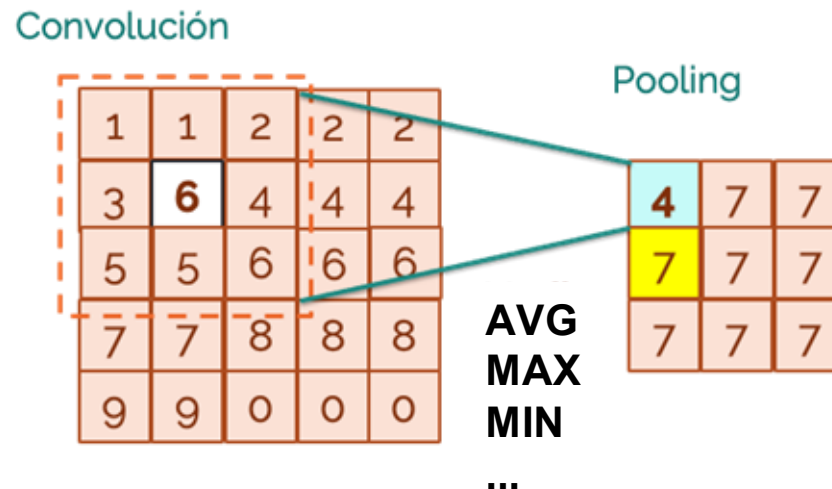
- Alternativa a un *strides* > 1 que puede ayudar a preservar mejor información importante que se esté aprendiendo
- **Opciones de agregación para el pooling:** *Max Pooling, Average Pooling, ...*



CAPAS DE MUESTREO (POOLING LAYERS)

Pooling layer: se añaden entre capas de convolución consecutivas para reducir gradualmente el tamaño de la información procesada (*feature maps*)

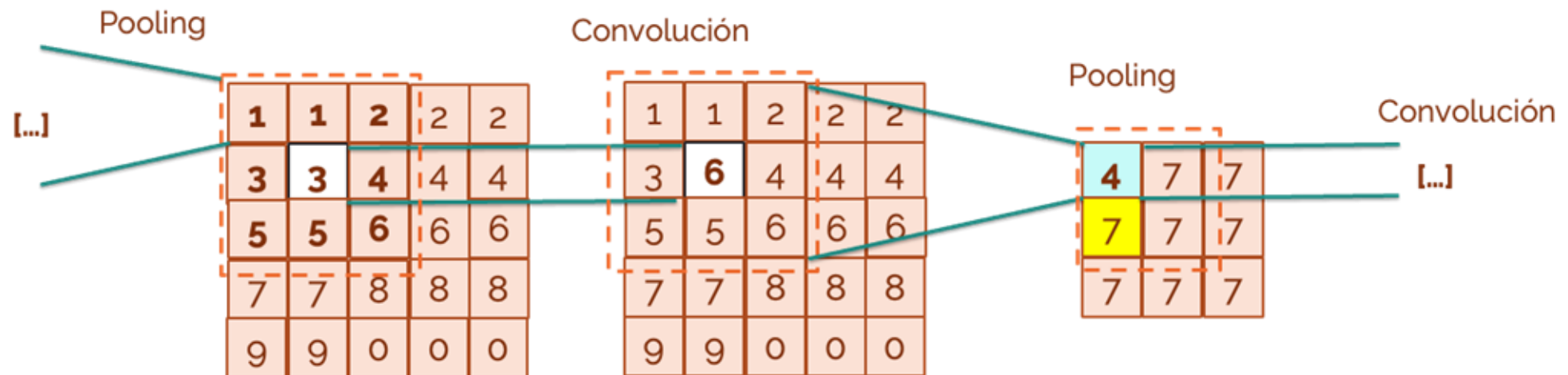
- Alternativa a un *strides* > 1 que puede ayudar a preservar mejor información importante que se esté aprendiendo
- **Opciones de agregación para el pooling:** *Max Pooling*, *Average Pooling*, ...



CAPAS DE MUESTREO (POOLING LAYERS)

Pooling layer: se añaden entre capas de convolución consecutivas para reducir gradualmente el tamaño de la información procesada (*feature maps*)

- Alternativa a un *strides* > 1 que puede ayudar a preservar mejor información importante que se esté aprendiendo
- **Opciones de agregación para el pooling:** *Max Pooling*, *Average Pooling*, ...



CAPAS DE MUESTREO (POOLING LAYERS)

Pooling layer over a convolution layer *in* Keras (using functional API)

```
from tensorflow.keras import layers

input_layer = layers.Input(shape=(64, 64, 1))
conv_layer_1 = layers.Conv2D(
    filters = 2,
    kernel_size = (3, 3),
    strides = 1,
    padding = "same"
)(input_layer)

pooling_layer = layers.MaxPooling2D(pool_size=(2, 2))(conv_layer_1)
# Alternatively we can use AveragePooling2D
```

PONGÁMOSLO TODO JUNTO PARA IMÁGENES A COLOR

Imágenes a color: píxel con 3 valores = **3 canales iniciales:** RGB.

Salida de una capa de convolución: un tensor 4-dimensional con esta forma:

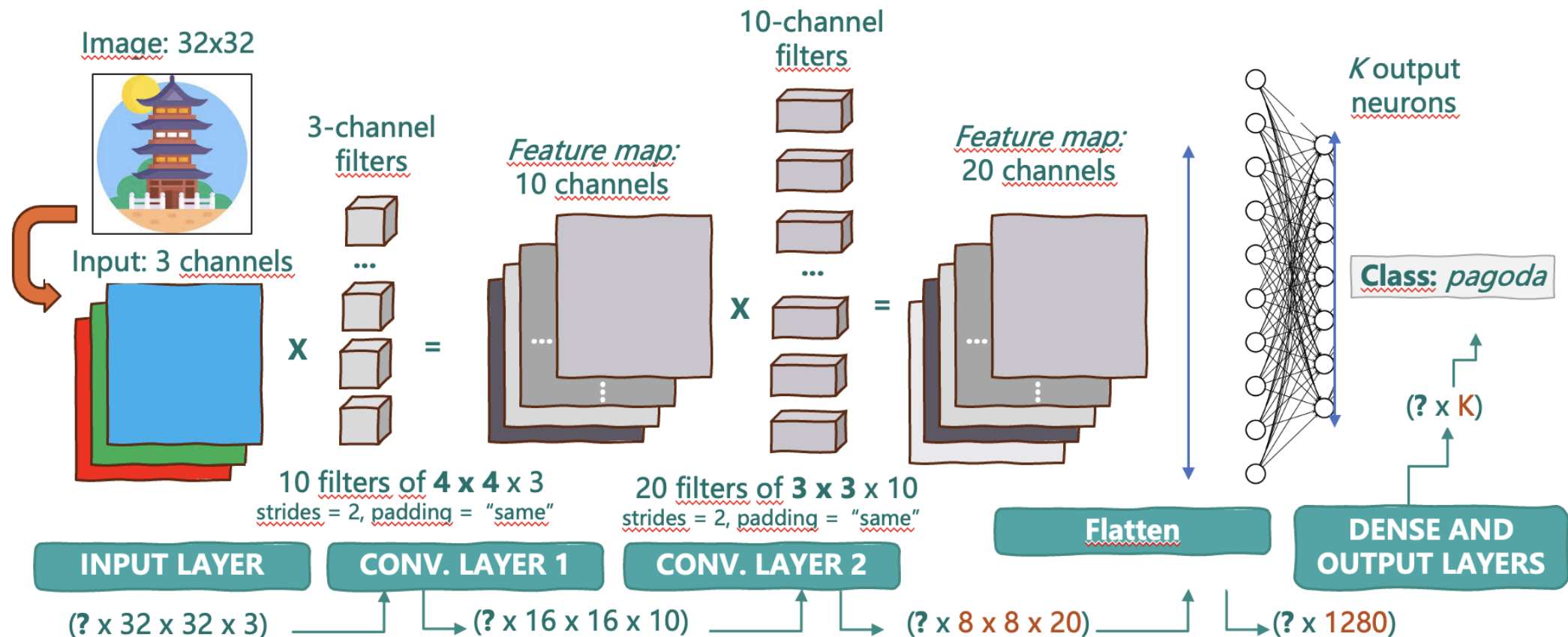
`(batch_size, height, width, filters)`

Consideraciones:

- Aumentar gradualmente el tamaño de los filtros de convolución ayuda a capturar primero detalles de bajo nivel, seguidos de detalles de más alto nivel, pero cómo/cuánto ir aumentándolo dependerá de cada problema.
- Normalmente, **el número de filtros por capa aumenta a medida que la información avanza por la red** para así ir capturando patrones cada vez más complejos.

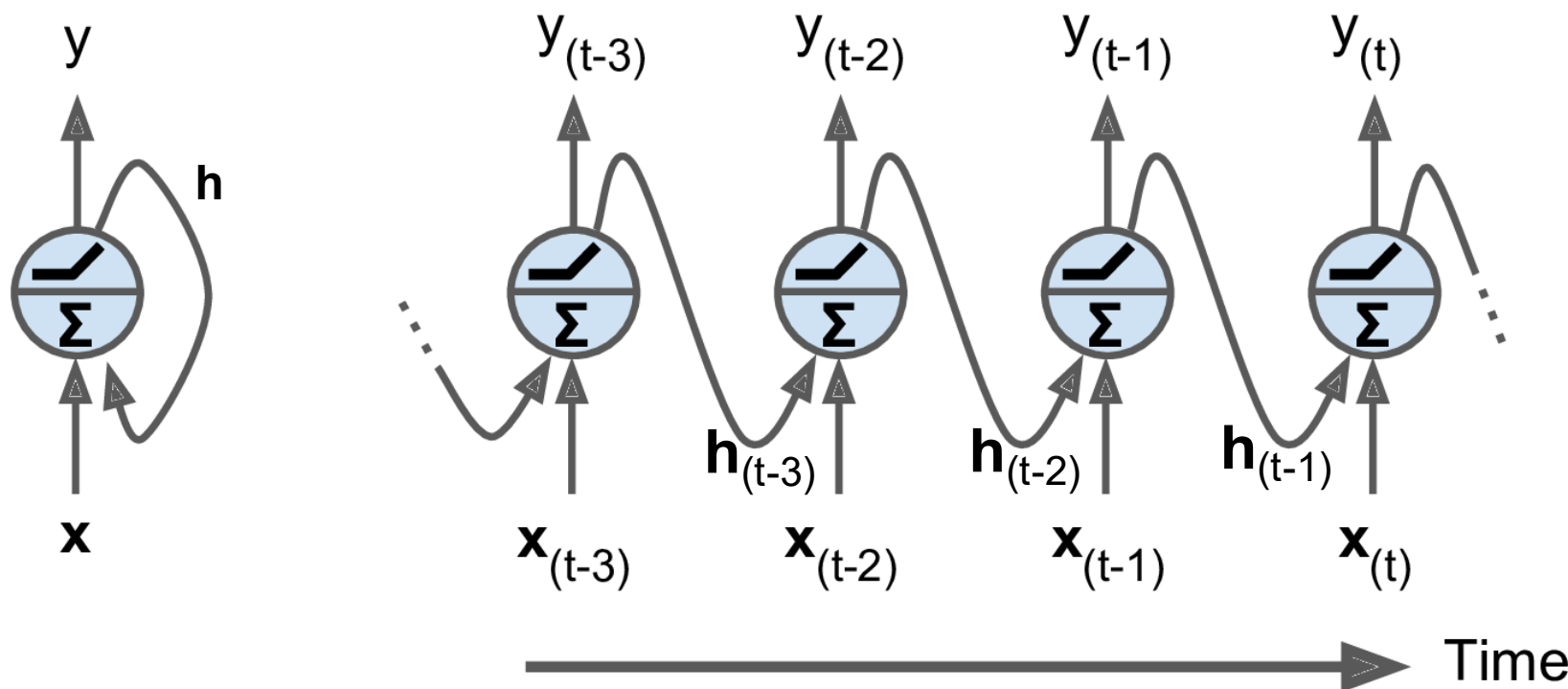
PONGÁMOSLO TODO JUNTO PARA IMÁGENES A COLOR

Salida de la CNN: dependerá de la tarea: clasificación, regresión, detección de objetos,



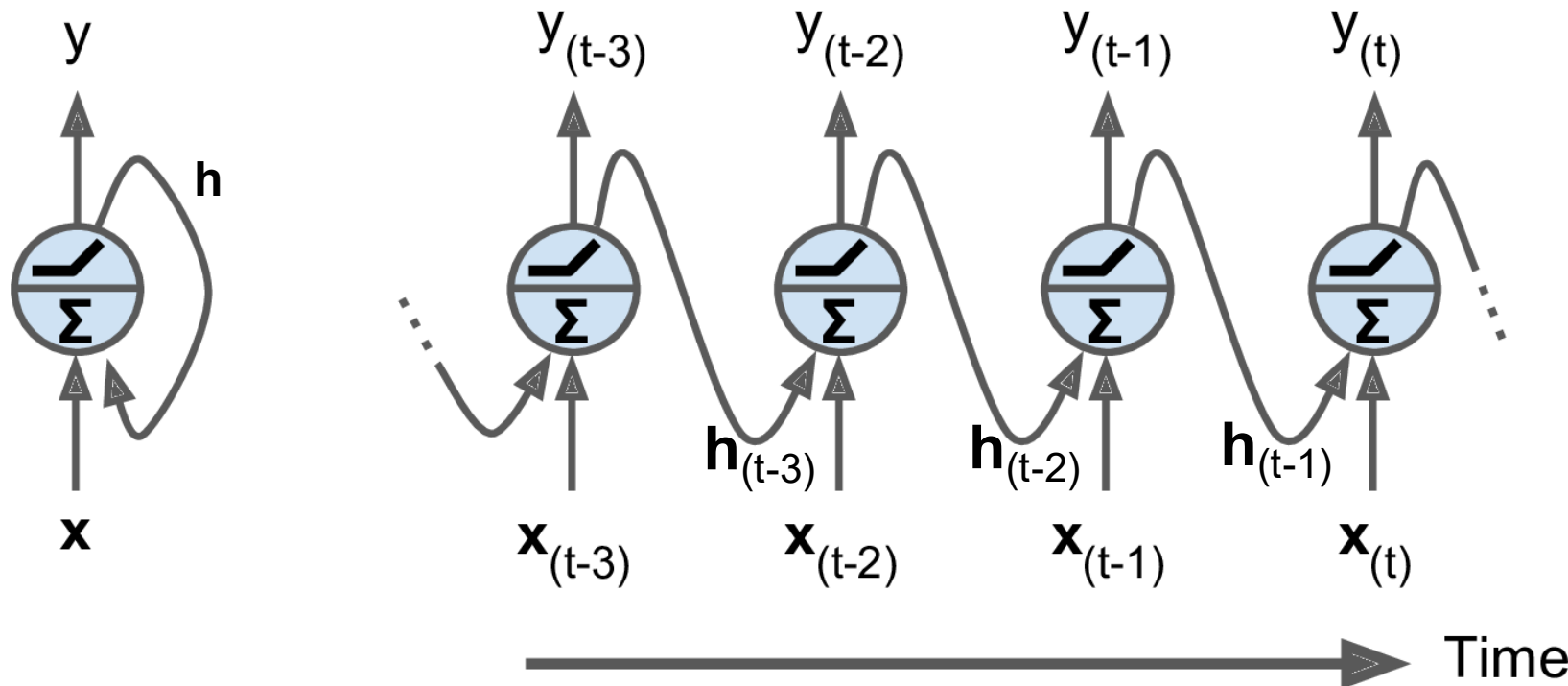
Redes Neuronales Recurrentes (RNNs)

Las neuronas recurrentes introducen conexiones que “apuntan” hacia atrás, así como la noción de estado oculto (**hidden state**). Estas neuronas se conocen también como *memory cells*, pues retienen una “memoria” de información pasada que fluyó por ellas.



Redes Neuronales Recurrentes (RNNs)

Las neuronas recurrentes introducen conexiones que “apuntan” hacia atrás, así como la noción de estado oculto (**hidden state**). Estas neuronas se conocen también como *memory cells*, pues retienen una “memoria” de información pasada que fluyó por ellas.



El estado de una célula en el instante t , $h_{(t)}$, es una función de sus entradas en t , y su previo estado $h_{(t-1)}$ en $t-1$

$$h_{(t)} = f(x_{(t)}, h_{(t-1)})$$

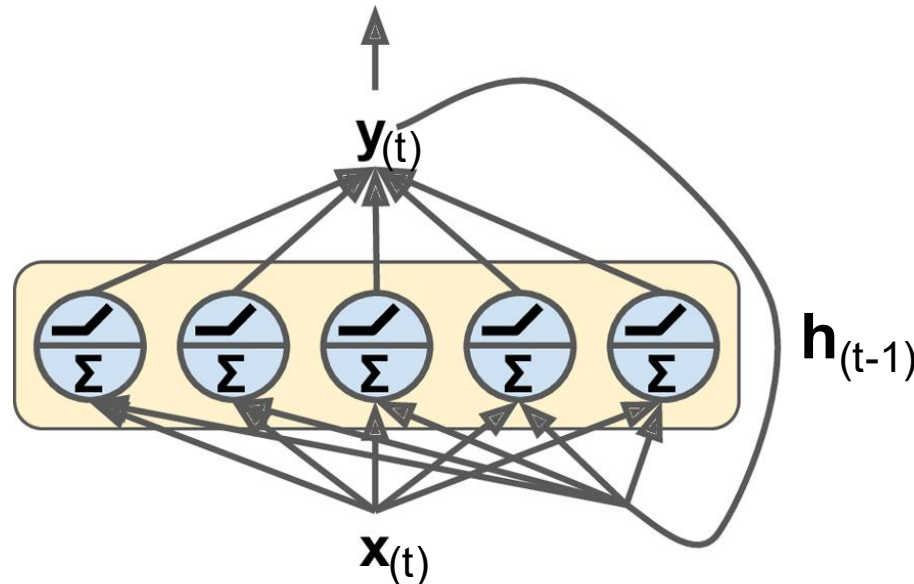
En células sencillas, $y_{(t)} = h_{(t)}$, pero en otras más complejas el cálculo puede cambiar

Redes Neuronales Recurrentes (RNNs)

Las neuronas tradicionales (*feed-forward*) tienen asociado un vector de pesos dado por sus conexiones entrantes.

- Así, una capa con n neuronas tendría asociada una matriz de pesos.

Las neuronas y capas recurrentes tienen asociados **dos vectores (o matrices) de pesos**: uno para información entrante ahora (t) y otro para información del estado previo ($t-1$).

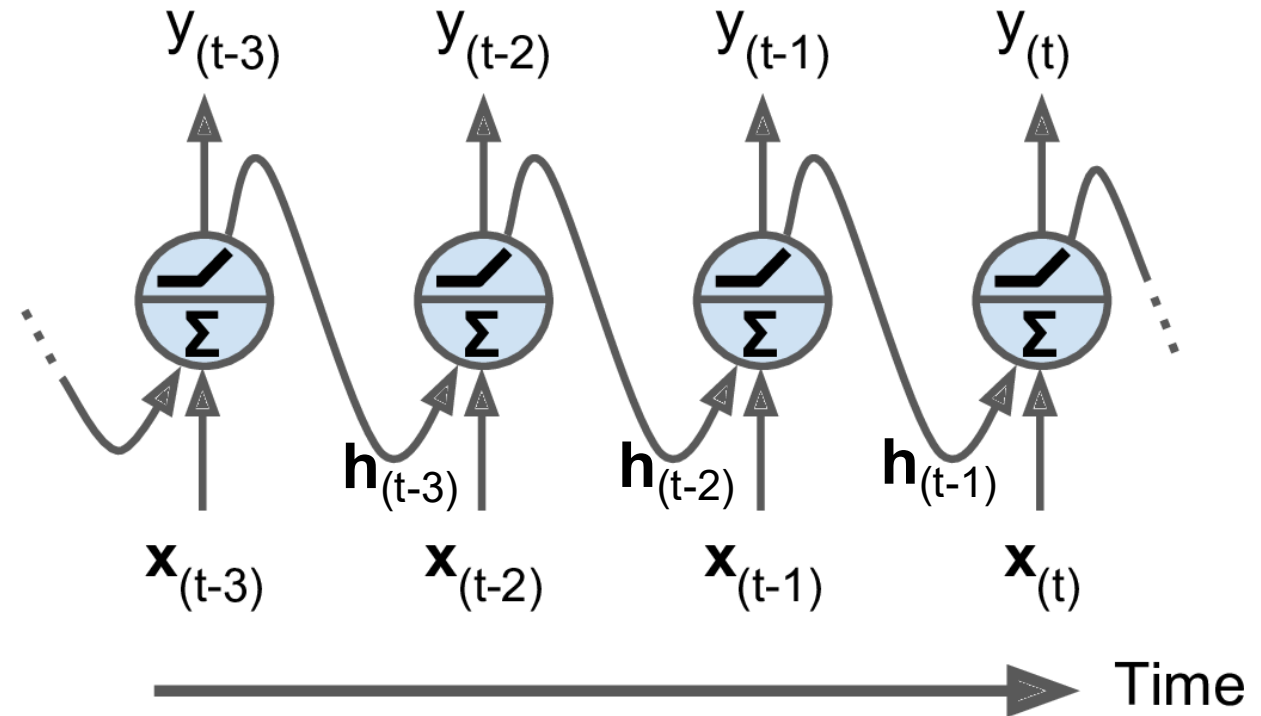


RNNs: Series Temporales

Una RNN toma una secuencia de entradas y devuelve una secuencia de salidas. Esto las hace aptas para problemas como:

- Predecir consumo energético diario
- Predecir ventas la próxima semana
- Pronosticar lluvia
- *Predecir la próxima palabra en una consulta de un buscador (NLP)*

Muchos de estos problemas se basan en datos de series temporales



RNNs: Series Temporales

Tareas habituales sobre series temporales:

- Pronóstico → más común
- Clasificación: clasificar una secuencia temporal en varias posibles clases.
 - Finance: classify a 1-month stock market time series
- Detección de anomalías
 - Energía: detectar picos peligrosos en estación de suministro.
- Imputación: rellenar valores perdidos pasados
 - IoT: un sensor temporalmente averiado del número de pasajeros por hora en una estación de metro



RNNs: Series Temporales

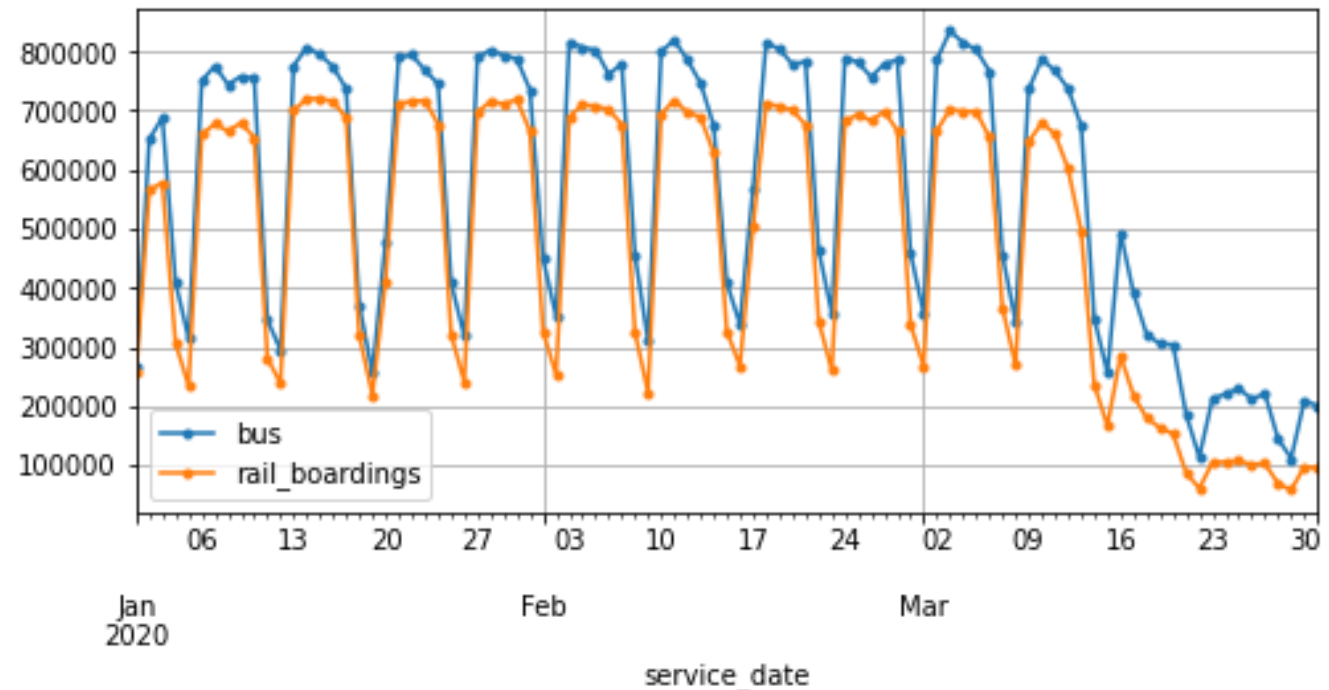
Una colección de observaciones recolectadas en una serie de instantes o periodos fijos temporales (p.ej. días).

- El orden temporal es fundamental.
- **Serie temporal univariante:** solo un valor o atributo por instante t .
 - $\langle time, bus_rides \rangle$
- **Serie temporal multivariante:** dos o más valores (atributos) por instante t
 - $\langle time, bus_rides, train_rides \rangle$

Chicago daily rides dataset

Contiene atributos que describen el número diario de pasajeros en distintos medios de transporte urbano

Link: <https://data.cityofchicago.org/Transportation/CTA-Ridership-Daily-Boarding-Totals/6iiy-9s97>



Construcción de RNNs

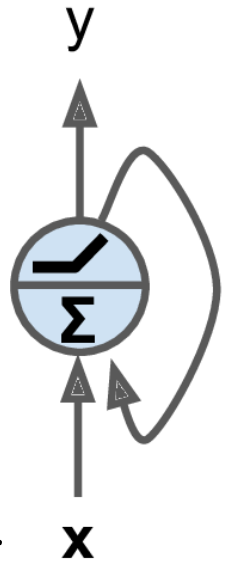
Antes de crear una RNN es necesario preparar debidamente los datos (en este caso, datos con forma de *time series*).



```
model = tf.keras.Sequential([  
    tf.keras.layers.SimpleRNN(1, input_shape=[None, 1])  
])
```

El tipo más sencillo de RNN: solo una capa con una neurona.

La función de activación por defecto para neuronas recurrentes es $\tanh(x)$



Construcción de RNNs

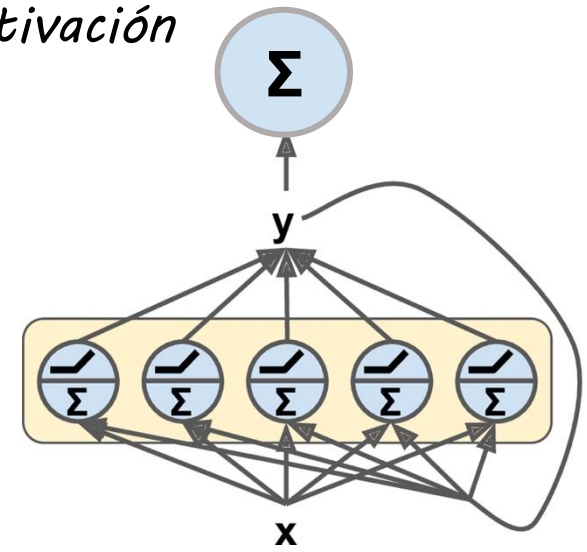
Antes de crear una RNN es necesario preparar debidamente los datos (en este caso, datos con forma de *time series*).



```
model = tf.keras.Sequential([  
    tf.keras.layers.SimpleRNN(1, input_shape=[None, 1])  
])
```

```
model2 = tf.keras.Sequential([  
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 1]),  
    tf.keras.layers.Dense(1)])
```

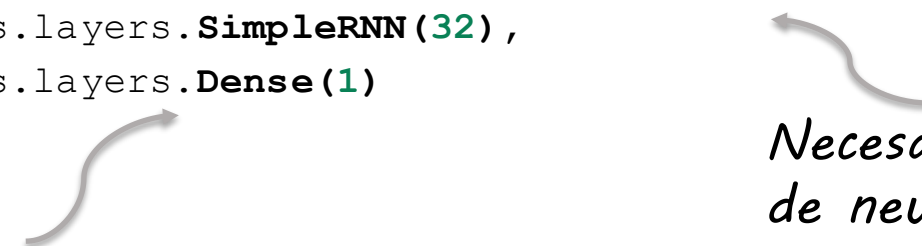
Modelo un poco más complejo: dos capas, la primera con 32 neuronas recurrentes, y una única neurona de salida sin función de activación



Construcción de RNNs

Antes de crear una RNN es necesario preparar debidamente los datos (en este caso, datos con forma de *time series*).

```
deep_model = tf.keras.Sequential([  
    tf.keras.layers.SimpleRNN(32, return_sequences=True, input_shape=[None, 1]),  
    tf.keras.layers.SimpleRNN(32, return_sequences=True),  
    tf.keras.layers.SimpleRNN(32),  
    tf.keras.layers.Dense(1)  
])
```



Hay varios tipos de capas recurrentes en Keras. SimpleRNN es la básica, pero hay otras con una lógica interna más sofisticada, como *LSTM* y *GRU*

Necesario en Keras cuando apilamos varias capas de neuronas recurrentes (excepto en la última) para devolver una secuencia de salidas → entradas para la siguiente capa, en lugar de devolver salidas aisladas

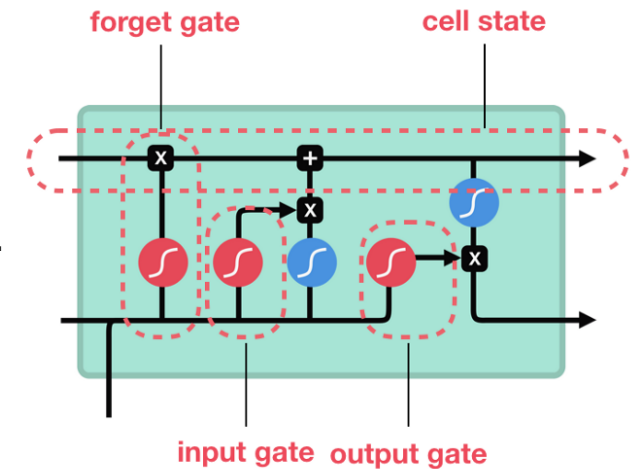
Arquitecturas RNN Populares: LSTM (Long-Short Term Memory)

Las RNNs tradicionales sufren de una memoria muy “a corto plazo”: olvidan la información de varios instantes atrás cuando procesan secuencias largas.

- El problema del **desvanecimiento de gradientes** anteriormente estudiado contribuye a esto.

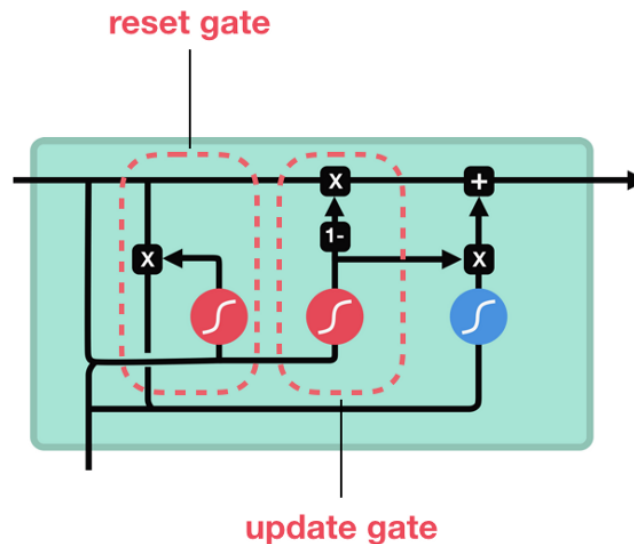
Las células LSTM y GRU fueron creadas para mejorar la “capacidad de memorización” de neuronas recurrentes:

- Tienen mecanismos sofisticados internos, basados en puertas lógicas.
- Estas puertas aprenden qué información es relevante para retener y cuál no → muy útil en tareas de NLP que precisan identificar palabras clave en texto y voz.
- Las entradas y salidas son las mismas que en neuronas recurrentes clásicas. Lo que cambia son los cálculos internos.



Arquitecturas RNN Populares: GRU (Gated Recurrent Unit)

- Las células GRU tienen menos puertas dentro, siendo algo menos complejas que las LSTM.
- GRU es preferible para datasets menores y resultados eficientes, LSTM es mejor para mayores datasets y para aprender de secuencias largas
- GRU tiene una “puerta de reseteo” que decide en el instante t si olvidar o no información memorizada de instantes previos pasados.

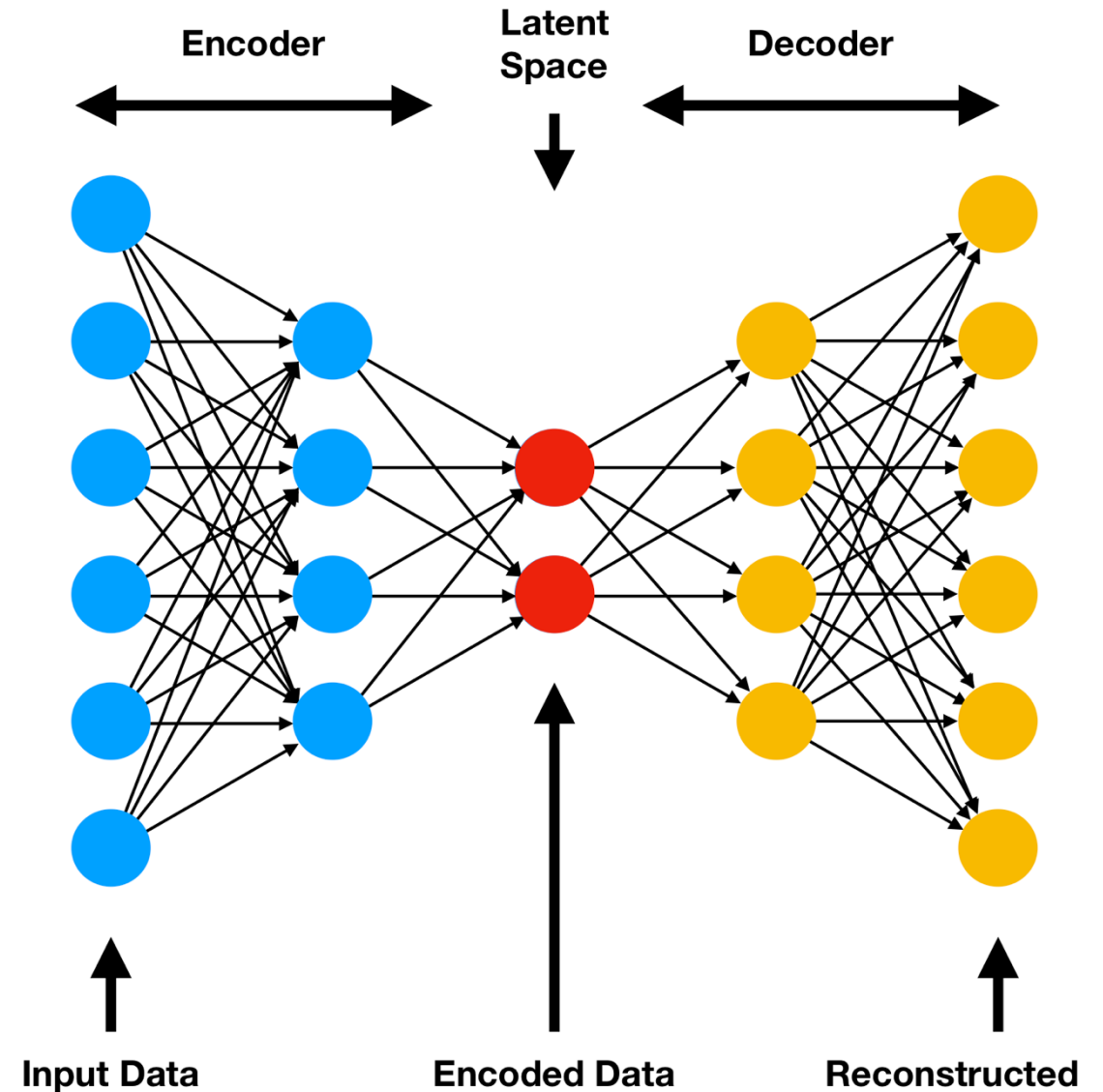


Arquitecturas Deep Learning Generativas: Autoencoder

Específicamente diseñada para tareas de **aprendizaje no supervisado**

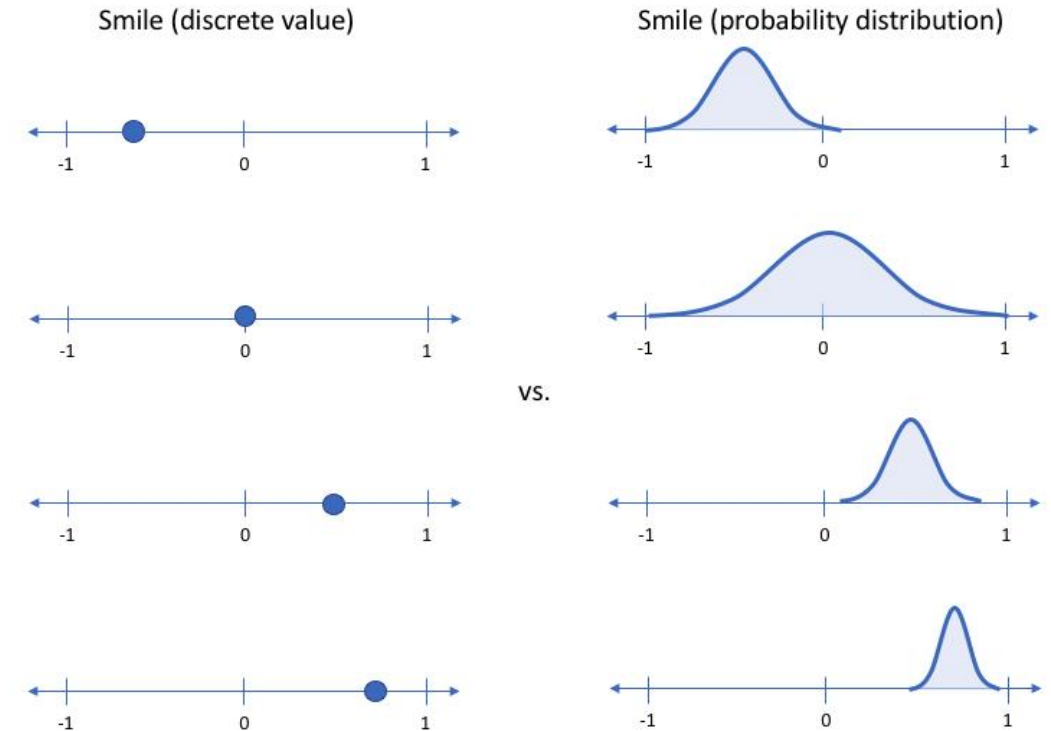
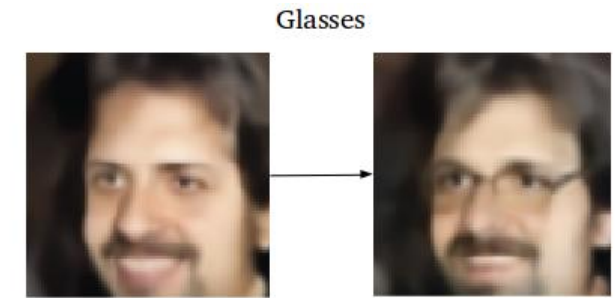
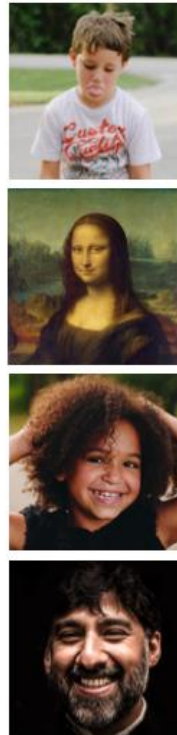
- Extraer información relevante de los datos de entrada, descartando detalles irrelevantes.
- Representación de información comprimida → procesamiento de imágenes, modelado de clientes, sistemas de recomendación...
- Puede ser comparable a métodos de reducción de dimensionalidad como PCA
- Estructura encoder-decoder → la información original puede ser “reconstruida” con el *decoder* (comportamiento generativo)

Diferencia con otras arquitecturas encoder-decoders: la salida final decodificada ha de ser similar a la propia entrada inicial



Autoencoder variacional

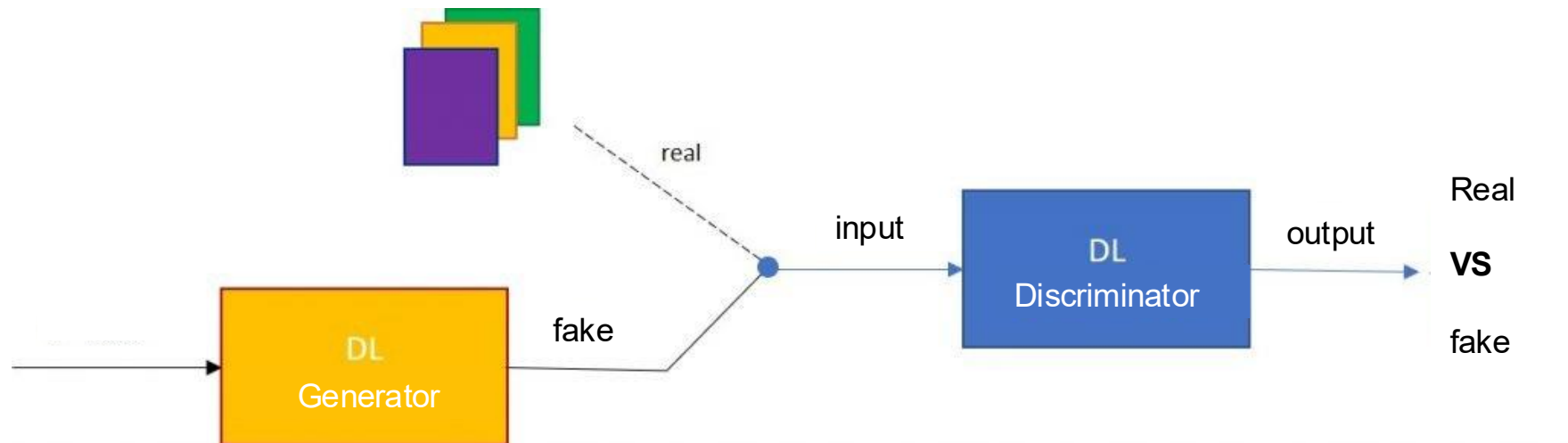
- Basados en inferencia variacional (subárea de la estadística)
- Inferir distribuciones continuas de los atributos en los datos de entrenamiento
- Generar $\langle \text{mean}, \text{std} \rangle$ de una distribución gaussiana, a partir de la cual crear ejemplos sintéticos
- Crear datos ficticios que se parecen mucho a los originales, pero con diferencias
- Usadas como modelos generativos



Redes Generativas Adversariales (GANs)

Usadas popularmente para generar imágenes y música no reales que parecen reales
Un modelo GAN es doble: consiste en dos ANNs, que compiten una contra la otra con objetivos opuestos

- *Generator*: su objetivo es generar instancias falsas de contenido que parezcan reales.
- *Discriminator*: debe decidir si una instancia— p.ej. Una imagen — es real o no.

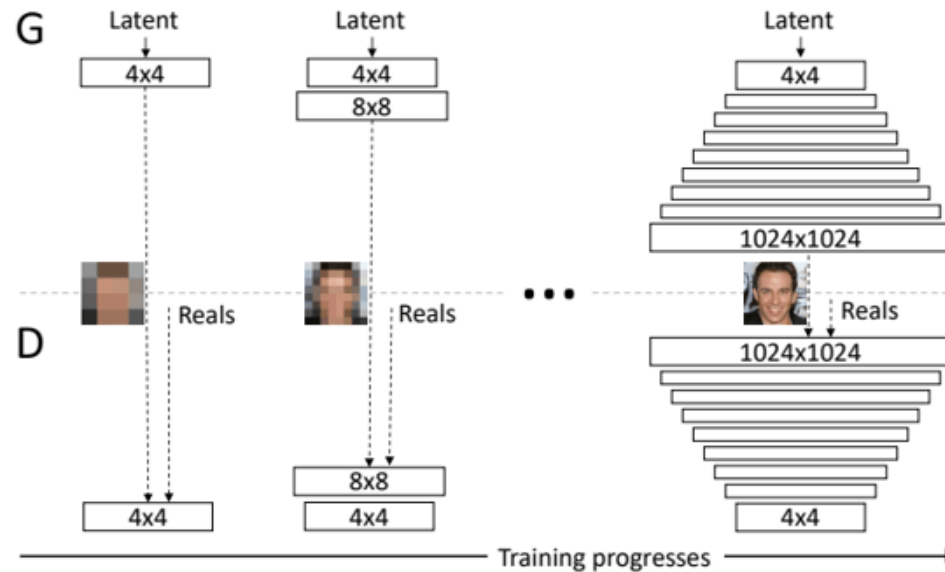


GANs: Generative adversarial networks

A medida que el generador mejora en la tarea de generar datos falsos, el discriminador empieza a tener dificultades para distinguir lo que es real de lo que es falso, viéndose forzado a mejorar también



Progressive GANs aprenden gradualmente a generar contenido falso de complejidad cada vez mayor (imágenes de mayor resolución, por ejemplo)



Aplicaciones de las Arquitecturas Generativas en Finanzas y Banca

- **Autoencoder y VAEs:** estas arquitecturas cumplen una importante función como sistemas de detección de anomalías (otra tarea de aprendizaje no supervisado).
 - En banca, por ejemplo, se usan para **detectar fraude** (tarjetas de crédito, blanqueo de capitales, transacciones atípicas).
- **GANs:** su carácter es más puramente generativo.
 - Pueden usarse para **crear perfiles de clientes y transacciones ficticias** que mantengan las propiedades estadísticas de datos reales confidenciales. Esto permite compartir datasets con investigadores o empresas colaboradoras sin violar la privacidad y regulaciones como RGPD.
 - La generación sintética de datos también puede servir para *balancear* datasets y ampliar datasets etiquetados para entrenar modelos supervisados.

Procesamiento de Lenguaje Natural (NLP)

NLP: Area de la IA que investiga comunicación hombre-máquina a través del lenguaje natural.

Algunas tareas de las que se ocupa el NLP...

- Clasificación de texto → análisis de sentimientos
- Predicción de la siguiente palabra/caracter
- Traducción
- Resumir texto
- Responder a preguntas
- Reconocimiento de voz

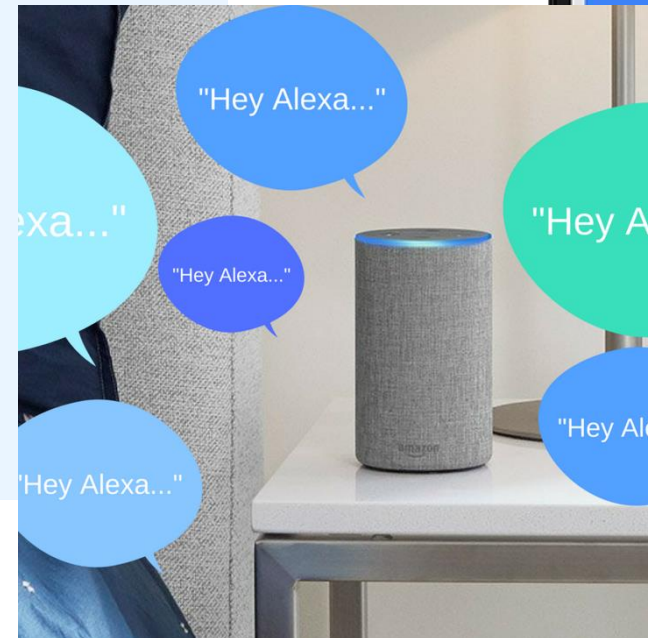
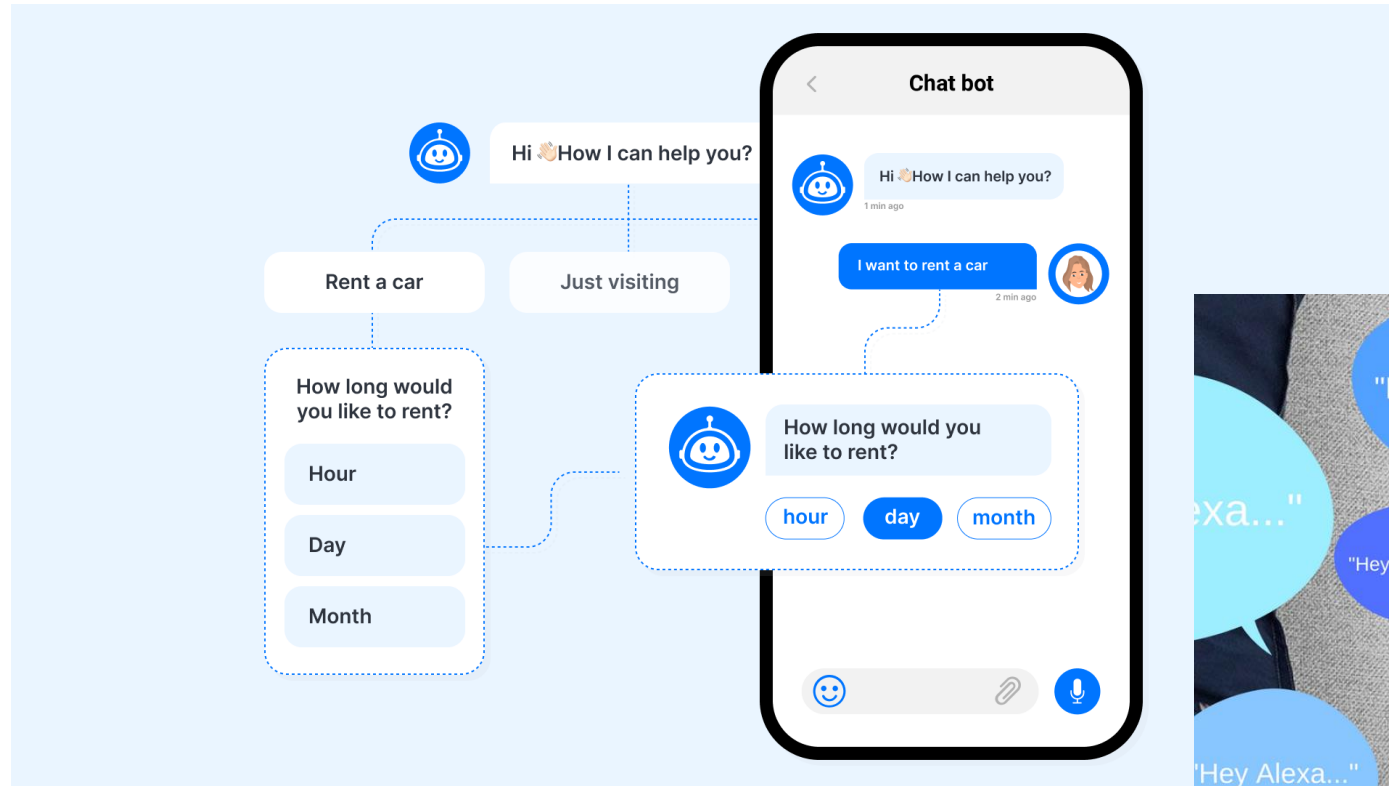
Aplicaciones comunes

- Chatbots
- Generación de texto y voz



Procesamiento de Lenguaje Natural (NLP)

Sistemas basados en NLP en la vida cotidiana



PLN: ¿Cómo representar el lenguaje humano computacionalmente?

Trabajar con lenguaje natural a nivel computacional requiere un proceso de modelado matemático para la representación numérica del mismo

*Los sistemas de IA **no entienden** el lenguaje humano, pero si entienden representaciones matemáticas de éste (tokenización, embeddings...).*



PLN: ¿Cómo representar el lenguaje humano computacionalmente?

- Antes de usar cualquier modelo, debemos convertir el texto en representaciones numéricas.
- Diferentes técnicas → diferentes propiedades (frecuencia, significado, contexto...).

Bag-of-Words (BoW)

- Representa el texto como un *conteo de palabras*.
- Se pierde el orden y el contexto.

EJEMPLO:

Vocabulario: ["movie", "great", "boring"]

Texto 1: "great movie" → [1, 1, 0]

Texto 2: "boring boring movie" → [1, 0, 2]

TF-IDF (Term Frequency – Inverse Document Frequency)

- La representación da más pesos a palabras que sean importantes en un documento, y menos frecuentes a nivel de colección de documentos.
- Los buscadores clásicos usaban este enfoque, así como modelos tempranos de clasificación de texto.

PLN: ¿Cómo representar el lenguaje humano computacionalmente?

Token IDs

- Cada token (palabras/subpalabra) → ID entero
- No capturan significado, solo orden y presencia de palabras

Ejemplo: **“I loved this movie”** → **[12, 584, 33, 7]**

Se suelen emplear tokens especiales para ciertas funciones de la representación del lenguaje:

ID	Token	¿Para qué sirve?
0	<PAD>	Rellenar secuencias para fijar misma longitud en todas
1	<START>	Marcar inicio del texto
2	<UNK>	Palabra fuera de vocabulario
3	<UNUSED>	Reservado, opcional

PLN: ¿Cómo representar el lenguaje humano computacionalmente?

Embeddings (Word2Vec, GloVe, LLM embeddings)

- Cada token ID pasa a convertirse en un vector numérico de valores aprendidos.
- Palabras con significado similar tendrán asociados vectores similares
- Permite que los modelos entiendan semántica a través de la similitud entre vectores

"king" → [0.27, -0.14, 0.88, ...]
"queen" → [0.29, -0.17, 0.90, ...]

Embeddings de token: permiten manejar vocabularios amplios (idiomas reales) y generalizar a palabras desconocidas.

Embeddings de secuencia (frase, párrafo, etc.): se puede obtener por la agregación de múltiples embeddings de token → útil para clustering de textos, clasificación de frases, etc.

- **Doc2vec:** modelo que produce directamente embeddings de secuencias.

Modelos de Lenguaje

Un **modelo de lenguaje** es un sistema capaz de comprender y generar texto de manera coherente a partir de patrones aprendidos.

Es un tipo específico de modelo de **NLP** cuyo propósito central es **predecir y generar secuencias de texto**, además de **comprenderlas** en cierto grado. Tienen una capacidad de **aprendizaje a nivel probabilístico** sobre el lenguaje.

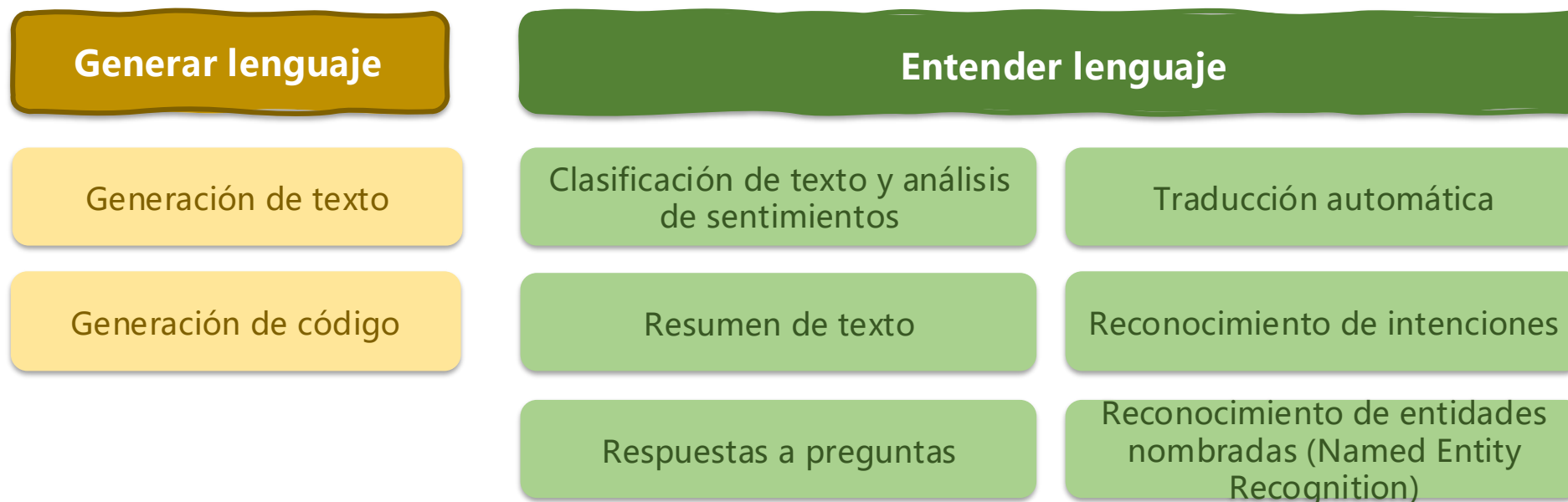
Sistema de NLP:

Conjunto más amplio de técnicas, modelos o herramientas diseñadas para realizar tareas sobre lenguaje natural (clasificación, resumen, extracción de entidades, análisis de sentimiento, traducción, etc.). Un sistema de NLP **puede usar o no un modelo de lenguaje** como parte de su arquitectura.

Modelos de Lenguaje

“Un **modelo de lenguaje** es un sistema capaz de comprender y generar texto de manera coherente a partir de patrones aprendidos.”

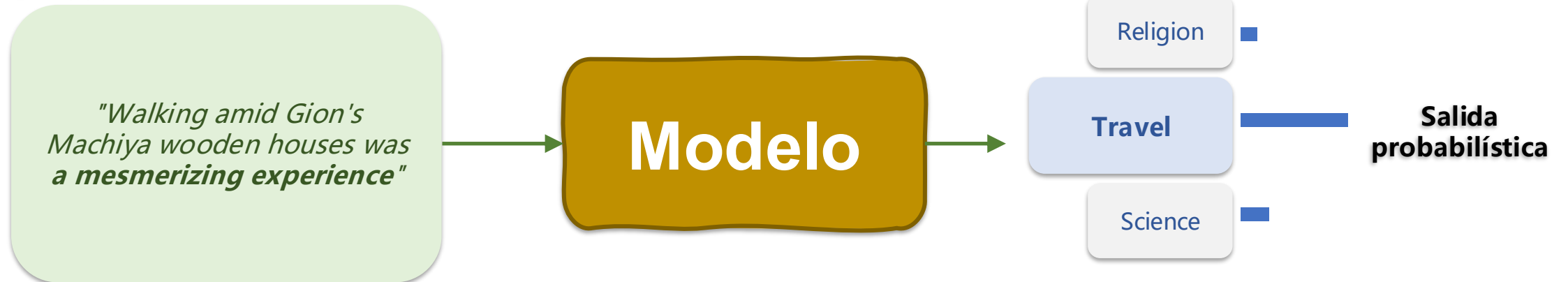
Es un tipo específico de **modelo de NLP** cuyo propósito central es **predecir y generar secuencias de texto**, además de **comprenderlas** en cierto grado. Tienen una capacidad de **aprendizaje a nivel probabilístico** sobre el lenguaje.



Clasificación de texto

La tarea en la que un modelo de lenguaje (u sistema de NLP de otro tipo) asigna una (o a veces varias) categoría predefinida a un texto

Entrada



Clasificación de texto: Análisis de Sentimientos

Una de las aplicaciones de NLP más habituales → un caso concreto de **clasificación de textos**.

Dado un documento o secuencia de texto (p.ej. una reseña), identificar el sentimiento asociado a la misma.

Caso más simple: *positivo vs negativo* → clasificación binaria

😊 · Everything was of top standard, nothing too much trouble , the decor throughout was lovely , clean and well maintained
Ali on reception, Chris with valet parking and Stephan our waiter were all exceptional hosts and made our stay most memorable , including a birthday cake in our room and a candle desert after dinner, all personal to us, thank you all



No sound absorbing materials, extremely loud. Party groups right next to people trying to dine. Fish the size of a thick cell phone battery. Good flatbread pesto. Excellent cinnamon ice cream. But never again.



Clasificación de texto: Análisis de Sentimientos

Otros escenarios de análisis de sentimientos:

- Positivo **vs** neutral **vs** negativo → clasificación bajo múltiples clases
 - *"I really like the new design of your website!"* → Positive
 - *"I'm not sure if I like the new design"* → Neutral
 - *"The new design is awful!"* → Negative
- Basado en aspectos o criterios → clasificación **multi-etiqueta** (múltiples variables de clase a predecir)

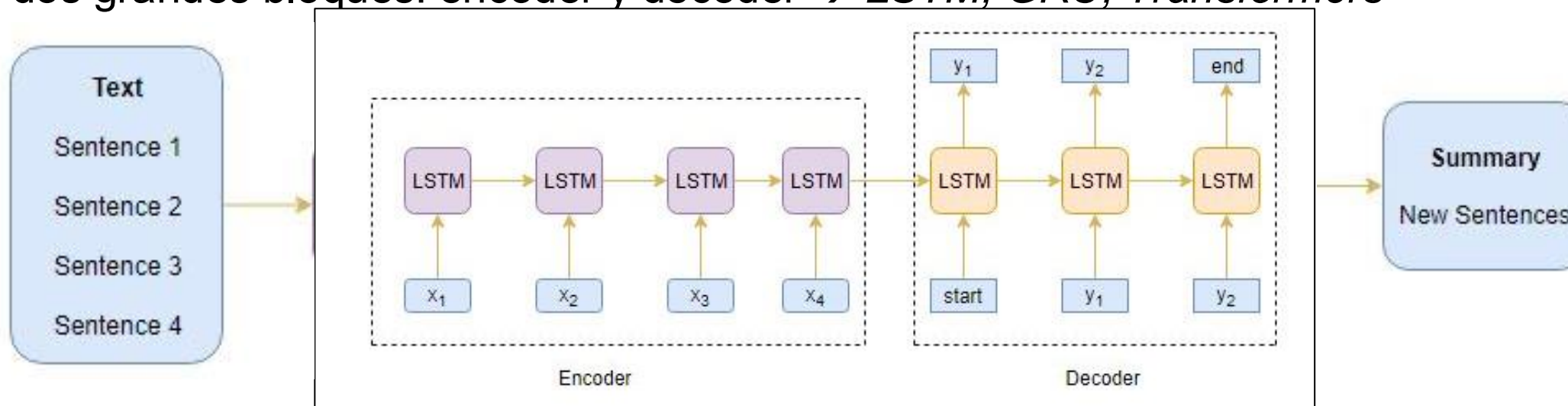
Opinion	Aspect	Sentiment
<i>"It's so easy to use. It looks less than a week to understand where everything is in Drift"</i>	UX-UI	Positive
<i>"The mobile app can be really glitchy and is definitely not user friendly"</i>	Mobile App	Negative
<i>"Their customer success team is amazing and there's always someone available from their support team on live chat to help you"</i>	Customer Service	Positive

Resumen de Texto

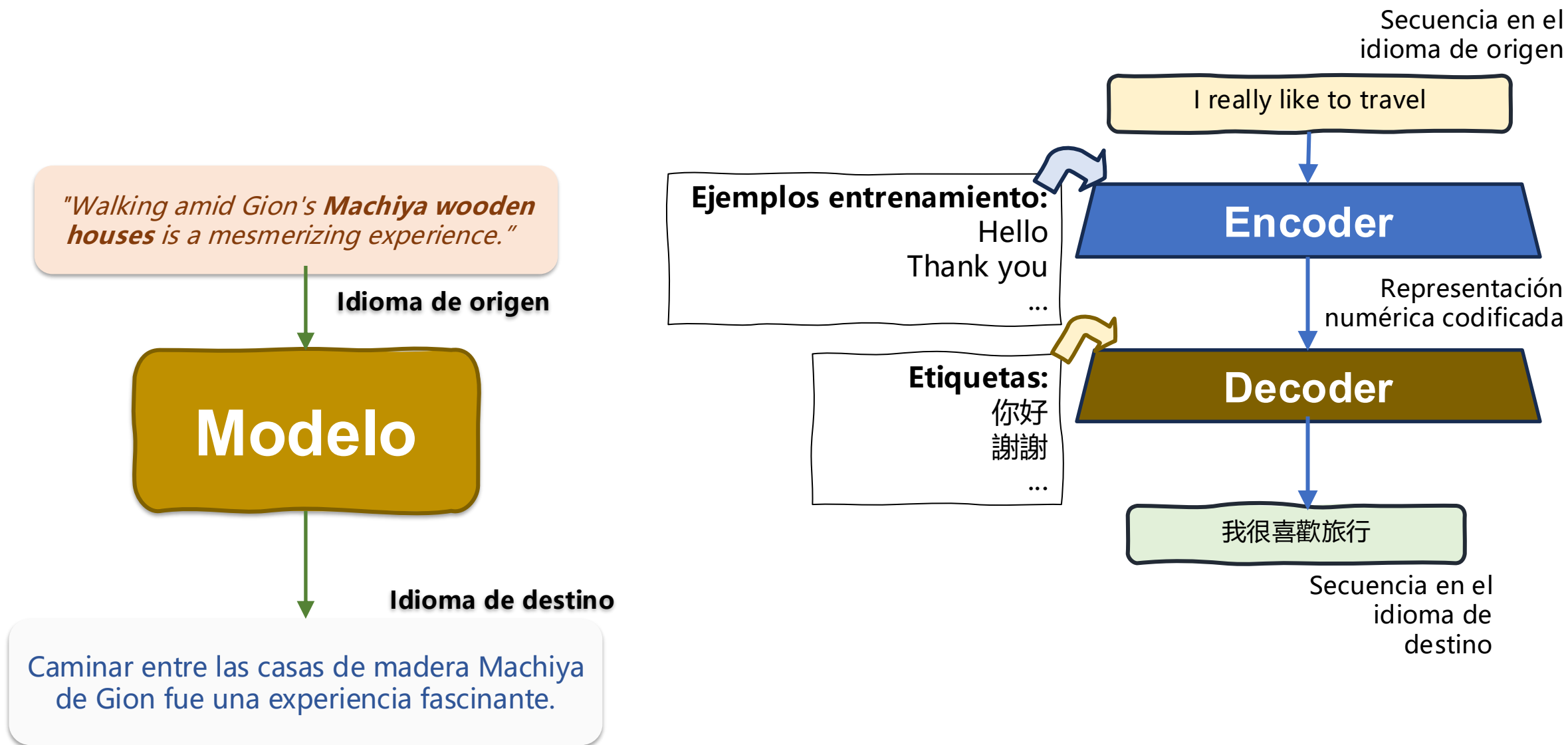
Inferir el contexto e información clave de un texto para crear una versión resumida del mismo, preservando la información y semántica principales

Sequence to sequence (seq2seq) problem: la entrada y la salida son ambas secuencias de texto, de diferente longitud.

- Se resuelven con arquitecturas especializadas de deep learning que consisten en dos grandes bloques: encoder y decoder → *LSTM, GRU, Transformers*

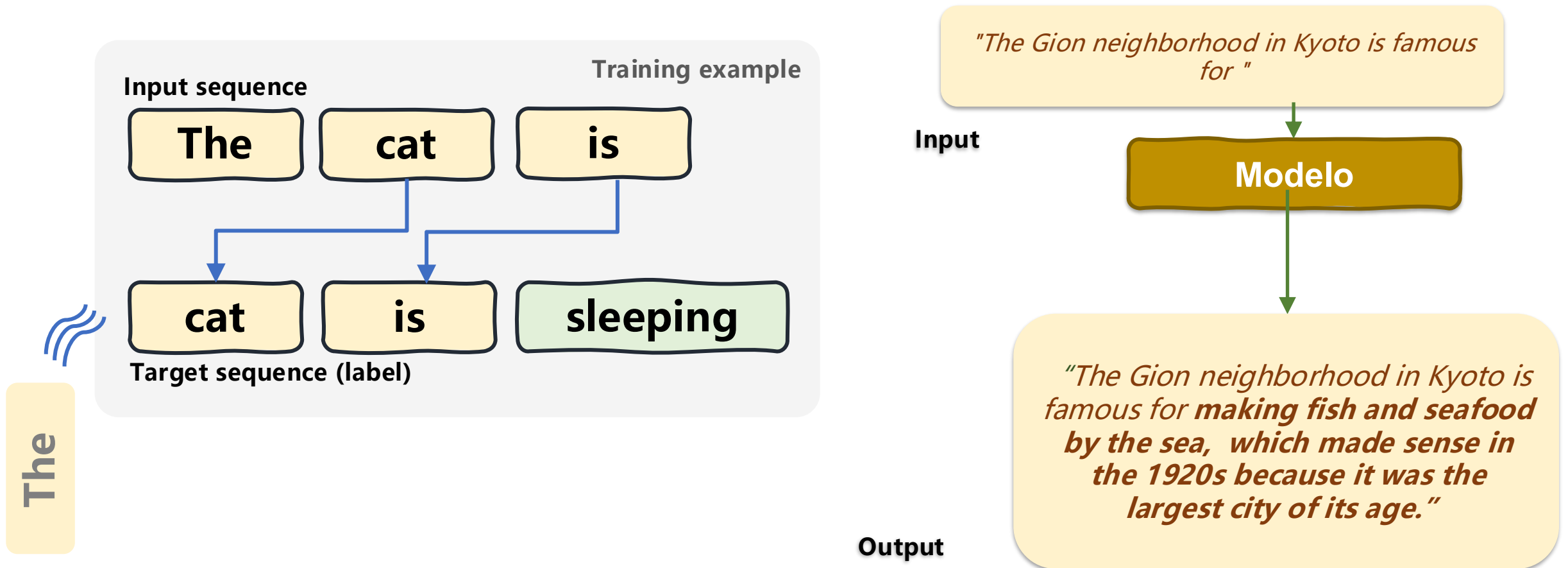


Traducción automática



Generación de texto

Generación de secuencia a partir de una entrada (*prompt*), generando iterativamente la palabra más “probable” de ser la siguiente en la respuesta (entre miles de palabras)

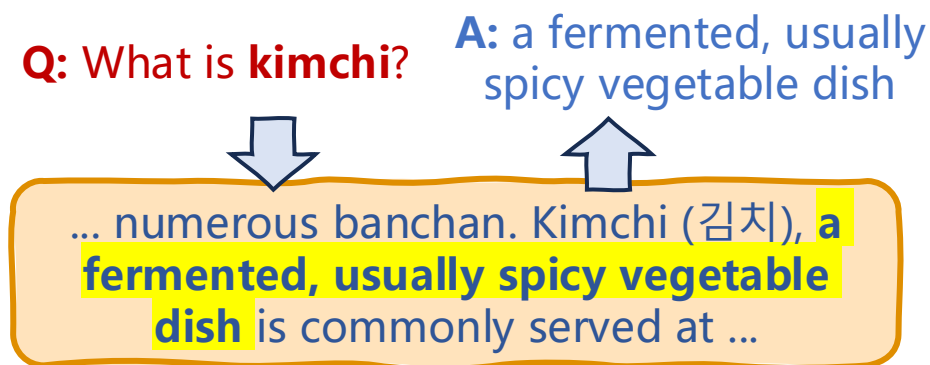
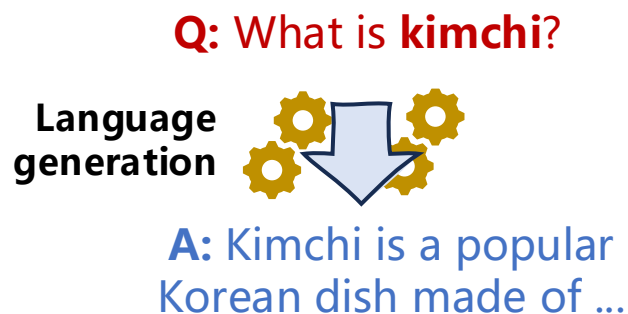


Respuestas a Preguntas (Question-Answering)

QA es una tarea que puede ser resuelta de forma extractiva o generativa

Dos entradas:

- **Contexto**: el texto sobre el que se formula la pregunta
- La propia **pregunta**

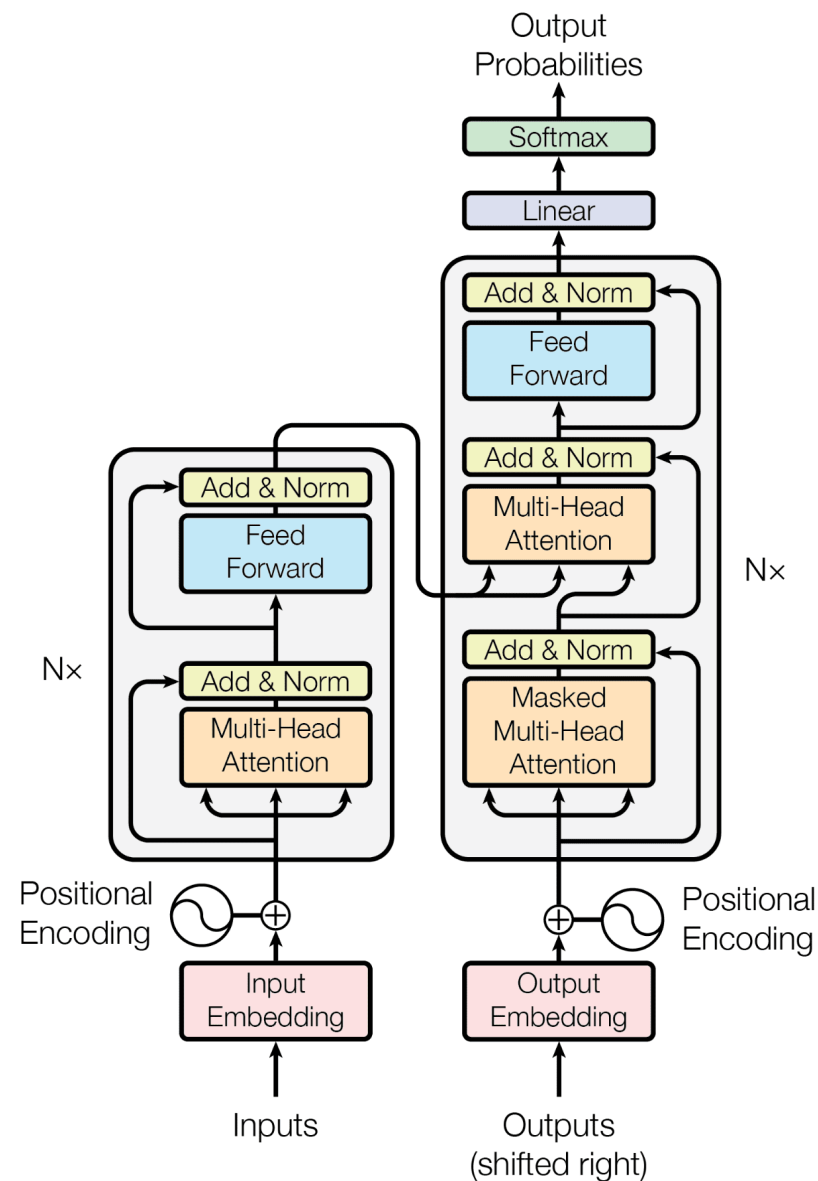


Transformers

Nacida en 2017, esta revolucionaria arquitectura de deep learning supuso una drástica mejora en el mundo del NLP gracias a sus mecanismos de atención, usando auto-atención o **self-attention**, eliminando la necesidad de arquitecturas recurrentes (RNNs) como las usadas hasta entonces para procesar lenguaje.

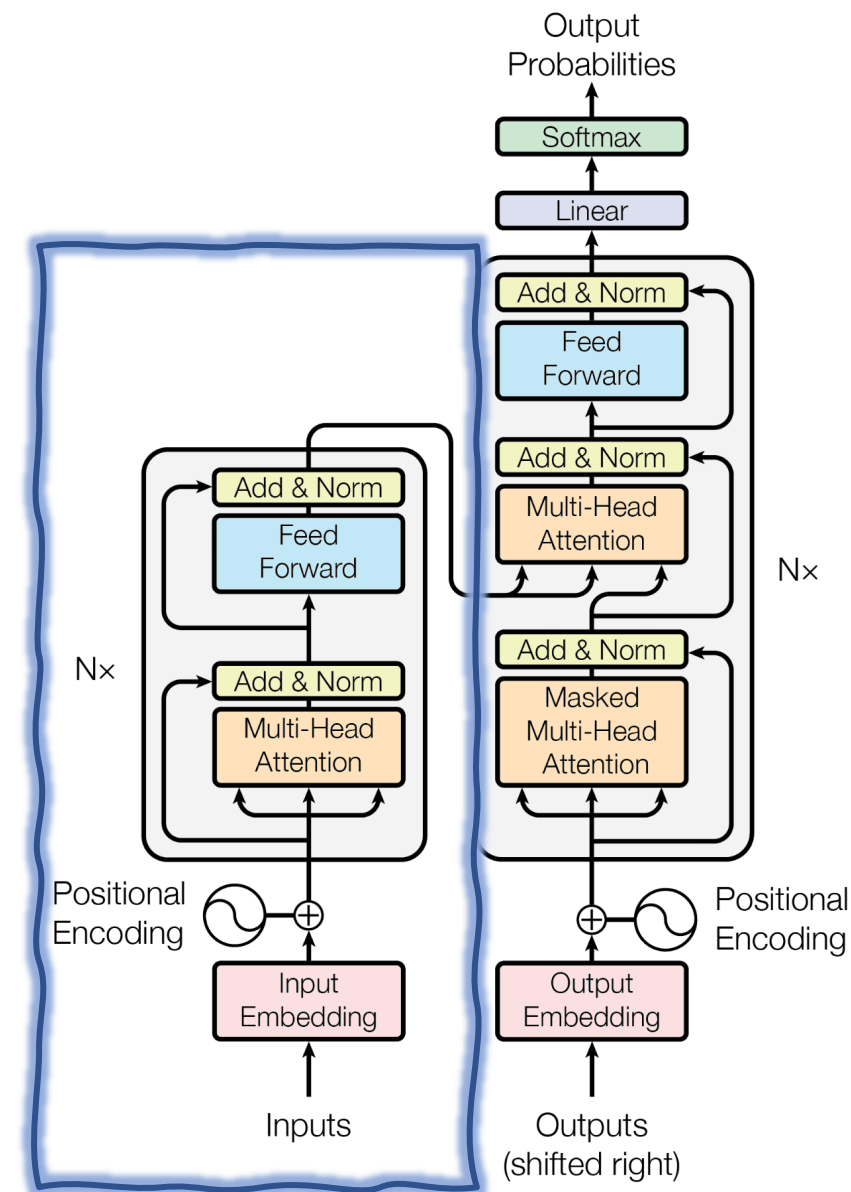
Consta de:

- Múltiples mecanismos de atención
- Conjuntos de capas (stacks) diferenciados: encoder y decoder; cada capa del encoder y decoder tiene:
 - Subcapas que alternan el mecanismo de atención con capas de neuronas secuenciales (feed forward).



Transformers

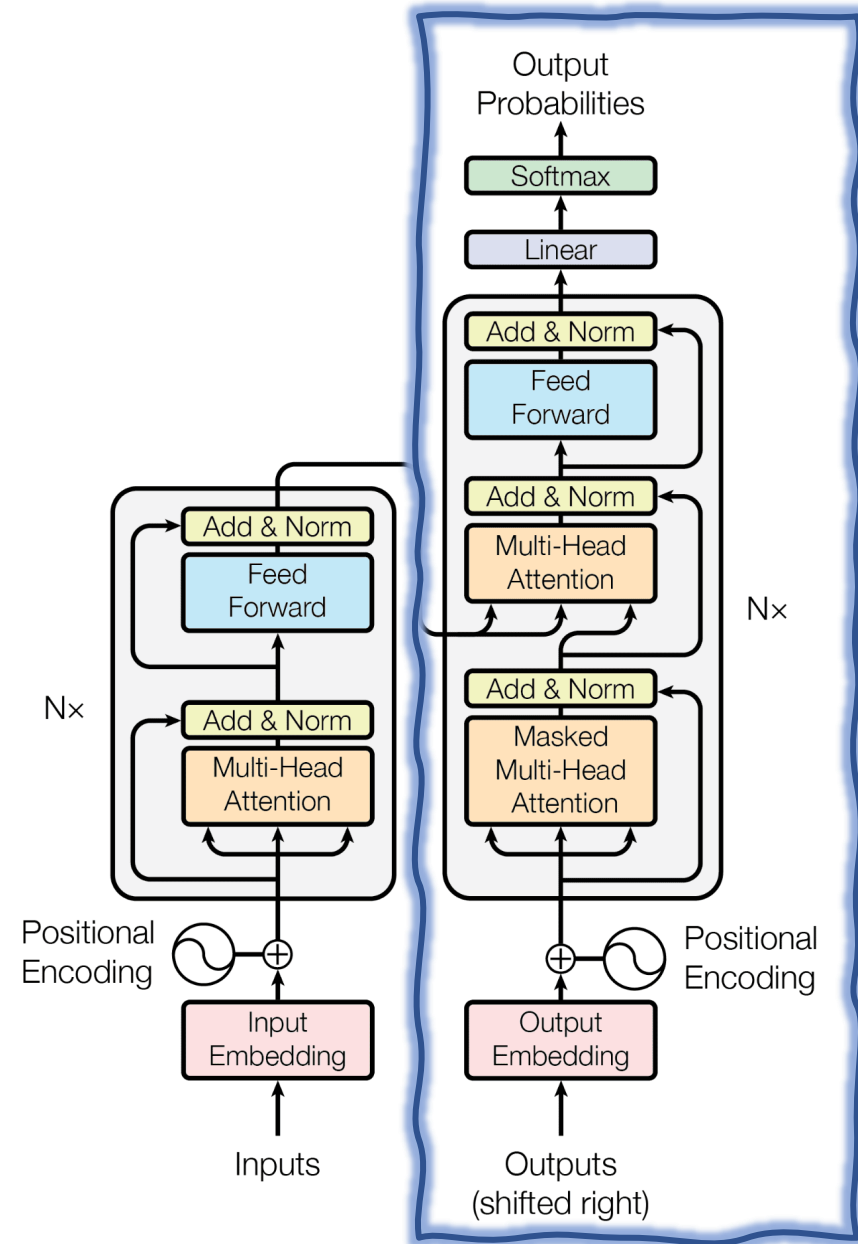
Encoder: gradualmente transforma la secuencia de entrada en una “representación oculta” (**hidden state representations**) que capturan el significado de la secuencia, relaciones y patrones lingüísticos de la misma con la mayor precisión posible.



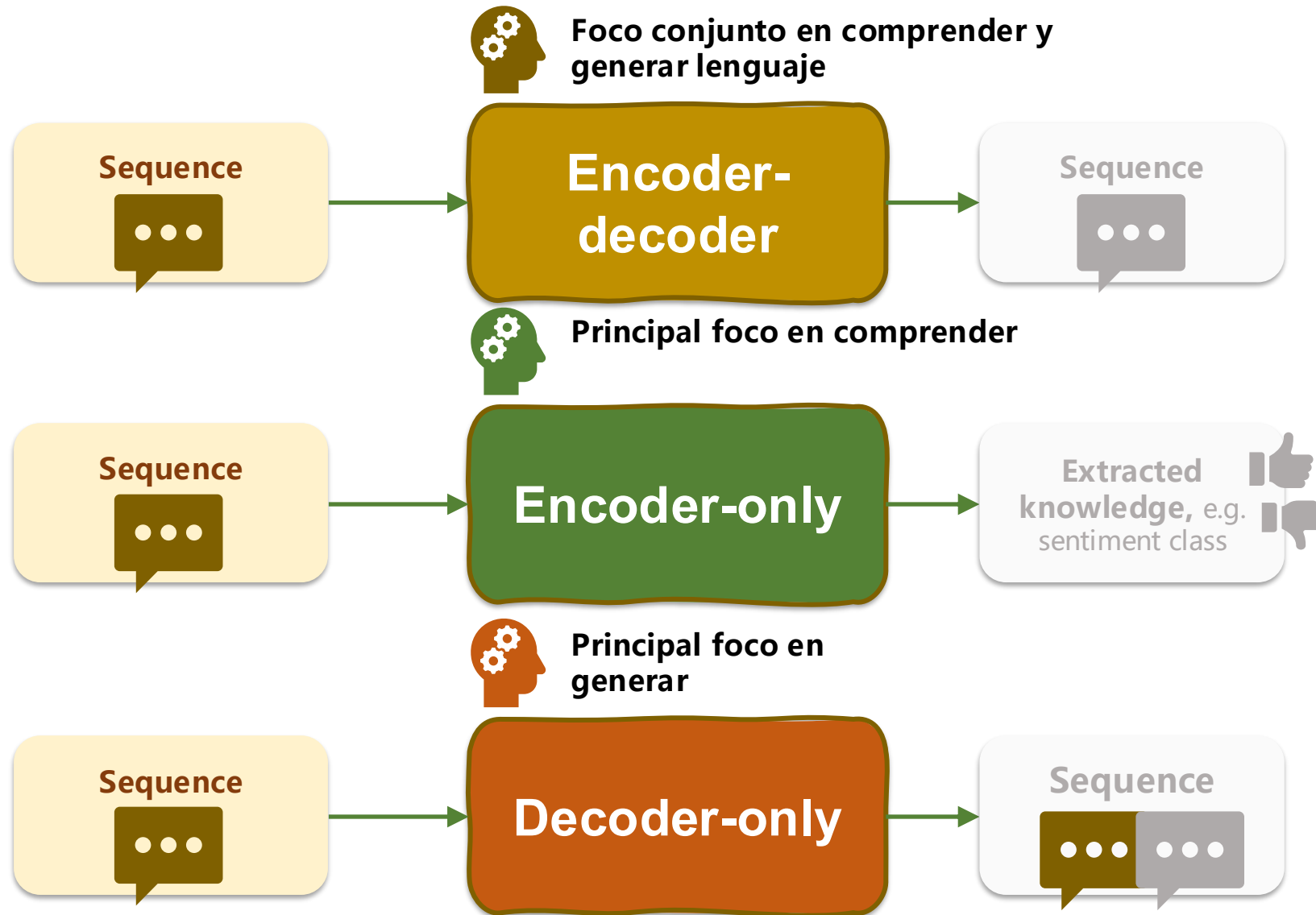
Transformers

Decoder: toma las representaciones aprendidas por el encoder además de la salida generada hasta el momento para aprender a generar de forma acertada el siguiente token de la secuencia de respuesta.

Suele ir seguido de una capa final de neuronas con activación softmax → devuelve las probabilidades de la siguiente palabra/token a generar



Tipos de arquitectura de Transformer



Modelos de lenguaje usados localmente: Ollama

Ollama es una herramienta de código abierto para la ejecución de **LLMs de forma local**, en tu propia máquina, sin depender de modelos en la nube y apps comerciales.

- **Privacidad:** los datos no salen de tu equipo.
- **Coste cero:** sin suscripciones ni pagos como cuota de uso (APIs)
- **Simplicidad:** se empaqueta el modelo, sus pesos y configuración en un mismo lugar

Modelos populares soportados por Ollama
Llama 3, Mistral, Gemma, Phi-3, etc.



Modelos de lenguaje usados localmente: Ollama

Ollama en sí es muy ligero y fácil de instalar en múltiples plataformas. Lo que "pesará más" serán determinados modelos que decidas instalarte en tu máquina para uso local.

- **Windows y macOS:** descargar instalador oficial de *ollama.com*

- **Linux:** mediante comando de terminal

```
curl -fsSL https://ollama.com/install.sh | sh
```

Comandos Básicos de Ollama (desde la terminal de tu equipo)

ollama pull [modelo] → descargar o actualizar un modelo

ollama list → lista de modelos instalados localmente

ollama run [modelo] → ejecuta un modelo para chat local.

Descargarlo si aún no está localmente instalado

ollama rm [modelo] → elimina modelo local y libera espacio en disco

ollama help → listado de comandos disponibles



Arquitecturas RAG: Introducción

¿Por qué necesitamos RAG (Retrieval Augmented Generation)?

- Los modelos de lenguaje grandes (LLMs) generan texto a partir de los datos sobre los que fueron entrenados.
- Por muchos y muy actuales que fueran estos datos en el momento de entrenarlos, pasado un tiempo, ya no tienen acceso directo a información totalmente actualizada, y esto es crítico en un sistema como son los LLMs.

Problemas:

- Información obsoleta.
- **Alucinaciones**: el modelo puede inventarse respuestas que parezcan plausibles pero no veraces, lo cual es peligroso en casos de uso críticos.
- Alto costo de re-entrenar el modelo con frecuencia para mantener actualizada su “base de conocimiento”.

¿SOLUCIÓN? ¡QUE EXISTA UNA BASE DE CONOCIMIENTO EXTERNA AL MODELO!

Arquitecturas RAG: Introducción

¿Qué nos aporta una arquitectura RAG?

Retrieval Augmented Generation (RAG) combina dos grandes elementos

1. **Recuperación de información external (Retrieval)** → como los motores de búsqueda tradicionales o las Vector DBs: bases de datos vectoriales con representaciones de textos (embeddings).
2. **Generación de texto/respuestas con LLMs** que usan la información recuperada como contexto que enriquecerá el *prompt* del usuario.

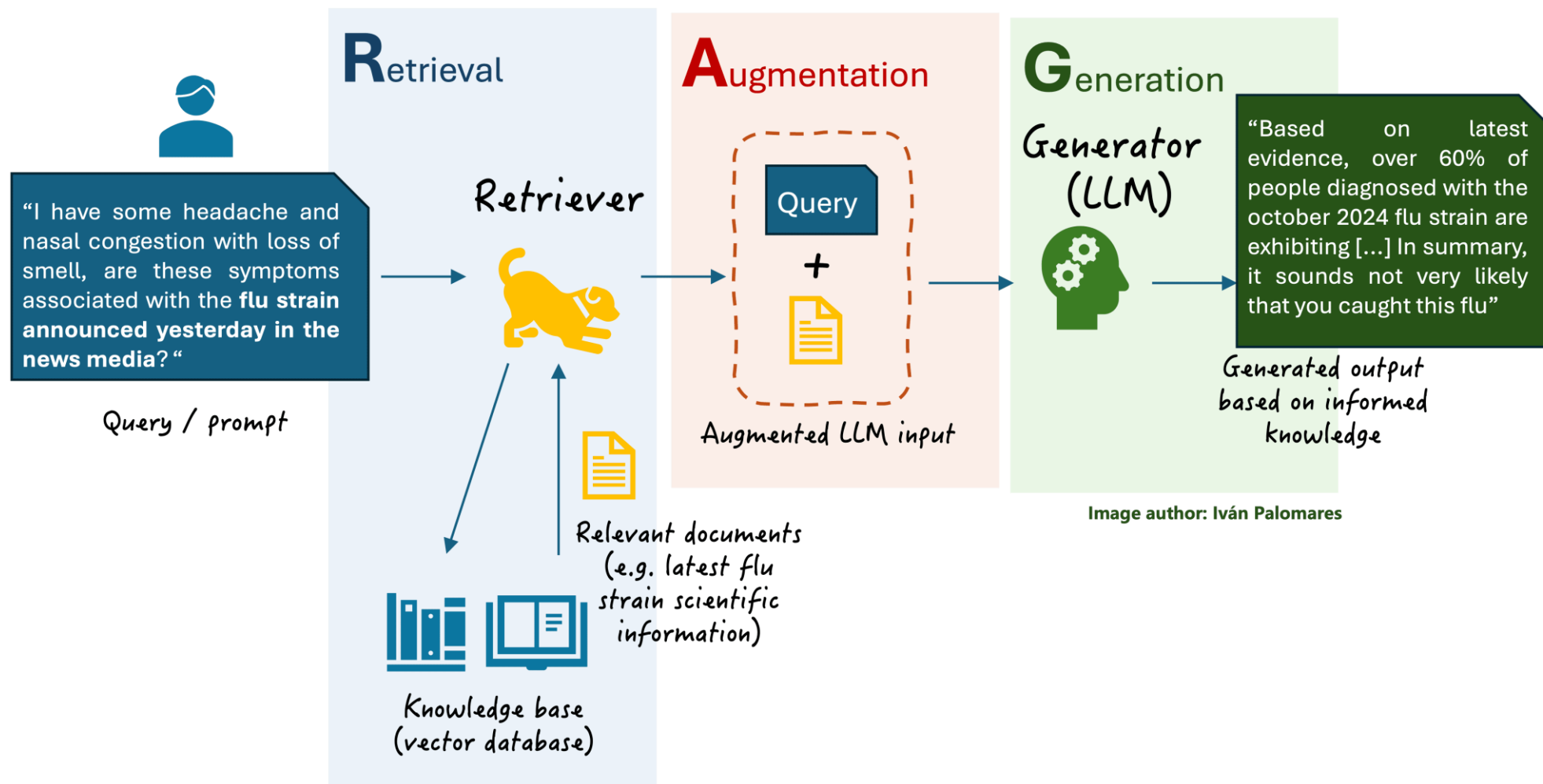


Beneficios principales del RAG

- ✓ Respuestas más precisas y fundamentadas
 - ✓ Conocimiento actualizado sin reentrenar el modelo
 - ✓ Menos alucinaciones, porque se basa en datos reales
- = Mejor **confianza y utilidad en aplicaciones reales**

Arquitecturas RAG: Introducción

¿Cómo es la arquitectura básica de un sistema RAG?



Arquitecturas RAG: Introducción

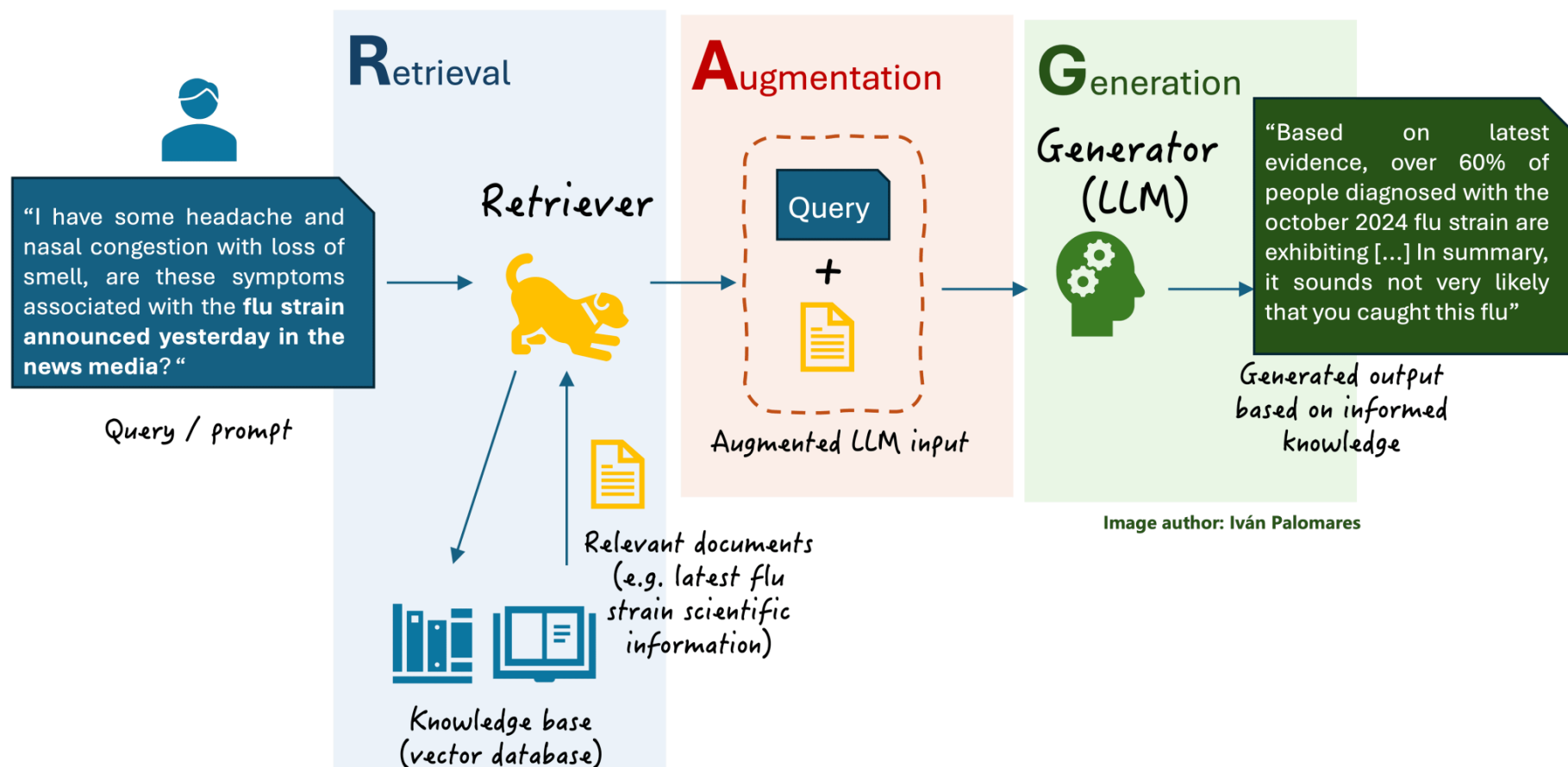
¿Cómo es la arquitectura básica de un sistema RAG?

- **LLM (Generator)**
 - Modelo de lenguaje que genera respuestas, recibiendo un texto de entrada.
- **Vector database o knowledge base**
 - Almacena *embeddings* de documentos o textos.
- **Recuperador / Retriever**
 - Se encarga de encontrar y extraer los textos más relevantes de la base de conocimiento, dada una consulta (*query*)
 - Usa una búsqueda basada en similitud entre vectores: *coseno*, *nearest neighbor*, *etc.*

Arquitecturas RAG: Introducción

Flujo Típico (3 Etapas)

1. **Retrieval (Recuperación):** la consulta del usuario se transforma en vector y se usa para buscar textos relevantes en la vector DB.
2. **Augmentation (Aumento) del prompt:** Los fragmentos relevantes recuperados se **anexan/contextualizan** con la consulta original.
3. **Generation (Generación):** El LLM recibe la consulta aumentada y genera el texto final, aprovechando contexto actualizado



Arquitecturas RAG: Introducción

¿Qué hace tan potente a esta arquitectura RAG?

- **Recuperación semántica:** la búsqueda que realiza el *retriever* se basa en significado -a nivel de embeddings de textos- no en palabras exactas.
- **Conocimiento externo desacoplado del LLM:** buena parte de la información reside fuera del LLM. Actualizar la base de conocimiento es mucho más sencillo que reentrenar el modelo.
- **Menos alucinaciones:** las respuestas del LLM se fundamentan en documentos reales en gran medida.

RAG Clásico = LLM + búsqueda inteligente + contexto relevante

Arquitecturas RAG: Introducción

Fusión en la recuperación (fusion retrieval) en RAG

- Fusion Retrieval es una mejora respecto al RAG clásico enfocado en la etapa de recuperación.
- Combina información de múltiples documentos de forma “inteligente”, no como una simple concatenación.
- El objetivo es mejorar la relevancia y coherencia del contexto construido para el LLM.

Fusion retrieval en la fase de recuperación...

- Reordena, filtra, y combina múltiples documentos para construir un contexto más útil.
- Usa una variedad de técnicas: reranking, agregación, y ensemble de búsquedas.

Pero también puede implementarse en la fase de generación...

- El LLM procesa cada documento recuperado por separado, y fusiona la información en la generación final.

Arquitecturas RAG: Introducción

Re-ranking en RAG

- Toma los documentos recuperados y lo reordena por su relevancia real (según el escenario concreto) antes de emplearlos como contexto.
- Así el LLM recibe un mejor contexto para generar respuestas más precisas y enfocadas.

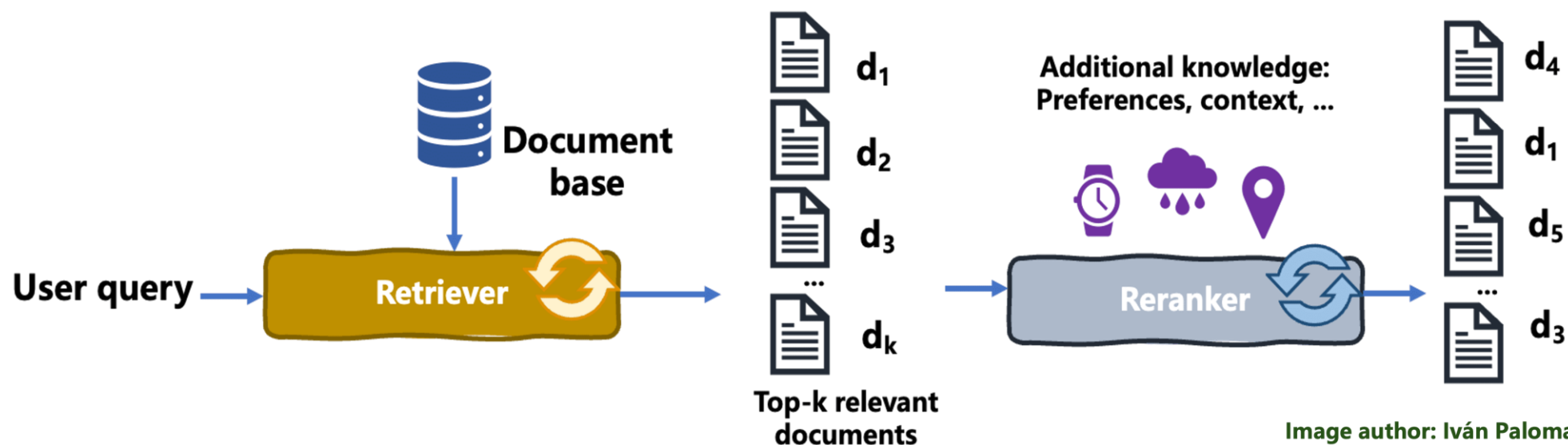


Image author: Iván Palomares

Arquitecturas RAG: Evaluación de sistemas RAG

Evaluar un sistema RAG no es solo evaluar la respuesta, sino también el contexto recuperado

Tres preguntas clave:

1. **¿Se ha recuperado la información correcta?** → calidad del retriever
2. **¿La respuesta usa el contexto construido a partir de la información recuperada?** → Faithful / grounded
3. **¿La respuesta es útil y relevante para resolver la pregunta del usuario?** → calidad LLM

RAGAs (Retrieval Augmented Generation Assessment)

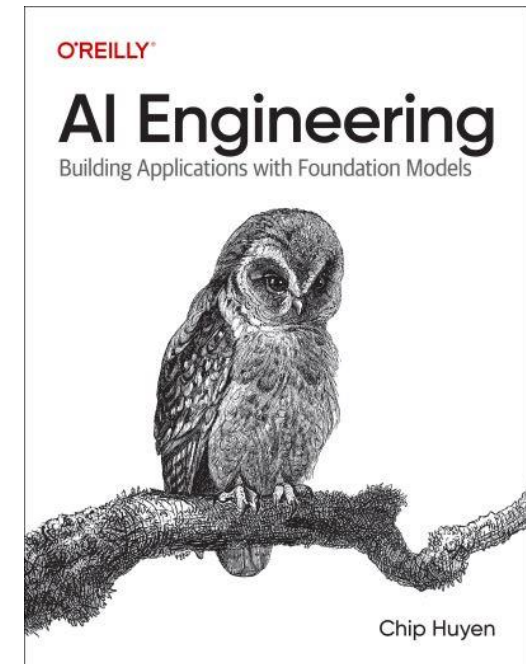
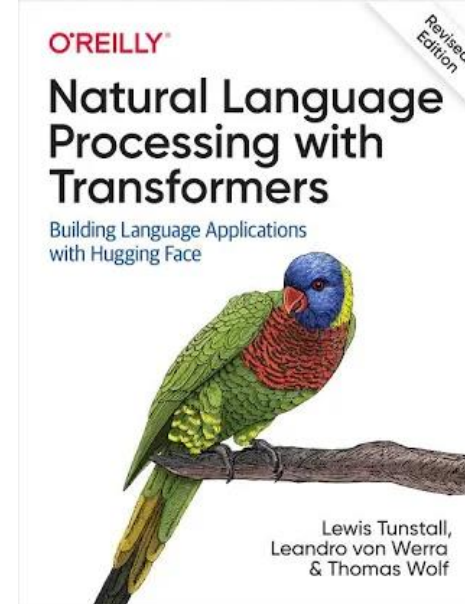
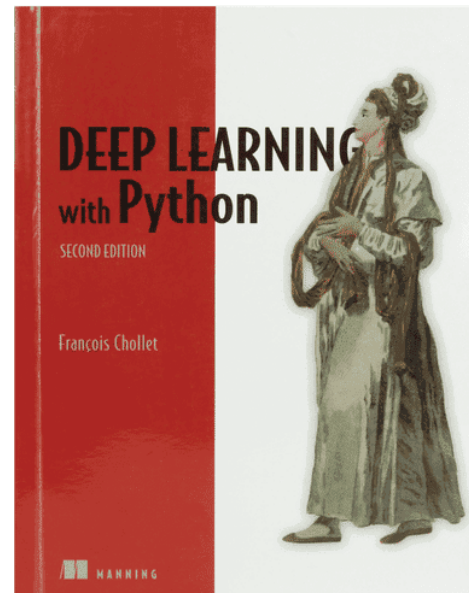
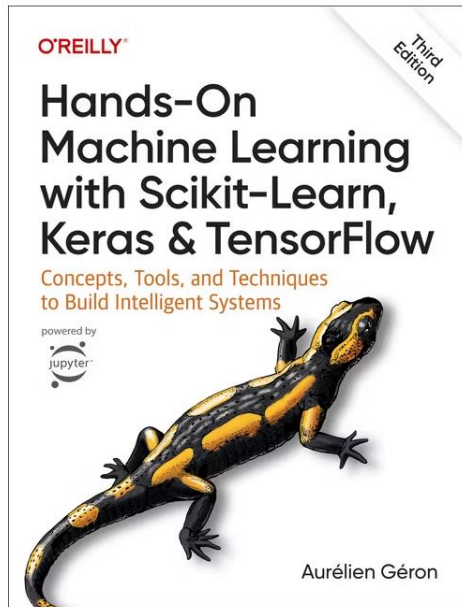
Marco ampliamente aceptado y *toolkit*, basado en métricas automáticas de LLMs, para evaluar:

1. **Recuperación** → Context recall, context precision
2. **Uso del contexto** → faithfulness, context relevancy
3. **Calidad de la respuesta** → Answer relevancy, answer correctness (requiere “referencias” = etiquetas)



Bibliografía

- **Géron, A.** (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- **Chollet, F.** (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
- **Tunstall, L., von Werra, L., & Wolf, T.** (2022). *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. O'Reilly Media.
- **Huyen, C.** (2024). *AI Engineering: Building Applications with Foundation Models*. O'Reilly Media.



Bibliografía - blogs

- Machine Learning Mastery Blog: <https://machinelearningmastery.com/blog/>
- Kdnuggets blog: <https://www.kdnuggets.com/>
- Statology blog: <https://www.statology.org/>
- Hugging Face blog: huggingface.co/blog
- Distill.pub: <https://distill.pub/>